Laminate: Succinct SIMD-Friendly Verifiable FHE

Kabir Peshawaria, Boston University
Zeyu Liu, Yale University
Ben Fisch, Yale University
Eran Tromer, Boston University

April 10, 2026

Abstract

In outsourcing computation to untrusted servers, one can cryptographically ensure privacy using Fully Homomorphic Encryption (FHE) or ensure integrity using Verifiable Computation (VC) such as SNARK proofs. While each is practical for some applications in isolation, efficiently composing FHE and VC into *Verifiable Computing on Encrypted Data (VCoED)* remains an open problem.

We introduce Laminate, the first practical method for adding integrity to BGV-style FHE, thereby achieving VCoED. Our approach combines the blind interactive proof framework with a tailored variant of the GKR proof system that avoids committing to intermediate computation states. We further introduce variants employing transcript packing and folding techniques. The resulting encrypted proofs are concretely succinct: 130 kB, compared to 1 TB in prior work, to evaluate a batch of $B = 2^{14}$ instances of size $n = 2^{20}$ and depth $d = 32$. Asymptotically, the proof size and verifier work is $O(d \log(Bn))$, compared to $\Omega(BN \log n)$ in prior work (for ring dimension N).

Unlike prior schemes, Laminate utilizes the full SIMD capabilities of FHE for both the payload circuit evaluation and proof generation; adds only constant multiplicative depth on top of payload evaluation while performing $\tilde{O}(n)$ FHE operations; eliminates the need for witness reduction; and is field-agnostic. The resulting cost of adding integrity to FHE, compared to assuming honest evaluation, is $\sim 5\times$ to $\sim 67\times$ overhead (for circuits of size 2^{20}), which is $> 2300\times$ faster than the state of the art.

Contents

1 Introduction

Delegation of expensive computation to cloud services is common nowadays, and raises privacy issues when clients' data is private or sensitive. Fully homomorphic encryption (FHE) [45] is a powerful cryptographic tool for *private* delegation of computation: it enables a server to perform arbitrary computations directly on encrypted data, without learning the underlying plaintext. Over the past decade, a series of breakthroughs [17, 35, 28, 25, 16, 15, 36] improved the efficiency of FHE, making it increasingly practical for some classes of privacy-sensitive delegation applications, including machine learning [55, 51, 56, 52, 32], secure data processing [44], private information retrieval [3, 69, 39, 87, 50], private set intersection [23, 22, 29], oblivious message retrieval [62, 63, 64, 57, 61], and single secret leader election [13, 82].

Despite its power, FHE inherently lacks *integrity guarantees* for computation: it is sound only if the server is honest. A malicious server can arbitrarily alter the computation results by evaluating a circuit of their choice instead of the intended one. Even though the server cannot directly observe the decrypted output, it can for example *choose* that output to be a maliciously-chosen value that harms the client, or some function of the secret inputs that would cause observable client behavior from which the secret can be deduced. To address this, Gennaro, Gentry, and Parno [44] introduced the concept of *Verifiable Computation over Encrypted Data (VCoED)*. VCoED enables computation delegation to an untrusted server while (1) preserving data confidentiality and (2) allowing the client to verify the correctness of the computation. However, achieving efficient VCoED remains an open problem for over a decade.

One approach to realizing VCoED is using cryptographic proofs of FHE evaluation. Specifically, SNARKs (Succinct Non-interactive Arguments of Knowledge), allow the server to produce a succinct proof that its output is indeed the result of applying all the designated ciphertext evaluations correctly.[1] While powerful, this approach suffers from very high proving costs, as known SNARKs inefficiently handle statements about *non-algebraic* ciphertext operations (e.g. relinearization).

A recent line of work [42, 4, 40, 86] explores an alternative approach to VCoED: *FHE-over-SNARK*, which flips the order of composition: it uses FHE to homomorphically compute the SNARK proof in encrypted form. The server first homomorphically evaluates the payload circuit storing ciphertexts representing intermediate wires. It then homomorphically evaluates the circuit of a SNARK prover on the statement corresponding to the desired circuit, and with inputs represented by the above ciphertexts. All these values, as well as the resulting SNARK proof, remain encrypted and hidden from the server. A malicious server could still manipulate the payload circuit evaluation and send an incorrect output, but this will be detected when the client decrypts and verifies the received proof.

The FHE-over-SNARK approach appears to yield much better performance than providing SNARK proofs of the whole homomorphic evaluation. Intuitively, this is because one pays the combined overhead of FHE and SNARK not for computing the payload circuit, but rather, only when homomorphically evaluating the SNARK prover applied to the wires of that circuit. Unlike the payload circuit, the SNARK can be chosen to be very structured and particularly amenable to FHE evaluation. However, substantial challenges remain, as explained next.

1.1 Challenges in FHE-over-SNARK VCoED

While the FHE-over-SNARK paradigm is promising, current realizations [4, 40, 86] remain impractical. They impose expensive overheads on the server and require a high bandwidth client capable of processing large proofs.

[1] SNARG (Succinct Non-Interactive Argument) suffices, as knowledge-soundness is not needed here.

1.1.1 Challenges for Server Efficiency

Inherent challenges stem from the setting of BFV/BGV homomorphic evaluation: multiplicative depth is very costly, the native algebraic structure is constrained, and performance relies on utilizing a very restrictive form of parallelism. Key examples follow.

Witness reduction overheads. An often-overlooked cost in SNARK-based verifiable computation is the witness reduction. After evaluating the desired payload computation and storing all the intermediate values, there can be an additional cost to transform these to the representation that the SNARK prover can handle (sometimes referred to as the *extended witness*).

A prominent case of such overhead occurs due to *mismatched fields and non-native arithmetic*. For instance, a payload computation consisting of a single $\mathbb{F}_q$ addition would intuitively require a proof for a single addition. However, if the SNARK prover operates over a different field $\mathbb{F}_p$ (with $p \neq q$), this is not the case: $x_1 + x_2 \equiv x_3 \pmod{p}$ does *not* imply that $x_1 + x_2 \equiv x_3 \pmod{q}$. Hence, the $\mathbb{F}_p$-based constraint system must emulate $\mathbb{F}_q$ arithmetic, which increases the constraint count and requires many additional witness variables to be homomorphically computed.

This problem is particularly severe in the FHE setting, where non-native arithmetic often requires operations like modular inverse, which are prohibitively expensive in terms of multiplicative depth. For example, computing $x^{-1} \pmod{q}$ via $x^{q-2} \pmod{q}$, via Fermat's Little Theorem, requires $\Theta(\log q)$ iterated modular multiplications, adding significant multiplicative depth to the homomorphic evaluation before even starting the prover evaluation.

Prior FHE-over-SNARK schemes largely overlook this challenge. For instance, [86] fixes the SNARK's base field at 50 bits. This is a poor match for many BFV/BGV applications [16, 15, 36], which use much smaller plaintext fields (e.g., < 20 bits in [62, 64, 39]) for efficiency. Forcing a 50-bit field can slow down the underlying FHE computation by over 20×.[2] Similarly, [40] employs generalized BFV/BGV (GBFV/GBGV) instead of standard variants, but this workaround comes at a high cost: GBFV provides 16× fewer plaintext slots than BFV, reducing evaluation throughput by the same factor.

Inefficient proof generation due to polynomial commitments. Prior works' server efficiency is bottlenecked by reliance on polynomial commitments (PCs) for the extended witness, which contains all intermediate values that arose during payload computation. Concretely, [40, 86] use hashing-based PCs, which are typically computed using the Number Theoretic Transform (NTT). Though an NTT over a length-n vector can be done in $O(n \log n)$ time, this can only be achieved at the cost of $\Omega(\log n)$ multiplicative depth. To achieve multiplicative depth $\leq t$, NTT requires $\Omega(t \cdot n^{1+1/t})$ operations.

For an n-gate payload circuit, the Fractal [27] proof system used in [40] requires an NTT over a length n vector. For [86], the situation is more complicated (see Section 1.1.2), but the NTTs that arise are still large enough to preclude a prover circuit that runs in $\tilde{O}(n)$ with constant multiplicative depth. High multiplicative depth in the prover circuit inflates the noise budget required for the *entire* FHE evaluation, including the original payload circuit. That is, a payload circuit of depth d would now require the FHE scheme to support a total depth of $d + \omega(1)$ by reserving sufficient noise or bootstrapping $\omega(1)$ times. FHE performance is highly sensitive to this total depth parameter.

Loss of FHE parallelism. Prior schemes [40, 86] use BFV/BGV encryption, which offers parallelism via SIMD (Single Instruction, Multiple Data) computation that concurrently operates on

[2] For example, in [62, 64], the multiplicative depth is $\sim \log p$ for a plaintext space $\mathbb{Z}_p$. Consequently, increasing the plaintext modulus from a ∼20-bit field to a 50-bit field raises the required depth from ∼20 to ∼50. Moreover, a larger plaintext space incurs higher noise growth per level, increasing the total noise budget from approximately $\sim(20 \times 30)$ to $\sim(50 \times 60)$. This alone results in a runtime overhead of at least ∼5×. In addition, maintaining the same security level requires increasing the ring dimension by $\geq 4\times$, implying that the overall runtime overhead exceeds 20×.

N "slots", typically with $N \approx 2^{14}$ or larger. However, all this capacity for parallel computation and storage is allocated to the prover circuit, and specifically to accelerating the expensive polynomial commitment schemes. The actual payload computation is performed in a single slot, without parallelism. Conversely, virtually all practical applications using BFV/BGV FHE (e.g., [62, 39, 23, 22, 29]) rely on SIMD parallelism being fully available and utilized by the payload computation. Reducing these to "single-threaded" execution (by using a single slot for the payload circuit) immediately reduces the system throughput by a factor of tens of thousands, making the total cost of adding integrity exorbitantly expensive even accounting for the additional costs of witness reduction and evaluation of the prover circuit.

1.1.2 Challenges for Client Efficiency

Ideally, a VCoED construction minimizes communication and client verification time. In an attempt to improve server efficiency, current FHE-over-SNARK schemes sacrifice this desideratum.

FHE-friendly polynomial commitments imply larger proofs. Previous attempts to realize VCoED through FHE-over-SNARK were built from proof systems based on the Polynomial IOP (PIOP) framework [18]. These proof systems require committing to a polynomial whose total degree is linear in the size of the extended witness. As discussed above, the requisite polynomial commitments do not admit both a constant multiplicative depth and quasi-linear time prover. To accelerate these commitments, prior works heavily exploit SIMD techniques, i.e. the fact that one BGV/BFV operation performs N operations on the underlying plaintext space (where N is the BGV/BFV ring dimension).

Blind Fractal server [40] has $\Omega(N \log n)$ size proofs and performs $\Omega(tn^{1+\frac{1}{t}}N^{-1})$ operations to compute an NTT on an extended witness vector of length n in multiplicative depth t. Notably, this server only runs in quasi-linear time in a parameter regime where $N = \tilde{\Omega}(n^{1/t})$. Thus, to homomorphically evaluate a Fractal prover circuit in $\tilde{O}(n)$ FHE operations and multiplicative depth $\leq t$, proof size and verifier time is at least $\tilde{\Omega}(n^{1/t})$.

Phalanx [86] improves the server efficiency, but with worse asymptotic implications to its client. It uses the Ligero/Brakedown PCS [2, 47],[3] which rather than performing one NTT of a length n vector, performs $\sqrt{n}$ NTTs over length $\sqrt{n}$ vectors.[4] The consequence is that the [86] server can homomorphically evaluate its polynomial commitment in multiplicative depth t with $\Omega(tn^{1+\frac{1}{2t}}N^{-1})$ FHE operations. However, these commitments require opening proofs of size $\Omega(n^{0.5})$. The proof size and client verification time of [86] are at least $\Omega(N \log n + \sqrt{n})$.

Concretely impractical proof sizes and verification time. Prior works such as [40, 86] discuss parameter regimes that make homomorphically evaluating the SNARK proof more tractable. However, the highlighted parameters are too server-friendly, coming at great cost to the client.

Concretely, [40, 86] benchmark their schemes for payload circuits with $n = 2^{20}$ gates using $N = 2^{14}$ SIMD slots. For this choice, [86] yields proofs of 58 MB, while [40] proof sizes are 116 MB (for a single instance). However, simply writing down all 2^{20} wire assignments requires only 17 MB (for a plaintext modulus $q \leq 2^{128}$). The client would be more memory-efficient, *and faster*, if it just performed the payload computation locally without delegation.

[3] Specifically, the Phalanx server [86] homomorphically evaluates the proof system defined by the Spartan [74] PIOP, compiled with a PCS that is implicit in Ligero [2] and explicitly given in Brakedown [47].

[4] More generally, the Brakedown [47] PCS constructs an $m \times n/m$ matrix and performs an NTT on each n/m sized row. An opening proof consists of m field elements. Phalanx [86] does not explicitly state the chosen matrix dimensions, but the setting $m = \sqrt{n}$ is implied by Section 5.1 stating that the depth-t commitment requires $O(tn^{1+\frac{1}{2t}}) = O(n^{0.5} \cdot tn^{0.5(1+1/t)})$ operations. Also, this PCS can be generalized beyond a matrix, to a tensor of dimension up to $\log n$ [14]; this was not explored by [86].

Thus, in this work, we ask the following question:

Can we build an FHE-over-SNARK construction that:
(1) has a quasi-linear prover circuit with constant multiplicative depth;
(2) achieves succinct proof size and verification time (polylogarithmic in payload circuit size);
(3) natively supports SIMD amortization; and (4) is field-agnostic?

We answer affirmatively by showing such a construction and showing its concrete efficiency.

1.2 Our Contribution

We introduce Laminate, a family of three protocols for Verifiable Computation over Encrypted Data that follow the FHE-over-SNARK paradigm. Laminate protocols perform fully-parallel BFV/BGV homomorphic evaluation of payload circuits, combined with homomorphic evaluation of (a variant of) Goldwasser-Kalai-Rothblum [46] proofs to ensure correctness.[5]

Laminate is the first concrete VCoED scheme to achieve the following properties:

Quasi-linear server circuit with constant multiplicative depth overhead. The multiplicative depth overhead for adding integrity is asymptotically *constant*, and concretely 1, 3.5, or 4.5 levels depending on the variant of Laminate (i.e., with a tradeoff against proof size).[6] The resulting noise budget overhead is only 32–150 bits (for $<$ 20-bit plaintext field).

Furthermore, the servers performs $O(n \log n)$ FHE operations *in all parameter regimes*. This is possible because Laminate avoids committing to the extended witness.

Native SIMD amortization.

Laminate preserves the native SIMD capabilities of BFV/BGV, allowing the payload circuit to utilize all N available slots, unlike prior works. To formalize the SIMD advantage, we distinguish two classes of payload circuits.

Definition 1.1 (B-Laned Payload Circuit (Informal))**.** A payload circuit is *B-laned* if it uses only B of the N available SIMD slots, involves no cross-slot interactions, and all plaintexts encode the same scalar in every slot. Equivalently, the B active slots execute the same single-slot arithmetic circuit independently, in SIMD parallel. See Definition 4.3 for the formal definition.

Laned payload circuits are a special case of the more general class of *wide* payload circuits.

Definition 1.2 (Wide Payload Circuit (Informal))**.** A payload circuit is *wide* if it operates on all N SIMD slots of each ciphertext, and may include arbitrary cross-slot operations such as rotations. Every B-laned circuit can be viewed as a wide circuit by treating unused slots as carrying dummy values. See Definition 5.1 for the formal definition.

Prior works [86, 40] only natively support 1-laned payload circuits, and can be extended to B-laned circuits only by running the proof generation procedure B times. Table 1 compares the asymptotics of these works to several version of Laminate, and in particular to $\text{Laminate}_{\text{laned}}$ which is optimized for the case of B-laned payloads. Concretely, comparing[7] against Phalanx [86] and Blind Fractal [40] with a B-laned payload circuit of $n = 2^{20}$ gates and ring dimension $N = 2^{14}$:

[5] The name Laminate alludes to the proof generation comprising a thin layer of extra computation, with very low multiplicative depth, that is added on top of the bulk of payload computation to ensure its integrity.

[6] We count scalar-by-ciphertext as 0.5 multiplicative levels, because they consume less noise than plaintext-by-ciphertext or ciphertext-by-ciphertext multiplication. We also assume that extension field multiplication can be done by a depth-1 circuit; otherwise add at most 0.5 levels, per Remark 7.1.

[7] The full end-to-end server runtime advantage is greater than the above bounds, since prior work did not account for a few critical costs; see Section 1.4.

Protocol	Mult Depth Overhead	Server FHE Ops	Proof Size	Use Case
Laminate$_{\text{base}}$	1 Level	$O(n \log n)$	$O(Nd \log n)$	High Throughput ($B \approx N$)
Laminate$_{\text{fold}}$	3.5 Levels	$O(n \log(Bn) + d \log(Bn) \log B)$	$O(\max\{d \log(Bn), N\})$	Bandwidth Constrained ($B \approx N$)
Laminate$_{\text{laned}}$	4.5 Levels	$O(n + \frac{Bn}{N} \log n)$	$O(\max\{d \log(Bn/d), N\})$	Few instances ($B \ll N$)
Phalanx [86]	$t+3$ Levels	$\Omega(\frac{B}{N}(n \log n + tn^{\frac{2t+1}{2t}}))$	$\Omega(B\sqrt{n} + BN \log n)$	2PC (private server inputs)
Blind Fractal [40]	$3t+3$ Levels	$\Omega(\frac{B}{N}(n \log n + tn^{\frac{t+1}{t}}))$	$\Omega(BN \log n)$	ZKP Delegation

Table 1: Comparison of VCoED Protocols for B-laned payload circuits (see Definition 4.3) with n gates and d layers. Ring dimension N denotes the BGV/BFV SIMD slot capacity. Lower bounds are used for prior work since some end-to-end costs were omitted in their evaluation, as discussed in Section 1.4. The parameter t (for prior work) denotes a tunable multiplicative depth for evaluating an NTT (or inverse NTT) under FHE. For all protocols, the client's runtime is quasi-linear in the proof size.

- For 1-laned payload circuits, Laminate$_{\text{laned}}$ achieves 2× faster server runtime than prior works.
- For 2^{14}-laned payloads (full SIMD utilization), Laminate$_{\text{laned}}$ is 2385× faster.

Our remaining variants, Laminate$_{\text{base}}$ and Laminate$_{\text{fold}}$, handle the general case of wide payload circuits, including rotations and other cross-slot operations, which are not supported by prior works. Since no prior scheme supports wide payloads, we benchmark these variants against honest payload evaluation (with no integrity proof) in Section 1.4.

Succinct proof sizes and verifier efficiency. Laminate$_{\text{fold}}$, our general case variant optimized for the client, is succinct, e.g. both proof size and verifier runtime are $O(N+d \log(Nn))$. Moreover, this is achieved while keeping the server circuit constant multiplicative depth and quasi-linear (in FHE operations). The novel technical ingredient is a "transcript packing" technique that is described in Theorem 6.1. No prior work achieved all of the above properties; see a comparison in Table 1.[8]

For the concrete common parameter set considered by prior work (i.e. $n = 2^{20}$ gates, $N = 2^{14}$ slots), Laminate$_{\text{laned}}$ produces proofs that are 446× smaller than Phalanx [86] for 1-laned payloads, and 7,322,384× smaller for 2^{14}-laned payloads (full SIMD utilization). In several cases, the proof includes just *one ciphertext* (along with other auxiliary components, namely hash digests and a SNARK proof).

Field-agnostic. Laminate protocols are agnostic of the plaintext field, thereby avoiding additional costs that come with forcing the delegated computation to be defined over a fixed large field and emulating non-native arithmetic.

Conceptual simplicity. The protocol separates logically into two phases: the payload evaluation and the proof evaluation. For Laminate$_{\text{base}}$ and Laminate$_{\text{fold}}$ (wide payloads), the proof generation accepts the intermediate ciphertexts of the payload evaluation *as is*, removing the need for witness reduction or extended witness formatting. For Laminate$_{\text{laned}}$ (laned payloads with $B < N$), a lightweight witness packing step packs the intermediate ciphertexts before proof generation, but this cost is minor, and dominated by payload computation. In its simplest form (Laminate$_{\text{base}}$), the protocol relies solely on a leveled homomorphic evaluation of the multivariate sum-check protocol, avoiding the complexity of Polynomial Interactive Oracle Proofs (PIOPs).

1.3 Technical Overview

Blind Interactive Proofs. We first recap the *blind interactive proof* framework from [40], which is essential to understanding our design choices. In a public-coin IP, the prover sends messages to

[8] Laminate requires the payload circuit to be *layered*, i.e. gates in layer i take inputs from layer $i+1$. Circuits that arise from natural computations are typically layered, or can undergo an inexpensive transformation to become layered (by adding forwarding dummy gates). For payload circuits intended to be evaluated under FHE *without bootstrapping*, Laminate variants support a gate set for which the payload circuit can be viewed as having $d = O(1)$ layers (see Section 5.2).

a verifier who replies with random challenges. Then interaction is removed via the Fiat-Shamir heuristic, i.e., replacing verifier challenges with hashes of the transcript. A key observation in [42, 4, 40] is that a VCoED scheme need not homomorphically evaluate the *entire* non-interactive prover (which includes hashing).[9] Instead, the server only needs to homomorphically evaluate the underlying interactive prover's circuit.

Fiat-Shamir verifier challenges can be computed *outside* of the FHE context; they are generated externally by hashing the ciphertexts of the running transcript. The intuition is that hashing of these ciphertexts (instead of the plaintexts they encode) remains adequately binding. Thus, this retains the temporal ordering that Fiat-Shamir enforces on the IP execution without requiring expensive homomorphic evaluation of hash functions.

Furthermore, the above implies that the verifier's challenges can be treated as *scalar inputs* to the homomorphic evaluation of the IP prover's circuit. This is a significant improvement, as it allows some of the prover's circuit to be evaluated via cheap scalar-ciphertext operations, rather than ciphertext-ciphertext operations that are costly in runtime and noise budget. Furthermore, even though the IP prover executes multiple rounds, each logically dependent on the previous ones, these rounds do not add up in terms of multiplicative depth, because the hashing of transcript ciphertexts into Fiat-Shamir challenge scalars effectively resets their FHE multiplicative level.[10] The powerful consequence is that we need only design a *public-coin IP* with an FHE-friendly prover, as compilation to a non-interactive proof system adds negligible overhead.

Core insight: GKR is the right tool. Our core insight is that the GKR public-coin IP is uniquely well-suited for homomorphic evaluation. Most other IP choices require a *polynomial commitment (PC) under FHE*, where the total degree of the polynomial is linear in the circuit size; such PCs force the server to sacrifice either constant multiplicative depth or quasi-linear runtime (from NTTs).[11] Furthermore, mitigating these costs leads to sacrificing the SIMD parallelism of payload circuit execution, as discussed in Section 1.1.

In contrast, the GKR protocol's IP prover consists of only two tasks: evaluating multilinear polynomials and computing sumcheck round polynomials.[12] Both of these tasks can be very FHE-friendly, *without* a transformation that incurs a super-logarithmic blowup to the size of the prover circuit to achieve constant multiplicative depth. Multilinear evaluation reduces to a simple linear combination, and the sumcheck prover circuit is discussed below. Computing the whole GKR proof then reduces to performing multiple rounds of this form, and as discussed above, the total multiplicative depth needed to homomorphically evaluate all rounds equals that of a *single* round.

FHE-friendly GKR sumchecks. The classic, linear-time sumcheck algorithm [31, 77] reuses work from previous rounds. This dependency gives it an $\mathcal{O}(\mu)$ multiplicative depth (where μ is the number of rounds), which is exactly the kind of super-constant depth we are trying to avoid. However, a different, quasi-linear time sumcheck algorithm from [30] offers a crucial work-depth trade-off. In this algorithm, the prover computes each of the μ round polynomials from scratch, without reusing work. While this is less efficient in the plaintext setting (total work is quasi-linear, not linear), its circuit attains an $O(1)$ multiplicative depth. The observation that *Spartan* sumcheck instances can be computed in low multiplicative depth was used in [86]. In our work, we optimize the prover circuit for *GKR* sumcheck instances (which are more complicated than those from Spartan),

[9] The observation was implicit in [42] and [4]. In defining the blind IP framework, [40] drew attention to this point.

[10] The actual homomorphic evaluation of the prover rounds remains temporally ordered by the need to compute the Fiat-Shamir challenges in sequence.

[11] One could consider using a non-NTT based PCS, such as Ligero [2] or Brakedown [47] with a linear code that admits fast, low-multiplicative depth encoding (e.g. RAA codes). However, this would result in larger proof sizes and verifier runtime, which is likely why prior works such as [86, 40] elected to not take this approach.

[12] See Section 2.6 for details on the [68] sumcheck protocol.

carefully tailoring them to be efficient for the blind prover, and attain a multiplicative depth only 1.5 or 2.5 (depending on the variant, which we now discuss).

For a d-layer circuit, the GKR protocol invokes sumcheck d times to reduce a claim about the output layer to one about input layer. Crucially, we tailor each GKR layer reduction to exploit structural properties inherent to BGV/BFV payload circuits (in particular, the data-parallel SIMD execution model) which substantially reduces both the prover's work and multiplicative depth per layer. The ith sumcheck is taken over a μ_i-variate polynomial for some $\mu_i > \log N$. In Section 5, we describe two variants of the GKR IP, which we term *base* and *folded* IPs. Letting $d_\times$ denote the multiplicative depth required to homomorphically evaluate the payload circuit, a circuit that evaluates the payload circuit and invokes the folded IP prover requires multiplicative depth $\leq d_\times + 1.5$. However, the base IP is tailored to be maximally FHE-friendly, doing so in two key ways. First, it terminates each sumcheck $\log N$ rounds early, which saves the server some FHE operations. Second, it skips the final steps performed by the folded IP prover, which can include evaluating a $\log N$-variate multilinear polynomial (which requires one additional level of multiplication). The consequence of these choices is that a circuit evaluating the payload circuit and invoking the base IP prover achieves a multiplicative depth of only $d_\times + 1.0$ at the cost of having the base IP *verifier* receiving larger proofs and finishing the computations the prover did not complete.[13]

Challenge: succinct proof sizes. In each round $i \in [r]$, our folded BhIP prover sends M_i ciphertexts to the verifier. When using a plaintext space of roughly 2^{16} targeting $\lambda = 100$ bits of security, this consists concretely of $M_i \approx 16$ ciphertexts. The folded verifier needs to learn M_i of the $M_i \cdot N$ communicated field elements (one from each ciphertext). However, we would then be sending M_i ciphertexts in every round, each of size $7N$ bytes.

For circuits with $d = 32$ layers, $n = 2^{20}$ gates, and $N = 2^{14}$ SIMD slots, there are $r \approx 1500$ total rounds, resulting in $252N$ kilobytes, or 4 GB. This is unacceptable, especially for a 1-laned payload circuit where only a single SIMD slot is active.

Solution: transcript packing. We develop a transcript packing compiler that transforms the BhIP into a designated-verifier interactive proof with dramatically reduced communication. We outline the compiler (considering only the folded BhIP for exposition), with a full treatment in Section 6.2.

Our first step is to reduce space consumption in each round. For round $i \in [r]$, let M_i denote the number of ciphertexts sent by the BhIP prover. It would be inefficient to send all M_i ciphertexts directly, as this wastes $M_i \cdot (N-1)$ slots. With a lightweight "flattening" procedure (masking, rotation, and addition), we consolidate the M_i ciphertexts into a *single* dense ciphertext. However, this approach still wastes $N - M_i$ slots *per round*.

Ideally, we would pack *all* round information into a handful of *packed* ciphertexts. However, it is not obvious *a priori* how to do so. Such ciphertexts would need to contain prover messages across *multiple rounds*, but the soundness analysis of IPs requires that the verifier *observe* round i messages before sending challenges that are used to compute the round $i+1$ messages. Our solution is to succinctly commit to the flattened ciphertexts in each round, each using unique global slot indices. In the final round, the prover sends just enough "packed ciphertexts" to contain $M := \sum_{i=1}^{r} M_i$ slots of information, consistent with the committed flattened ciphertexts, along with a SNARK proof of this consistency. Intuitively, these commitments bind the prover to underlying *plaintext*

[13] One can think of the base IP prover as spawning N threads to compute each round message, but rather than joining the results, it sends each thread's result directly to the base IP verifier, which locally performs aggregation. Aggregation often entails an evaluation of a multilinear extension of the results at a random evaluation point, though occasionally only requires computing a sum of N terms. The "folded" descriptor refers to how a folded prover "folds" the results of its N threads before presenting it to the verifier.

slots, thereby preserving state-restoration soundness. As we discuss in Section 6.2, this "proof of packing" adds negligible overhead to proof size and is tractable for the prover.

When our folded BhIP is compiled using transcript packing, the resulting VCoED scheme (after further Fiat-Shamir compilation) is called $\mathsf{Laminate}_{\mathsf{fold}}$. For a payload circuit with $d = 32$ layers, $n = 2^{20}$ gates, $N = 2^{14}$ slots, and plaintext modulus $q = 2^{16} + 1$, $\mathsf{Laminate}_{\mathsf{fold}}$ achieves proof sizes of 270 kilobytes. When the payload circuit is $B = 2^{14}$-laned (Definition 1.1), fully utilizing the SIMD capacity, this implies a *per-lane* proof size of only 16.5 *bytes*, in stark contrast to [86]'s per-lane proof size of 60 *megabytes*.

Specialized protocol for laned payload circuits. The base and folded BhIPs described above handle wide payload circuits (Definition 1.2). However, many practical payload circuits are B-laned (Definition 1.1): they run B independent copies of the same single-slot computation in SIMD parallel, with no cross-slot interactions. Since prior works [86, 40] natively support only 1-laned payloads, this is also the common ground on which we compare.

For this important special case, we design $\mathsf{Laminate}_{\mathsf{laned}}$, a protocol that exploits the laned structure by customizing the GKR layer reductions to the slot-independent wiring of laned circuits. After the server evaluates the B-laned payload circuit (which uses only B slots per ciphertext), a lightweight *witness packing* step packs the intermediate ciphertexts densely into fewer ciphertexts that utilize all N slots. The laned BhIP then operates on these packed ciphertexts, amortizing proof generation across the full SIMD width. In the regime $B \ll N$, the cost of payload computation dominates; proof generation is comparatively inexpensive. $\mathsf{Laminate}_{\mathsf{laned}}$ uses transcript packing (as described above) to achieve succinct proof sizes, with a total multiplicative depth overhead of 4.5 levels.

Limitations. We discuss two limitations of our framework. First, our blind proof framework is inherently designated verifier: only a holder of the secret key can verify a blind proof. This contrasts with "SNARK-over-FHE" approaches, where the server proves statements about ciphertexts that anyone can verify. However, as demonstrated in [40], designated-verifier blind proofs can be lifted to public verifiability via proof composition. Second, the "FHE-over-SNARK" paradigm exhibits a fundamental one-bit leakage if the adversary observes the verification result. While this is an inherent trade-off, the alternative SNARK-over-FHE paradigm faces its own security challenges: as discussed in Remark 2.6, our approach offers resistance against known IND-CPA-D key-recovery attacks.

1.4 Summary of Evaluation

1.4.1 Comparisons with Prior Works

Since prior works [86, 40] only support *laned payload circuits* (Definition 4.3), we compare using $\mathsf{Laminate}_{\mathsf{laned}}$, our variant optimized for this setting. We compare all our variants to prior work in detail in Section 7.

Homomorphically evaluating the blind SNARG. Following prior works [86, 40], we provide an estimation of the runtime of our construction using detailed operation counts and microbenchmarks. Concretely, for a circuit with 2^{20} gates as in [86, 40], the runtime[14] of $\mathsf{Laminate}_{\mathsf{laned}}$ for evaluating the blind SNARG is 2× faster than prior works for 1-laned payloads, 245× faster for 128-laned payloads, and 2385× faster for 2^{14}-laned payloads (full SIMD utilization). Moreover, these advantages increase as circuit size grows, since the servers in prior work require more than quasi-linear FHE operations to homomorphically evaluate the blind SNARG under FHE in constant multiplicative depth (due to NTT-based polynomial commitments, see Section 1.1).

[14] Via operation counts (see Remark 4.1) and microbenchmarks (see Section 7) as prior works.

Proof size. $\mathsf{Laminate}_{\mathsf{laned}}$'s proof size is 446× smaller than Phalanx [86] for 1-laned payloads, and 7,322,384× smaller for 2^{14}-laned payloads (full SIMD utilization).

Input ciphertext blowup. Should prior schemes tune parameters to make their server perform quasi-linear (needed for larger circuits), then they must introduce an overhead of $\Omega(\log n)$ multiplicative depth for evaluating the prover. This necessitates a noise budget increase (concretely: at least 200 extra bits), and thus larger input ciphertexts and correspondingly slower evaluation of the payload circuit. Laminate reduces this overhead to (depending on the variant) between 1.5 and 4.5 multiplicative levels (concretely, 32 to 150 bits of noise budget).

For example, OMR [62] uses ∼800 bits of noise budget for honest evaluation; thus the ciphertext size overhead is between ∼6.25% and ∼15% with Laminate, compared to $> 25\%$ with prior schemes. This advantage further increases as the circuit size grows, since the multiplicative depth of our quasi-linear server circuit is always *constant*, unlike prior works.

Witness reduction. Our constructions do not incur any witness reduction cost. For $\mathsf{Laminate}_{\mathsf{laned}}$ (laned payloads), a lightweight witness packing step is required, costing only $O(n)$ plaintext-by-ciphertext multiplications (concretely, minutes), and this cost is already included in our reported runtimes. In contrast, prior works incur potentially huge witness reduction costs that are inherent but not accounted for, as discussed in Section 1.1.

1.4.2 Comparisons with Honest Payload Generation

We now compare $\mathsf{Laminate}_{\mathsf{fold}}$, our recommended variant for wide payload circuits (Definition 5.1), to *honest payload generation*, where the circuit is homomorphically evaluated without any integrity proof. For a circuit with 2^{20} gates, evaluation time varies with the circuit's topology, specifically the distribution of gate types (multiplications vs. additions) and the multiplicative levels at which they occur. Across all scenarios we have considered, honest evaluation ranges from 0.2 hours to 7.0 hours.

Compared to this honest evaluation, Laminate incurs as little as ∼5.5× overhead when the circuit is multiplication-heavy and most operations occur near the input layer. Across all tested settings, the overhead remains below 67×, with the worst case arising when the circuit is dominated by additions concentrated near the output layer. This cost includes the additional payload generation overhead incurred by the extra multiplicative depth required by Laminate.

1.5 Related Work

FHE-over-SNARK. The paradigm of homomorphically generating a proof under FHE for Verifiable Computation on Encrypted Data (VCoED) was first proposed by [42]. A sequence of follow-up works [4, 40, 86] attempt to realize this paradigm with concrete efficiency. HELIOPOLIS [4] instantiates it using a homomorphic evaluation of the FRI Interactive Oracle Proof (IOP). Blind Fractal [40] utilizes the Fractal proof system, while Phalanx [86] employs the Spartan polynomial interactive oracle proof (PIOP) system. Notably, these works extend the utility of FHE-over-SNARK to broader cryptographic objectives beyond basic VCoED:

- **ZK Proof Delegation:** Blind Fractal [40] applies this paradigm to delegate the generation of a zero-knowledge proof. A client provides the instance and an encrypted witness to a server, which homomorphically evaluates the ZK-SNARK prover circuit. While the resulting proof is initially a designated-verifier proof, the client can subsequently generate a recursive proof to demonstrate knowledge of the secret key required for verification.

- **Maliciously Secure 2PC:** Phalanx [86] leverages FHE-over-SNARK to achieve maliciously secure Two-Party Computation (2PC). Here, FHE ensures the privacy of the client's inputs, while the server homomorphically evaluates a ZK-SNARK and re-randomizes the resulting ciphertexts. The ZK property ensures the server's private inputs remain hidden from the client, while the soundness of the SNARK guarantees the client receives the correct result.

However, all of these works suffer from the issues discussed in Section 1.1. We compare the concrete efficiencies in more detail in Section 7.

SNARK-over-FHE. An alternative line of work uses SNARKs to directly prove that homomorphic operations were performed correctly, co-designing a SNARK and FHE combination so that the SNARK can efficiently capture supported ciphertext operations without degrading the efficiency of the underlying FHE computation.

The first notable work in this direction is [38], which builds a SNARK whose constraints are native to a *quotient polynomial ring* (as used in Ring-LWE-based HE constructions) rather than a finite field. A follow-up [12] modifies the SNARK to operate over an *unquotiented* polynomial ring, citing gains in prover efficiency. However, constraints of either ring still struggle to capture ciphertext operations such as relinearization. Likewise, [41] cannot verify these operations and thus cannot support arbitrary-depth computations. Later, [5] resolved this limitation by designing a SNARK that can prove FHE (as opposed to SHE) computations, though costs remain impractical for circuits requiring multiple levels of multiplicative depth. For further discussion, see the analysis in [81].

Other work [80, 60] proves that an FHE bootstrapping procedure was correctly computed, thereby allowing one to prove homomorphic evaluation of unbounded depth. However, proving the TFHE bootstrapping procedure, which already takes ∼10ms to perform honestly, takes > 5 seconds per circuit gate.

ZFHE [88] proves *in zero-knowledge* that it correctly evaluated an FHE payload circuit, with the primary goal of hiding private server inputs from the client. Their proofs are linear in the circuit size (not succinct), and despite sacrificing succinctness, ZFHE still incurs at least[15] a two-order-of-magnitude overhead over honest execution of the FHE payload computation.

Recent work of [20] makes significant progress towards practical SNARK-over-FHE. Using lookup-argument-based range-checking techniques, they are able to prove ciphertext relinearizations with much less overhead than prior SNARK-over-FHE constructions, and their framework is flexible enough to handle BGV/BFV as well as CKKS computations. For the latter, they introduce a "proof-friendly" variant of CKKS that is slightly less efficient for honest execution but allows for substantially more efficient proof generation. Nevertheless, the server overhead remains large: on a circuit from [81] consisting of a single CKKS multiplication and noise flooding, their estimated proof generation time exceeds 25 seconds with 48 threads, compared to 0.014 seconds for the honest computation with a single thread, demonstrating a gap of 4-5 orders of magnitude. Asymptotically, their proofs are also large, scaling as $\omega(\sqrt{n} \cdot N)$ (from polynomial commitments) for payload circuits with n HE operations and ring dimension N.

2 Preliminaries

2.1 (Leveled) Homomorphic Encryption

A (leveled) homomorphic encryption (HE) scheme provides four algorithms: $\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Eval}$. The Eval algorithm allows for public computation on encrypted data. Given a function f and ci-

[15] Note that the extended version of [88] contains corrected benchmarks.

phertexts $\mathsf{ct}_i = \mathsf{Enc}(\mathsf{pk}, m_i)$, Eval computes $\mathsf{ct}' \leftarrow \mathsf{Eval}(\mathsf{evk}, f, \mathsf{ct}_1, \ldots, \mathsf{ct}_w)$ such that $\mathsf{Dec}(\mathsf{sk}, \mathsf{ct}') = f(m_1, \ldots, m_w)$. We use leveled HE, which supports circuits up to a predetermined depth $\mathsf{MAXDEPTH}$ without a costly bootstrapping operation. We denote semantic security by IND-CPA.

Notation. We say that a ciphertext ct encrypts a plaintext x if $x = \mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$ where sk is the client's decryption key, which will usually be implicit.

Let $\mathbb{F}_p$ be the (base) plaintext space for some finite field order p. For plaintexts $\vec{x} \in \mathbb{F}_p^k$, let $\mathsf{ct}[\vec{x}] = (\mathsf{ct}[\vec{x}_1], \ldots, \mathsf{ct}[\vec{x}_k])$ denote a vector of ciphertexts encrypting $\vec{x}$. For any two vectors of ciphertext $\vec{\mathsf{ct}}_1, \vec{\mathsf{ct}}_2$ of size k, we use $\vec{\mathsf{ct}}_1 \times \vec{\mathsf{ct}}_2$ to denote the homomorphic multiplication $\mathsf{Eval}(\mathsf{pp}, \mathsf{evk}, \times, \vec{\mathsf{ct}}_1[i], \vec{\mathsf{ct}}_2[i])_{i \in [k]}$. We similarly define $\vec{x} \times \vec{\mathsf{ct}}$ for plaintext vector $\vec{x}$ and ciphertext vector $\vec{\mathsf{ct}}$. Homomorphic additions $(+)$ are defined analogously.

We also consider an extended plaintext space $\mathbb{F}_{p^R}$ for some $R > 1$. The extended ciphertext (ect) encrypting an extended plaintext $x \in \mathbb{F}_{p^R}$ consists of R ciphertexts. For $\vec{x} \in \mathbb{F}_{p^R}^k$, let $\mathsf{ect}[\vec{x}] = (\mathsf{ect}[\vec{x}_1], \ldots, \mathsf{ect}[\vec{x}_k])$ denote a vector encrypting $\vec{x}$, where $\mathsf{ect}[\vec{x}_i]$ $(i \in [k])$ consists of R ciphertexts. The homomorphic operations $\times$ and $+$ are defined analogously; if any of their inputs is an extended plaintext in $\mathbb{F}_{p^R}$, or the encryption of such a plaintext, then the operation is done in $\mathbb{F}_{p^R}$; otherwise it is $\mathbb{F}_p$.

We use $\mathsf{depth}(\mathsf{ct}[\cdot])$ to denote the maximum depth of the circuit over which a ciphertext can be evaluated (and in particular $\mathsf{depth}(\mathsf{ct}[\cdot]) \leq \mathsf{MAXDEPTH}$ for all ciphertexts).

Leveled HE and FHE. Bootstrapping [45] can transform leveled HE (LHE) to FHE. In all existing FHE constructions, the input ciphertexts have some inherent initial noise, which accumulates with each level of ciphertext evaluation. Bootstrapping is a method that is used to reduce the accumulated noise in ciphertexts back to some initial level. Unfortunately, all the existing methods of bootstrapping are very costly [54, 43, 66, 70, 65]. Thus, in practice, most applications only use the leveled version of HE, where the depth $\mathsf{MAXDEPTH}$ of the computation is bounded, thereby the focus of this work.

2.2 Indexed Languages and Proof Systems

Indexed languages. We are concerned with indexed languages $\mathcal{L} = \{\mathcal{L}_i\}$, where an index i (e.g., a circuit) defines a specific language $\mathcal{L}_i$ (e.g., valid input-output pairs). We use preprocessing proof systems where i is preprocessed into a short encoding $e[i]$. A verifier can then use $e[i]$ to check proofs in time sublinear in the size of i.

Interactive proofs. Our work builds on public-coin *Holographic Interactive Proofs (pc-hIPs)* for indexed languages [6, 27]. In each round, the prover sends the verifier a message and the verifier replies with a uniformly random response. At the end of the protocol, the verifier inspects the transcript and makes a decision to accept or reject the proof.

IP to NARG compilation. These interactive protocols can be compiled into *Non-interactive Arguments (NARGs)* by applying the *Fiat-Shamir transformation* in the Random Oracle Model (ROM) [37], the protocol is non-interactive.

State-restoration soundness. Standard soundness for interactive proofs bounds the success probability of a *sequential* cheating prover. A stronger notion, *state-restoration soundness* [8], considers an adversary $\mathcal{A}$ that may "rewind" the protocol: in a game parameterized by a *move budget* t, the adversary repeats the following up to t times (or until it decides to stop): it sends a partial transcript to the challenger and receives a fresh verifier challenge. After exhausting its budget, the adversary outputs a complete transcript. The adversary wins if it produces a valid, accepting transcript (i.e., all challenges are consistent with the partial transcripts from which they were derived) for an instance $x \notin \mathcal{L}$. We say that an interactive proof has *state-restoration soundness*

error $\varepsilon^{\mathrm{sr}}(t, n)$ if no adversary with move budget t can win the above game with probability greater than $\varepsilon^{\mathrm{sr}}(t, n)$ on instances of size n.

From IPs to NARGs via Fiat-Shamir. State-restoration soundness is the key property that enables sound Fiat-Shamir compilation in the Random Oracle Model. Concretely, if a public-coin IP has state-restoration soundness error $\varepsilon^{\mathrm{sr}}_{IP}(t, n)$ and we apply the Fiat-Shamir transform using a random oracle with λ-bit output, then the resulting NARG has soundness error

$$\varepsilon_{\mathsf{arg}}(\lambda, t, n) \;\leq\; \varepsilon^{\mathrm{sr}}_{IP}(t, n) \;+\; \frac{t^2}{2^\lambda},$$

where t is the query budget of the adversary in the ROM and n is the instance size [26]. Thus, an IP with negligible state-restoration soundness error yields a NARG with negligible soundness error against query-bounded adversaries.

2.3 Public-Coin Blind Holographic Interactive Proofs (BhIPs)

We depart slightly from the original definition from [40]. Instead, we define a public-coin blind holographic interactive proof as a particular type of public-coin holographic interactive proof, and it is designated-verifier.

Definition 2.1. Given a public-coin hIP Π for an indexed language $\mathcal{L}$, and an HE scheme $\mathcal{E}$, we define $\mathcal{E}[\Pi]$ as a *public-coin blind holographic interactive proof* system (for the same indexed language $\mathcal{L}$).

- $\mathcal{E}[\pi].\mathsf{Setup}(1^\lambda, i)$: For a public index i, this algorithm:
  - Generates $(\mathsf{sk}, \mathsf{evk}) \leftarrow \mathcal{E}.\mathsf{KeyGen}(1^\lambda)$.
  - Generates the plaintext encoding $e[i] \leftarrow \pi.\mathsf{Ind}(i)$.
  - Outputs $(\mathsf{evk}, (\mathsf{sk}, e[i]))$.
- $\mathcal{E}[\pi].\mathsf{P}_{\mathsf{evk}}(i, \mathsf{ct}[x])$: The prover, given index i, encrypted instance $\mathsf{ct}[x]$, and evaluation key evk, interacts with the verifier. In round $j \in [\mu]$, it receives the "transcript so far" tr (which contains encrypted prover messages and scalar verifier challenges) and computes the next round message (and optionally any saved state for the next round) by homomorphically evaluating the plaintext prover $\pi.\mathsf{P}_j$:

$$(\mathsf{ct}[m_j], \mathsf{st}_j) \leftarrow \mathcal{E}.\mathsf{Eval}_{\mathsf{evk}}(\pi.\mathsf{P}_j, (i, \mathsf{ct}[x], \mathsf{tr}, \mathsf{st}_{j-1}))$$

- $\mathcal{E}[\pi].\mathsf{V}_{\mathsf{sk}}(e[i], x)$: The verifier, given $e[i]$, x, and sk, interacts with the prover.
  - **Rounds $j = 1 \ldots \mu$ (Message Rounds):** $\mathcal{E}[\pi].\mathsf{V}$ receives the ciphertext messages $\mathsf{ct}[m_j]$ and stores it in its transcript tr. It then returns a fresh random (scalar) challenge ρ_j.
  - **Round $\mu + 1$ (Decision Round):** $\mathcal{E}[\pi].\mathsf{V}$ decrypts the prover's messages in the transcript tr to obtain a transcript free of ciphertexts, denoted tr'. It then runs the decision algorithm of $\pi.V$ on tr'.

Remark 2.2. We modify the original definition from [40] for simplicity. This definition is specific to the *public-coin* setting, which is the only setting of practical interest.

Theorem 2.3. *If π has state-restoration soundness error ε^{sr} for indexed language $\mathcal{L}$, and $\mathcal{E}$ is a correct HE scheme, then $\mathcal{E}[\pi]$ has state restoration soundness error at most ε^{sr} for $\mathcal{L}$.*

Proof. This was shown in [40] for a related notion of soundness (called round-by-round soundness), but the idea is the same. A state-restoration adversary $\mathcal{A}$ for π can simulate the challenger for state-restoration adversary $\mathcal{A}'$ for $\mathcal{E}[\pi]$ and copy its strategy. □

2.4 Making blind hIPs non-interactive

We compile the public-coin BhIP from into a non-interactive argument using a standard application of the Fiat-Shamir transform. The (blind) non-interactive verifier receives the prover's portion of the transcript, derives the challenges (with the random oracle) to complete the transcript, and runs the (blind) interactive verifier's decision logic. If the hIP has negligible state-restoration soundness error, then a query-bounded adversary cannot forge a blind non-interactive proof with more than negligible probability.

The verifier (who holds sk) runs the non-interactive protocol, decrypts the necessary queries from the proof ciphertexts, and runs the plaintext verifier's logic to accept or reject.

Remark 2.4 (Designated Verifier proof systems)**.** The blind proof systems described are inherently designated-verifier, since a secret key sk is needed for verification. This is shared across all prior works that follow the FHE-over-SNARK paradigm [42, 4, 40, 86].

Remark 2.5 (One-bit leakage for FHE-over-SNARK)**.** As mentioned in prior works [4, 40, 86], in the FHE-over-SNARK paradigm, if a malicious prover can learn whether the proof it generated (under homomorphic evaluation) passed verification, then it can learn 1 bit of information about the encrypted input. For example, an adversary could replace one encrypted bit of the input with an encryption of 1, and follow the rest of the protocol honestly. The blind proof would pass the check if the replaced bit was already 1, and (likely) fail otherwise. With access to the verification result, the adversary learns this bit of input. This is inherent for the FHE-over-SNARK paradigm, since proofs are generated with respect to underlying plaintexts. The SNARK-over-FHE avoids this since the proof attests that each intermediate *ciphertext* was properly generated. On the other hand, the difference in the statements being proven give an advantage to the FHE-over-SNARK paradigm in other settings, as discussed in Remark 2.6.

Remark 2.6 (IND-CPA-D of FHE)**.** While our scheme (or in general any SNARK-over-FHE and FHE-over-SNARK schemes) guarantees integrity for *correctness*, it does not *a priori* rule out a security break (e.g., input leakage) if decrypted plaintexts are leaked (even if all the ciphertexts are guaranteed to be correctly formed, captured by the IND-CPA-D model [58, 59, 1, 9, 19] for FHE). In other words, even though the FHE-over-SNARK or SNARK-over-FHE paradigms guarantee that the evaluation process is correct, FHE schemes might still be insecure in the presence of a decryption oracle.

It is worth noting that the FHE-over-SNARK paradigm has an advantage over the SNARK-over-FHE approach in this setting. Specifically, for "exact FHE" schemes such as BGV, BFV, FHEW, and TFHE [16, 15, 36, 35, 28], to our knowledge, all known attacks on their IND-CPA-D security exploit the fact that accumulated errors and bootstrapping (or the accumulated noise) can alter the underlying plaintexts during decryption [24, 21, 67]. Such plaintext changes may occur *even when* the ciphertext evaluations are computed correctly, allowing these attacks to bypass the SNARK-over-FHE paradigm. Intuitively, FHE-over-SNARK can prevent this attack, since the decrypted SNARK proves a statement about the plaintexts directly. We leave a formal exploration of this observation as future work.

2.5 Multilinear Polynomials and Multilinear Extensions

Definition 2.7. A *multilinear polynomial* $f(X_1, \ldots, X_n) \in \mathbb{F}[X_1, \ldots, X_n]$ is linear in each variable.

Fact 2.8. *A multilinear polynomial on n variables can be uniquely defined by its evaluations over the Boolean Hypercube domain $\{0,1\}^n$. As a corollary, any function $h : \{0,1\}^n \to \mathbb{F}$ has a* unique multilinear extension (MLE) *$\tilde{h} : \mathbb{F}^n \to \mathbb{F}$ such that $\forall \vec{v} \in \{0,1\}^n$, $f(\vec{v}) = \tilde{h}(\vec{v})$.*

The equality multilinear polynomial. We define $\tilde{\mathsf{eq}}_n(X_1, \ldots, X_n, Y_1, \ldots, Y_n)$ as the multilinear extension of the function $\mathsf{eq} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\} \subseteq \mathbb{F}$ defined as follows:

$$\mathsf{eq}(X_1, \ldots, X_n, Y_1, \ldots, Y_n) = \begin{cases} 1 \text{ if } X_i = Y_i \text{ for all } i \in [n], \\ 0 \text{ otherwise.} \end{cases}$$

The multilinear extension $\tilde{\mathsf{eq}}_n \in \mathbb{F}[X_1, \ldots, Y_n]$ is defined as follows:

$$\tilde{\mathsf{eq}}_n[X_1, \ldots, X_n, Y_1, \ldots, Y_n] = \prod_{i=1}^{n} \left(X_i Y_i + (1 - X_i)(1 - Y_i)\right)$$

Note that this is multilinear and its Boolean hypercube evaluations agree with the function eq.

Arbitrary multilinear extensions. Fix an arbitrary list of values $(a_{\vec{v}})_{\vec{v} \in \{0,1\}^n}$ and let $h : \{0,1\}^n \to \mathbb{F}$ be the function such that $h(\vec{v}) = a_{\vec{v}} \in \mathbb{F}$ for all $v \in \{0,1\}^n$. We express the multilinear extension of h, denoted $\tilde{h}$, via Lagrange Interpolation.

$$\tilde{h}(X_1, X_1, \ldots, X_n) = \sum_{\vec{v} \in \{0,1\}^n} a_{\vec{v}} \cdot \tilde{\mathsf{eq}}_n(X_1, X_2, \ldots, X_n, v_1, v_2, \ldots, v_n) \tag{1}$$

Definition 2.9. We call $(a_{\vec{v}})_{\vec{v} \in \{0,1\}^n}$ the *Lagrange Coefficients* of $\tilde{h}$.

Evaluation in Extension Field. Let $\mathbb{E}$ be an extension field of $\mathbb{F}$. It is well defined to view $\tilde{h}$ as a multilinear polynomial defined over $\mathbb{E}$, and thus we can evaluate it at any point $\vec{\gamma} \in \mathbb{E}^n$. As seen in Equation (1), we can evaluate $\tilde{h}$ at $\vec{\gamma}$ by evaluating each Lagrange Basis polynomial $L_{\vec{v}}(X_1, \ldots, X_n) := \tilde{\mathsf{eq}}(X_1, \ldots, X_n, \vec{v})$ for $\vec{v} \in \{0,1\}^n$ at $\vec{X} = \vec{\gamma}$ and taking an inner product with the Lagrange coefficients.

2.6 Multivariate Sumcheck Protocol

We briefly recall the multivariate sumcheck protocol from [68], as used in modern proof systems. Let $\mathbb{F}$ be a field, and let $\mathbb{E}$ be an extension field of $\mathbb{F}$ that is cryptographically large, i.e. $|\mathbb{E}| \geq 2^\lambda$ for security parameter λ.

The prover has access to a multivariate polynomial $f(X_0, \ldots, X_{\mu-1}) \in \mathbb{F}[X_0, \ldots, X_{\mu-1}]^{\leq d}$ of maximium individual degree d. (Typically d is a small constant, like 2.) The verifier has oracle access to f. The prover wants to convince a distrustful verifier that

$$\sum_{x_0, \ldots, x_{\mu-1} \in \{0,1\}} f(x_0, \ldots, x_{\mu-1}) = s_0 \in \mathbb{F}.$$

in such a way that the verifier needs only evaluate f one time (which requires 2^μ times without the prover, since the verifier needs to compute the entire procedure itself).

This takes place over μ rounds. In round $i \in \{0, \ldots, \mu - 1\}$, the prover reduces a claim about the sum of $2^{\mu-i}$ values to a claim about the sum of $2^{\mu-i-1}$ values. The protocol proceeds like so. That is, the initial claim is that $\sum_{x_0, \ldots, x_{\mu-1} \in \{0,1\}} f(x_0, \ldots, x_{\mu-1}) = s_0$. Then in round 0, the prover sends a degree d univariate round polynomial $R_0(X) \in \mathbb{F}[X]^{\leq d}$ claiming that

$$R_0(X) = \sum_{x_1, \ldots, x_{\mu-1} \in \{0,1\}} f(X, x_1, \ldots, x_{\mu-1})$$

If the prover is honest, then $R_0(0) + R_0(1) = \sum_{x_0,\ldots,x_{\mu-1}\in\{0,1\}} f(x_0,\ldots,x_{\mu-1})$. So, the verifier checks that $R_0(0)+R_0(1) = s_0$. Next, the verifier samples a uniform random challenge $\gamma_0 \xleftarrow{\$} \mathbb{E}$ and evaluates $s_1 := R_0(\gamma_0)$. The next *round claim* is that $\sum_{x_1,\ldots,x_{\mu-1}\in\{0,1\}} f(\gamma_0, x_2,\ldots,x_n) = s_1$. This process repeats in round $1,\ldots,n-1$ until we are left with a final round claim $f(\gamma_0,\ldots,\gamma_{n-1}) = s_n$

Completeness intuition If the initial claim is true and the prover follows the protocol, then all subsequent round claims will be true.

Soundness intuition Suppose in round i the current claim is false. The dishonest prover has two choices. If it sends the "real" round polynomial $R_i(X)$ such that

$$R_i(X) = \sum_{x_{i+1},\ldots,x_{\mu-1}\in\{0,1\}} f(\gamma_0,\ldots,\gamma_{i-1}, X, x_{i+1},\ldots,x_{\mu-1})$$

then $R_i(0) + R_i(1) \neq s_i$, and the verifier will reject. If it sends a fake round polynomial, then $R_i(X) \neq \sum_{x_{i+1},\ldots,x_{\mu-1}\in\{0,1\}} f(\gamma_0,\ldots,\gamma_{i-1}, X, x_{i+1},\ldots,x_{\mu-1})$, so with probability at least $1-\frac{d}{|\mathbb{E}|}$, $R_i(\gamma_i) \neq \sum_{x_{i+1},\ldots,x_{\mu-1}\in\{0,1\}} f(\gamma_0,\ldots,\gamma_{i-1},\gamma_i, x_{i+1},\ldots,x_{\mu-1})$, in which case the next round claim will also be false. So, the dishonest prover has μ rounds to turn a false claim into a true claim, but in each round, the probability of doing so is at most $\frac{d}{|\mathbb{E}|}$. By Union Bound, a cheating prover fools the verifier with probability at most $\frac{d\mu}{\mathbb{E}}$.

Remark 2.10 (Simple optimization: omit one round polynomial evaluation)**.** Typically, the prover specifies a degree d round polynomial with $d+1$ evaluations (e.g. $R(0),\ldots,R(d)$ if the field has characteristic more than d). However, since the verifier is going to check $R(0)+R(1) = s$, the prover can save time and omit evaluating $R(1)$, as the verifier can simply deduce it. Thus, we will assume that the prover uses d evaluations (e.g. $R(0), R(2),\ldots,R(d)$) to specify degree d round polynomials.

3 Requisite hIP and HE Schemes

We review the GKR protocol (specifically the refinement of [30] described in [78]) and the BGV/BFV FHE scheme.

3.1 GKR Protocol

Let Ckt be a depth d layered fan-in-two arithmetic circuit over field $\mathbb{F}_q$. By convention, we say L_0 is the list of *output* wires, L_d is the list of *input* wires, and L_i is the list of wires in layer i. The *layered requirement* states that every wire in L_i is the output of an ADD or MUL gate applied to two wires in L_{i+1}. The GKR protocol describes how a prover P convinces verifier V of membership in the language

$$L_{\mathsf{Ckt}} = \{(\mathsf{in}, \mathsf{out}) \mid \mathsf{Ckt}(\mathsf{in}) = \mathsf{out}\}$$

with perfect completeness and negligible soundness error.

Formally, the GKR protocol can be described as a public-coin holographic Interactive (Oracle) Proof System $\mathsf{HOL} = (\mathbf{I}, \mathbf{P}, \mathbf{V})$. The indexer $\mathbf{I}$ processes Ckt in an offline phase as follows.

GKR Online Phase

1. **Initial Claim:** The protocol begins with the prover's initial claim that the circuit output matches the provided output: $L_0 \stackrel{?}{=} \mathsf{out}$.
2. **Output Randomization:** The verifier V samples a random challenge $\gamma \in (\mathbb{F}_{q^R})^{s_0}$ and sends it to the prover P. The claim is updated to be an evaluation of the multilinear extension:

$$\tilde{L}_0(\gamma) \stackrel{?}{=} \tilde{\mathsf{out}}(\gamma)$$

   By the Schwartz-Zippel lemma, this step incurs a soundness loss of at most $\frac{s_0}{|q^R|}$.
3. **Layer Reduction Loop:** For each layer $i = 0, \ldots, d-1$:
   - P and V engage in a $2s_{i+1} + 1$ round "layer reduction" subroutine.
   - This subroutine reduces the claim about $\tilde{L}_i$ (from the previous step) to a new claim of the form $\tilde{L}_{i+1}(r_a) + \alpha\tilde{L}_{i+1}(r_b) \stackrel{?}{=} v$ for some r_a, r_b, α, v derived during the subroutine.
   - This reduction step incurs a soundness loss of $\frac{8s_{i+1}+1}{q^R}$.
4. **Final Check:** After the loop finishes (at $i = d-1$), the verifier holds a final claim about the input layer L_d:

$$\tilde{L}_d(r_a) + \alpha\tilde{L}_d(r_b) \stackrel{?}{=} v$$

   The verifier can check this claim directly using the known circuit inputs in.

Figure 2: GKR Online Phase

For each layer $j \in \{0, ..., d\}$, it computes and stores $s_j := \log_2(|L_j|)$.[16] Then, for each layer $j \in \{0, \ldots, d-1\}$, it computes gate indicator functions $\mathsf{add}_j, \mathsf{mul}_j : \{0,1\}^{s_j} \times \{0,1\}^{2s_{j+1}}$:

$$\mathsf{add}_j(g,x,y) = \begin{cases} 1 & \text{if } L_j[g] = L_{j+1}[x] + L_{j+1}[y], \\ 0 & \text{otherwise.} \end{cases}$$

$$\mathsf{mul}_i(g,x,y) = \begin{cases} 1 & \text{if } L_j[g] = L_{j+1}[x] \cdot L_{j+1}[y], \\ 0 & \text{otherwise.} \end{cases}$$

Using this, indexer $\mathbf{I}$ creates $2d$ commitments to multilinear extensions $\tilde{\mathsf{add}}_j, \tilde{\mathsf{mul}}_j$ for all $j \in \{0, \ldots, d-1\}$. The index i stores the s_j's (log size of each layer) and the $\mathsf{add}_j, \mathsf{mul}_j$ indicator functions. The encoded index $e[i]$ stores the values $\{s_j\}_{0 \le j \le d}$ and the multilinear commitments to $\tilde{\mathsf{add}}_j, \tilde{\mathsf{mul}}_j$ for $j \in \{0, \ldots, d-1\}$.

Remark 3.1. For the context of this paper, we will make a simplifying assumption that the verifier has access to $\mathsf{add}_j, \mathsf{mul}_j$, rather than commitments. We will justify this assumption further in Section 3.1.3.

After the offline phase, the prover and verifier engage in an interactive online phase, given in Figure 2 using the subroutines below.

[16] For ease of exposition, we will assume all layers have power-of-two many wires. If this is not the case, one can pad with dummy zero wires.

3.1.1 The Layer Reduction Subroutine

Recall from the high-level overview that at the beginning of round i (for $i \in \{0, \ldots, d-1\}$), the verifier V holds a single claim about layer i. For $i > 0$, this claim is a batched evaluation:

$$\tilde{L}_i(\vec{r}_a) + \alpha \tilde{L}_i(\vec{r}_b) \stackrel{?}{=} v_i$$

where $\vec{r}_a, \vec{r}_b$ and α are challenges from the previous round's reduction. The layer reduction subroutine reduces this to a new, single claim about layer $i+1$.

This reduction exploits the following fundamental identity, which relates the evaluation of $\tilde{L}_i$ at any point $\vec{r}$ to a sum over all possible input wire pairs from layer $i+1$.

$$\tilde{L}_i(\vec{r}) = \sum_{\vec{x} \in \{0,1\}^{s_{i+1}}} \sum_{\vec{y} \in \{0,1\}^{s_{i+1}}} g^{(i)}(\vec{r}, \vec{x}, \vec{y}) \tag{2}$$

$$\begin{aligned} \text{where } g^{(i)}(\vec{r}, \vec{x}, \vec{y}) := \tilde{\mathsf{add}}_i(\vec{r}, \vec{x}, \vec{y}) \left(\tilde{L}_{i+1}(\vec{x}) + \tilde{L}_{i+1}(\vec{y}) \right) \\ + \tilde{\mathsf{mul}}_i(\vec{r}, \vec{x}, \vec{y}) \left(\tilde{L}_{i+1}(\vec{x}) \cdot \tilde{L}_{i+1}(\vec{y}) \right) \end{aligned}$$

Applying this identity to the verifier's batched claim, we get:

$$\begin{aligned} v_i &\stackrel{?}{=} \tilde{L}_i(\vec{r}_a) + \alpha \tilde{L}_i(\vec{r}_b) \\ &= \sum_{\vec{x}, \vec{y}} g^{(i)}(\vec{r}_a, \vec{x}, \vec{y}) + \alpha \sum_{\vec{x}, \vec{y}} g^{(i)}(\vec{r}_b, \vec{x}, \vec{y}) \\ &= \sum_{\vec{x} \in \{0,1\}^{s_{i+1}}} \sum_{\vec{y} \in \{0,1\}^{s_{i+1}}} \left(g^{(i)}(\vec{r}_a, \vec{x}, \vec{y}) + \alpha g^{(i)}(\vec{r}_b, \vec{x}, \vec{y}) \right) \end{aligned}$$

Define the batched polynomial for the sumcheck as:

$$g^{(i)}_{\text{batch}}(\vec{x}, \vec{y}) := g^{(i)}(\vec{r}_a, \vec{x}, \vec{y}) + \alpha g^{(i)}(\vec{r}_b, \vec{x}, \vec{y}) \tag{3}$$

The claim v_i is now equivalent to $\sum_{\vec{x}, \vec{y}} g^{(i)}_{\text{batch}}(\vec{x}, \vec{y}) \stackrel{?}{=} v_i$. The prover and verifier use the sumcheck protocol to verify this, as can be seen in Figure 3.

Remark 3.2. The first layer reduction ($i = 0$) is a special case. The initial claim is $\tilde{L}_0(\vec{\gamma}) \stackrel{?}{=} v_0$ (where $v_0 = \tilde{\mathsf{out}}(\vec{\gamma})$). This can be seen as the general form $\tilde{L}_0(\vec{r}_a) + \alpha \tilde{L}_0(\vec{r}_b) \stackrel{?}{=} v_0$ by setting $\vec{r}_a = \vec{\gamma}$, $\alpha = 0$, and $\vec{r}_b$ to an arbitrary vector (e.g., $\vec{0}$). The verifier's check in Step 2 simplifies accordingly, as $v_a = \tilde{\mathsf{add}}_0(\vec{\gamma}, \vec{\rho}, \vec{\phi})$ and $v_m = \tilde{\mathsf{mul}}_0(\vec{\gamma}, \vec{\rho}, \vec{\phi})$.

Theorem 3.3 ([30, 78]). *The prover of the GKR refinement described in [30, 78] can be run in quasi-linear time with respect to the size of the circuit. Furthermore, the proof size and verifier time are linear in the number of GKR rounds.*

Remark 3.4. Libra [83] describes a GKR variant that can be run in *linear* time, but the approach is not FHE-friendly.[17]

[17] The [83] sumcheck prover reuses work between rounds, which leads to super-constant multiplicative depth.

GKR Layer Reduction ($i \to i+1$)

1. **Sumcheck:** P and V engage in a $2s_{i+1}$-round sumcheck protocol to prove the claim:

$$\sum_{\vec{x}\in\{0,1\}^{s_{i+1}}} \sum_{\vec{y}\in\{0,1\}^{s_{i+1}}} g^{(i)}_{\text{batch}}(\vec{x},\vec{y}) \stackrel{?}{=} v_i$$

   At the end of the sumcheck, V has sampled random challenges $\vec{\rho},\vec{\phi} \in (\mathbb{E})^{s_{i+1}}$ and P has provided a final claimed value v_{sum}. The original claim is reduced to a new claim:

$$g^{(i)}_{\text{batch}}(\vec{\rho},\vec{\phi}) \stackrel{?}{=} v_{\mathsf{sum}}$$

2. **Evaluate and Send:** The verifier will check this new claim. By factoring $g^{(i)}_{\text{batch}}$, it becomes:

$$\begin{aligned} v_{\mathsf{sum}} \stackrel{?}{=} &\left(\tilde{\mathsf{add}}_i(\vec{r}_a,\vec{\rho},\vec{\phi}) + \alpha\tilde{\mathsf{add}}_i(\vec{r}_b,\vec{\rho},\vec{\phi})\right)\cdot\left(\tilde{L}_{i+1}(\vec{\rho}) + \tilde{L}_{i+1}(\vec{\phi})\right) \\ &+\left(\tilde{\mathsf{mul}}_i(\vec{r}_a,\vec{\rho},\vec{\phi}) + \alpha\tilde{\mathsf{mul}}_i(\vec{r}_b,\vec{\rho},\vec{\phi})\right)\cdot\left(\tilde{L}_{i+1}(\vec{\rho})\cdot\tilde{L}_{i+1}(\vec{\phi})\right)\end{aligned}$$

   To verify this, the parties proceed as follows:
   - V computes the batched gate evaluations *locally* (using the assumption in Remark 3.1):

$$v_a \leftarrow \tilde{\mathsf{add}}_i(\vec{r}_a,\vec{\rho},\vec{\phi}) + \alpha\tilde{\mathsf{add}}_i(\vec{r}_b,\vec{\rho},\vec{\phi}); \qquad v_m \leftarrow \tilde{\mathsf{mul}}_i(\vec{r}_a,\vec{\rho},\vec{\phi}) + \alpha\tilde{\mathsf{mul}}_i(\vec{r}_b,\vec{\rho},\vec{\phi})$$

   - P computes and sends the two values V does not know:

$$v_0 \leftarrow \tilde{L}_{i+1}(\vec{\rho}); \qquad v_1 \leftarrow \tilde{L}_{i+1}(\vec{\phi})$$

3. **Verifier's Local Check:** V checks that the values provided by P are consistent with the sumcheck result.

$$V \text{ checks: } v_{\mathsf{sum}} \stackrel{?}{=} v_a\,(v_0+v_1) + v_m\,(v_0\cdot v_1)$$

   If this check fails, V rejects. Otherwise, V is now convinced of the $i \to i+1$ reduction *if and only if* P's two claims from the previous step are true.
4. **Batching:** To reduce to one claim, V samples and sends batching challenge $\alpha' \in \mathbb{E}$ to P.
5. **Resulting Claim:** The two claims are combined into a single, new claim about layer $i+1$:

$$\tilde{L}_{i+1}(\vec{\rho}) + \alpha'\tilde{L}_{i+1}(\vec{\phi}) \stackrel{?}{=} v_0 + \alpha' v_1$$

   This new claim becomes the input for the next layer reduction (for layer $i+1 \to i+2$).

Figure 3: GKR Layer Reduction ($i \to i+1$)

3.1.2 Round Complexity and Proof Size

The GKR protocol as described above is an r-round holographic IP that is information-theoretically succinct, where $r = \sum_{i=1}^{d}(2s_i+1)$. Instantiating the vector oracles with a trivial vector commitment scheme, we view GKR as a holographic IP. In each round, the prover sends at most three extension field elements. We give a convenient upper bound on GKR proof size, that depends only on the total number of gates n and the depth of the circuit d.

Claim 3.5. *For a circuit of depth d and n gates, the number of GKR rounds is upper bounded by*

$$d\left(2\log\left(\frac{n}{d}\right)+1\right)$$

Proof. The round complexity is defined as $r = \sum_{i=1}^{d}(2s_i+1)$. Substituting $s_i = \log S_i$, we rewrite

this sum as:

$$r = 2\sum_{i=1}^{d} \log S_i + d.$$

By definition, we know that $\sum_{i=0}^{d} S_i = n$, which implies $\sum_{i=1}^{d} S_i \leq n$. Since the logarithm function is concave, by Jensen's inequality, the term $\sum_{i=1}^{d} \log S_i$ is maximized when all S_i are equal (i.e., $S_i = n/d$). Therefore:

$$\sum_{i=1}^{d} \log S_i \leq d \cdot \log\left(\frac{n}{d}\right).$$

Substituting this back into the expression for r:

$$r \leq 2\left(d \cdot \log\left(\frac{n}{d}\right)\right) + d = d\left(2\log\left(\frac{n}{d}\right) + 1\right).$$

□

Lemma 3.6. *For a circuit of depth d and n gates, the proof size of the GKR hIP is upper bounded by $2 \cdot d\left(2\log\left(\frac{n}{d}\right) + 1\right)$ extension field elements.*

Proof. In the proof of Claim 3.5, we showed that the GKR protocol has at most $r = d\left(2\log\left(\frac{n}{d}\right)\right)$ sumcheck rounds and d other rounds. In each sumcheck round, the prover sends a degree 2 univariate sumcheck round polynomial. Typically, this would require the sumcheck prover to send three evaluations to uniquely specify this round polynomial. However, the simple optimization described in Remark 2.10 allows the omission of one evaluation. In the d non sumcheck rounds, the prover sends two multilinear evaluations. This requires two extension field elements.

The claimed upper bound on total proof size follows. □

3.1.3 Leveraging Circuit Structure

When the circuit possesses structural properties—common in real-world applications—the GKR protocol achieves significantly higher efficiency.

Data-Parallel Circuits. As observed by [77], circuits with regular wiring patterns admit a more efficient layer reduction subroutine. For these *data-parallel circuits*, the ith layer reduction step can be optimized from a sparse sumcheck over $2s_{i+1}$ variables to a dense sumcheck over s_i variables.

MLE-Structured Functions. Standard multilinear evaluation requires an inner product between Lagrange coefficients and basis polynomial evaluations, as shown in Equation (1). However, certain functions possess structure that permits more efficient evaluation of their multilinear extensions. In [75], these are termed *MLE-structured* functions. For example, consider the function $\mathsf{OR} : \{0,1\}^n \to \{0,1\} \subseteq \mathbb{F}$ defined by

$$\mathsf{OR}(X_1, \ldots, X_n) = \begin{cases} 0 & \text{if } X_1 = \ldots = X_n = 0, \\ 1 & \text{otherwise.} \end{cases}$$

Its multilinear extension is $\tilde{\mathsf{OR}}(X_1, \ldots, X_n) = 1 - \prod_{j=1}^{n}(1 - X_j)$, which can be evaluated in $O(n)$ field operations rather than $O(2^n)$.

MLE-Structured gate indicators. In Remark 3.1, we assumed the verifier had oracle access to multilinear extensions of the gate indicator functions $\mathsf{add}_j, \mathsf{mul}_j$. For circuits where these indicators are MLE-structured, we can remove this assumption. The indexer $\mathbf{I}$ simply provides the verifier

with a succinct arithmetic circuit describing the evaluation of the indicator's MLE; the verifier then runs this locally whenever an evaluation is required.

Unstructured circuits via commitments. In the general case where gate indicator functions are *not* MLE-structured, we must justify the assumption in Remark 3.1 differently. The indexer instead provides the verifier with binding multilinear commitments to the gate indicators. During layer reduction, when the verifier queries $\tilde{\mathsf{add}}_i$ or $\tilde{\mathsf{mul}}_i$, the prover supplies the claimed evaluation. As a post-processing step, the prover provides opening proofs (with respect to the binding commitments) to validate all evaluation claims made during the protocol.

Field-Agnostic sparse commitments. To handle the general case efficiently, we require a multilinear commitment scheme where prover costs scale with the sparsity (number of non-zero entries) of the polynomial (otherwise, the prover would incur an $\Omega(n^2)$ cost). Spark [74, 75] provides a sumcheck-based compiler that transforms a dense multilinear commitment scheme into a sparse one, preserving the desired prover efficiency. Instantiating Spark with a field-agnostic dense PCS, such as BaseFold [84], satisfies these requirements.

3.1.4 Algorithmically Dealing with Gate Indicator Functions

A naive implementation of the layer reduction sumcheck appears to require $\Omega((S_{i+1})^2)$ work, as the sum is over $2^{2s_{i+1}} = (S_{i+1})^2$ terms. Indeed, if any layer L_{i+1} is *fat* (i.e., has $S_{i+1} = \Omega(n)$ wires), this implies an $\Omega(n^2)$ cost, which is far from the (quasi)-linear time claim.

The key to an efficient prover is that the gate indicator functions add_i and mul_i are **sparse**. By definition, there are at most S_i gates in layer i, so the functions add_i and mul_i can have at most S_i non-zero entries in total. This sparsity in the Lagrange-coefficient domain is inherited by their multilinear extensions, $\tilde{\mathsf{add}}_i$ and $\tilde{\mathsf{mul}}_i$.

The GKR sumcheck prover's main task is to compute the round polynomials $R_j(X)$. This, in turn, requires evaluating the $g^{(i)}_{\text{batch}}$ polynomial, which depends on $\tilde{\mathsf{add}}_i$ and $\tilde{\mathsf{mul}}_i$. A standard technique (used in [30] and beyond) allows us to compute the evaluations of these sparse MLEs at each round j of the sumcheck.

Specific to GKR, a consequence of the gate indicators' sparsity is the following lemma, whose proof we defer to Appendix A.

Lemma 3.7. *Fix a layer reduction $i \in \{0, \ldots, d-1\}$. The prover can compute (sparse) tables $\hat{A}_j$ (and $\hat{M}_j$) at the beginning of the jth round of sumcheck such that for layer challenge $\vec{r} \in \mathbb{F}_{q^R}^{s_i}$, sumcheck challenges $\vec{\gamma} = (\gamma_0, \ldots, \gamma_{j-1}) \in \mathbb{F}_{q^R}^{j}$, $X \in \{0, 2\}$, and $w \in \{0,1\}^{2s_{i+1}-j-1}$*

$$\hat{A}_j[X][w] = \tilde{\mathsf{add}}_i(\vec{r}, \vec{\gamma}, X, \vec{w})$$
$$\hat{M}_j[X][w] = \tilde{\mathsf{mul}}_i(\vec{r}, \vec{\gamma}, X, \vec{w})$$

Moreover, the prover requires $O(S_i \cdot (s_i + s_{i+1}))$ $\mathbb{F}_q$-ops in total across all sumcheck rounds.

Remark 3.8 (Layered Circuits vs. General Circuits)**.** The GKR protocol as presented above requires layered circuits, where each gate in layer i takes its inputs exclusively from layer $i+1$. This is not a fundamental limitation: a GKR variant due to [85] supports general (non-layered) arithmetic circuits, in which a gate in layer i may take inputs from *any* layer below it. The key idea is that the layer reduction sumcheck sums not only over the input wires from the immediately next layer, but over input wires from all necessary deeper layers, using an extended wiring predicate that

encodes cross-layer connections. The resulting protocol is efficient and uses only standard sumcheck techniques.

However, there is a well-known tension between the generality and the efficiency of GKR-based protocols (see [79, Section 4.6.8]). The general-circuit construction of [85] is more complex to describe and analyze. In the worst case (maximally unstructured circuits), it admits a *slightly* less efficient prover than the layered construction of [30, 78], due to the increased complexity of the layer reduction step. When the circuit is layered, the [85] construction converges exactly to the [30, 78] construction described above, which is simpler to present and analyze. Most real-world circuits are naturally layered (or close to layered), so we choose to focus on the layered construction for clarity of exposition and cost analysis.

In Sections 4 and 5, we present our BhIP constructions for alternating layered payload circuits. We emphasize that this layered assumption is made for clarity of exposition and cost analysis, not out of necessity. The techniques of [85] can be applied to extend our layer reductions so that gates draw inputs from all layers below, rather than just the immediately next layer, at the cost of a more involved (but still efficient) sumcheck structure. We discuss this further when introducing each construction.

3.2 BGV/BFV FHE Protocol

The BGV [16] and BFV [15, 36] line of works are popular choices of leveled homomorphic encryption schemes that are both compatible with the Laminate protocols we will present in Section 6. While we overview the BGV construction below, we remark that BFV is very similar.

Given a polynomial y that lives in the ring $\mathcal{R}_t = \mathbb{Z}_t[X]/(X^N + 1)$, the BGV scheme encrypts y into a ciphertext consisting of two polynomials, where each polynomial is from a larger cyclotomic ring $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N + 1)$ for some $Q > t$. We refer to t as the plaintext modulus, Q as the ciphertext modulus, and N as the ring dimension. We assume that t satisfies that $t \equiv 1 \mod 2N$, where N is a power of two, which is typical in practice [7, 72].

Plaintext encoding. To encrypt a plaintext $\vec{m} = (m_1, \ldots, m_N) \in \mathbb{Z}_t^N$, BGV creates polynomial $m(X) = \sum_{i\in[N]} m_i X^{i-1}$, and then *encodes* it by constructing *another* polynomial $y(X) = \sum_{i\in[N]} y_i X^{i-1}$ where $m_i = y(\eta_j)$, for some predetermined points η_j. Such encoding can be done using an Inverse Number Theoretic Transformation (INTT).

Encryption and decryption. The BGV ciphertext encrypting $\vec{m}$ under $\mathsf{sk} \leftarrow \mathrm{dist}$ has the format $\mathsf{ct} = (a, b) \in \mathcal{R}_Q^2$, satisfying $b - a \cdot \mathsf{sk} = y + t \cdot e$ where $\lfloor Q/t \rfloor \cdot y \in \mathcal{R}_Q$ and y is the polynomial encoded in the manner above, and e is some small error term sampled from some Gaussian distribution over $\mathcal{R}_Q$.

Symmetric key encryption can be done by sampling a random $a \in \mathcal{R}_Q$ and constructing $b \in \mathcal{R}_Q$ accordingly using sk. Public key encryption can also be achieved easily but it is not relevant to our paper so we refer the reader to [15, 36, 53] for details.

Decryption is calculated via $y' \leftarrow \lceil (t/Q) \cdot (b - a \cdot \mathsf{sk}) \rfloor \in \mathcal{R}_t$ (note that $(b - a \cdot \mathsf{sk})$ is done over $\mathcal{R}_Q$), followed by a decoding process (which is also a linear transformation). Henceforth, we assume $\mathsf{BGV.Dec}$ outputs plaintext $\in \mathbb{Z}_t^N$, which is the decoded form, for simplicity. Similarly, we assume $\mathsf{BGV.Enc}$ contains the encoding process.

BGV operations. BGV essentially supports addition (including scalar and plaintext variants), multiplication, and rotation, satisfying the following properties:

- (Ciphertext addition) $\mathsf{BGV.Dec}(\mathsf{ct}_1 + \mathsf{ct}_2) = \mathsf{BGV.Dec}(\mathsf{ct}_1) + \mathsf{BGV.Dec}(\mathsf{ct}_2)$
- (Scalar addition) $\mathsf{BGV.Dec}(c + \mathsf{ct}) = \mathsf{BGV.Dec}(\mathsf{ct}) + c$ for $c \in \mathbb{Z}_t$

- (Plaintext addition) $\mathsf{BGV.Dec}(\vec{c} + \mathsf{ct}) = \mathsf{BGV.Dec}(\mathsf{ct}) + \vec{c}$ for $\vec{c} \in \mathbb{Z}_t^N$
- (Scalar multiplication) $c \times \mathsf{BGV.Dec}(\mathsf{ct}) = \mathsf{BGV.Dec}(c \times \mathsf{ct})$ for $c \in \mathbb{Z}_t$
- (Plaintext multiplication) $\vec{c} \times \mathsf{BGV.Dec}(\mathsf{ct}) = \mathsf{BGV.Dec}(\vec{c} \times \mathsf{ct})$ for $\vec{c} \in \mathbb{Z}_t^N$
- (Ciphertext multiplication) $\mathsf{BGV.Dec}(\mathsf{ct}_1 \times \mathsf{ct}_2) = \mathsf{BGV.Dec}(\mathsf{ct}_1) \times \mathsf{BGV.Dec}(\mathsf{ct}_2)$
- (Rotation) $\mathsf{BGV.Dec}(\mathsf{rot}(\mathsf{ct}, j))[i] = \mathsf{BGV.Dec}(\mathsf{ct})[i + j \pmod N]$ for all $i, j \in [N]$.[18]

We assume all necessary evaluation keys are correctly and implicitly taken. All operations are operated over the entire plaintext vector $m \in \mathbb{Z}_t^N$, element-wise. Thus, all messages need to be evaluated using the same operation by default: i.e., the additions and multiplications are simply done element-wise. This is also known as the Single Instruction Multiple Data (SIMD) [76] property of BGV.

Since we use all of these operations as blackboxes, we omit the details of their realization and refer the reader to [16, 15, 36, 53].

Multiplicative depth. One important aspect of BFV/BGV is the multiplicative depth. To evaluate a circuit with k levels of multiplications, the runtime is essentially $\Omega(k)$ per operation, unless bootstrapping is used (which is often too expensive to use in practice). The main reason is that each operation enlarges the input RLWE ciphertext noise by a certain amount. When the noise crosses a certain threshold ($\frac{Q}{2t}$), the correctness guarantee is broken. Thus, when generating the initial (fresh) ciphertexts, one needs to account for the final noise, such that the "noise budget" is enough. Concretely, this means setting the ciphertext modulus Q accordingly. We elaborate below how much of the noise budget each operation consumes.

Noise Growth and Depth. We characterize the cost of homomorphic operations based on their impact on the noise budget. k of Homomorphic addition incurs a noise growth factor of approximately $\sqrt{k}$; in practice, this is negligible compared to multiplication, so we consider additions to be free.

In contrast, multiplication consumes a significant noise budget. We distinguish between two types:

- **Scalar Multiplication:** Multiplying a ciphertext by a plaintext scalar increases noise by a factor of t (i.e., k consecutive scalar multiplications introduce t^k noise). In this work, we allocate this a cost of 0.5 levels of depth.
- **Ciphertext multiplication:** Multiplying two ciphertexts increases noise by a factor of roughly $t \cdot N$ (i.e., k consecutive ciphertext multiplications introduce $(t \cdot N)^k$ noise). We allocate this a cost of 1 level of depth.

Lastly, for rotations, similar to additions, most works in FHE simply ignore it as its noise consumption is additive instead of multiplicative. Specifically, for k rotations, the noise is $k \cdot \sqrt{N} \cdot \ell$ for some tunable parameter ℓ (see, e.g., [53] for details). Concretely, in the default implementation of the SEAL library [72], the default parameters consume ≥ 10 bits of noise budget for 14 consecutive rotations, which is $< 1/3$ of the noise budget consumption as a single multiplication. Thus, we do not directly count it towards multiplicative depth, but we will count it towards the noise budget estimation in our evaluation.

[18] In fact, for BGV and BFV, the plaintext is divided into two $\vec{m}_1, \vec{m}_2 \in \mathbb{Z}_t^{N/2}$ vectors and the rotations are performed within these two vectors independently (i.e., let $\vec{m}'_b$ be the rotated vector, for $b \in [1, 2]$; then $\vec{m}'_b[i] = \vec{m}'_b[i + j]$ for $i, j \in [N/2]$). These two vectors can be swapped in their positions as well. For simplicity, we ignore this since it does not affect our scheme.

The runtime of these operations is linear in the circuit depth, for the following reason. Operations are performed over a large ciphertext modulus Q. To implement this efficiently, the BGV implementations normally employ the Residue Number System (RNS) to decompose Q into d small primes $q_1, \ldots, q_d$, such that $\prod q_i \approx Q$. Each q_i is chosen to fit within a standard machine word ($\lessapprox$ 60 bits). Since the required size of Q (and thus the number of RNS limbs d) scales linearly with the multiplicative depth of the circuit, the total runtime scales linearly with the depth. For large t (e.g., 50 bits), a single multiplication level consumes the noise budget provided by one limb. For smaller t (e.g., 17 bits), it can take 2 multiplications to consume the same noise budget. For simplicity, one may assume d to be equal to the number of multiplications needed unless otherwise specified.

4 Special Case: BhIP for Laned Payload Circuits

4.1 Common BhIP Notation and Machinery

In this and the following section, we present Blind Holographic IPs (BhIPs) based on variants of the GKR protocol. These protocols verify computations with SIMD-structure exhibited by typical payload circuits evaluated under BGV or BFV. We distinguish two classes of payload circuits. A *B-laned* payload circuit uses $B < N$ of the available SIMD slots, with no cross-slot interactions; see Definition 4.3. A *wide* payload circuit utilizes all N SIMD slots and may include cross-slot operations such as rotations; see Definition 5.1.[19]

Recall from Section 2.3 that a BhIP is fully specified by its two components:

1. The HE Scheme $\mathcal{E}$: We choose the BGV/BFV FHE scheme.
2. The hIP π: We choose a suitable variant of the GKR protocol.

Each hIP induces a BhIP as per the transformation discussed in Section 2.3.

Remark 4.1. Total operation counts that are presented in this and the following section represent *conservative upper bounds*. For example, our simplification using the global maximum width $s_{\max} := \max\{\log(S_i)\}$ introduces additional slack for circuits with varying layer widths. Thus, the actual computational cost for any specific circuit execution will be within (and likely lower than) the reported upper bounds.

4.1.1 Ciphertext and Plaintext Notation

We use the notation $\mathsf{mask}[w]$ to denote an encrypted value, which corresponds to a particular slot of a particular ciphertext. For typographical reasons, we henceforth use the convenient notation that

$$\mathsf{ct}[x] \equiv \left(\mathsf{mask}[x(0^{\log N})], \cdots, \mathsf{mask}[x(1^{\log N})]\right),$$

where $\mathsf{ct}[x]$ denotes a single ciphertext and $x : \{0,1\}^{\log N} \to \mathbb{F}_q$ is a function with inputs on the Boolean hypercube. For plaintexts, we adopt a similar convention:

$$\mathsf{pt}[y] \equiv \left(y(0^{\log N}), \cdots, y(1^{\log N})\right).$$

[19] The term "wide" reflects the fact that the payload computation spans the full width of the ciphertext (all N slots), but not necessarily in a laned fashion, as cross-slot interactions are permitted.

It will be convenient to define the notion of *extended* ciphertexts and plaintexts that store an N-tuple of $\mathbb{F}_{q^R}$ extension field elements. We use the respective notation ect and ept. Internally, a ect (or ept) is represented by R ct (or pt) limbs. Similarly, we use the notation emask to denote the encryption of an $\mathbb{F}_{q^R}$ extension field value. We will use the following equivalences

$$\mathsf{ect}[x] \equiv \left(\mathsf{emask}[x(0^{\log N})], \ldots, \mathsf{emask}[x(1^{\log N})]\right),$$
$$\mathsf{ept}[y] \equiv \left(y(0^{\log N}), \ldots, y(1^{\log N})\right),$$

where $x, y : \{0,1\}^{\log N} \to \mathbb{F}_{q^R}$ are functions.

4.1.2 The Folding Operator

The FOLD operator acts on a list of N values and returns one value that stores the sum. For typographical convenience, we overload notation and sometimes invoke FOLD on an input ciphertext (or a plaintext) which has the same output type. The resulting ciphertext (or plaintext) stores the sum of the N input slots in its **first slot**. We generalize this to the field extension, by allowing FOLD to take as input a ect (or ept), which runs R parallel copies of FOLD on the R ct (or pt) limbs of its input. It will sometimes be convenient to think of FOLD as taking as input a ct (or ect) and returning a mask (or emask).

Remark 4.2. The fold operator FOLD can be implemented in BFV/BGV using $\log N$ iterations of rotation and addition. As discussed in Section 3.2, rotations and additions are typically not counted toward the multiplicative depth; however, they do consume noise budget. In our setting, the noise consumption is modest (less than 10 bits; see Section 7 for details) and we do not count it toward the multiplicative depth overhead of $\mathsf{Laminate}$.

4.2 Overview and Prerequisites

We begin with BhIPs for payload circuits that do not fully exploit the available SIMD parallelism. In Section 5, we generalize to *wide* payload circuits that utilize all N slots, including cross-slot operations.

Definition 4.3 (B-Laned Payload Circuit)**.** A payload circuit is *B-laned* (where B is a power of two dividing N) if it satisfies the following three constraints: (i) it uses only $B < N$ of the available SIMD slots for payload computation, (ii) it involves no cross-slot interactions (i.e., no rotation operations), and (iii) all plaintexts in the payload computation encode the same scalar in every slot.[20]

Together, these constraints mean that the B active slots execute the same single-slot arithmetic circuit independently, in SIMD parallel.

Motivation and high-level approach. Given a B-laned payload circuit, one could simply ignore the unused $N-B$ slots and apply one of the wide BhIPs from Section 5 to the B active slots. However, this wastes the available SIMD capacity: the server already holds N-slot ciphertexts, and the unused slots represent idle computational resources. Our approach instead exploits the full N-slot ciphertexts for efficient proof generation. After the server evaluates the payload circuit

[20] The name reflects the fact that the payload computation proceeds in B independent *lanes*, with no crossing between lanes. This is analogous to lanes on a highway.

(which uses only B slots per ciphertext), a *witness packing* step packs the intermediate ciphertext assignments densely into fewer ciphertexts that utilize all N slots. The BhIP then operates on these packed ciphertexts, amortizing proof generation across the full SIMD width.

Cost regime. In the B-laned setting with $B \ll N$, the cost of payload computation dominates the cost of proof generation. Each of the n payload FHE operations costs $\Omega(N)$ CPU operations regardless of B, and witness packing adds $O(n)$ additional FHE operations. Proof generation, operating on $O(nB/N)$ packed ciphertexts, is comparatively inexpensive. We therefore prioritize simplicity and clean structure in the proof generation phase, accepting minor overhead relative to a hypothetical optimally-tuned prover.

4.2.1 Payload Representation

During evaluation of the payload circuit, each ciphertext stores B meaningful values, replicated N/B times across all N SIMD slots. Concretely, a ciphertext encoding the vector $(x_0, x_1, \ldots, x_{B-1}) \in \mathbb{F}_q^B$ stores the N-tuple

$$(\underbrace{x_0, x_1, \ldots, x_{B-1}}_{B}, \underbrace{x_0, x_1, \ldots, x_{B-1}}_{B}, \ldots, \underbrace{x_0, x_1, \ldots, x_{B-1}}_{B})$$

with N/B identical copies. Since the payload circuit performs B independent slot-wise computations (no rotations), all intermediate ciphertexts maintain this replicated structure throughout the payload evaluation. The supported operations are *CT-CT addition*, *scalar-CT addition* (a special case of PT-CT addition, where the plaintext encodes a uniform scalar), *scalar-CT multiplication* (likewise, for multiplication), and *CT-CT multiplication*. All of these act slot-wise and preserve B-periodicity.

4.2.2 Dense Alternating Layered Arithmetization

We represent the B-laned payload circuit in a *dense alternating layered arithmetization* with $d = 2d_\times + 1$ layers.

Odd-indexed layers consist of SIMD multiplication gates and even-indexed layers consist of affine gates. **Multiplication gates** capture CT-CT multiplications. **Affine gates** capture the remaining slot-uniform operations: CT-CT addition, scalar-CT addition, and scalar-CT multiplication. Correspondingly, $d_\times$ counts the multiplicative depth of the payload circuit from CT-CT multiplications only.[21]

The distinguishing feature of the dense variant is a *dense wiring* constraint on the multiplication layers. That is, the first half of layer $2i$ provides the left inputs to layer $2i-1$ and the second half provides the right inputs. This is achieved by duplicating intermediate values in layer $2i$ where necessary, so that every multiplication gate has a private pair of input wires.[22]

Gate counts. Let n denote the total number of FHE operations performed during payload evaluation. Let $m := \sum_{i=1}^{d_\times} S_{2i-1}$ denote the total number of multiplication gates and $p := \sum_{i=0}^{d_\times} S_{2i}$ the total number of affine gates. Since each multiplication gate corresponds to exactly one CT-CT multiplication during payload evaluation, $m \leq n$. The dense wiring constraint enforces $S_{2i} = 2S_{2i-1}$ for each i. As a simplifying assumption (ignoring boundary effects at the input and output layers), we take $p = 2m$: each affine gate in an even layer serves as input to a unique multiplication gate in

[21] We do not count scalar-by-CT multiplication as part of the multiplicative depth of the payload circuit, but as we will see, we do count it as 0.5 depth during proof generation.

[22] In the regular (non-dense) GKR arithmetization, two multiplication gates could share an input wire. The dense arithmetization eliminates this sharing by copying the shared wire, resulting in $S_{2i} = 2S_{2i-1}$ for all $i \in \{1, \ldots, d_\times\}$.

the subsequent odd layer. The total gate count of the dense alternating arithmetization is therefore $m + p = 3m \leq 3n$.

Why dense wiring? The dense structure yields a particularly clean multiplication layer reduction (Section 4.4.1) in which the sumcheck sums over each input variable *once* rather than twice. This halves the number of sumcheck rounds compared to the non-dense case and simplifies witness packing (Section 4.3). In the $B \ll N$ regime where proof generation is not the bottleneck, the modest increase in circuit size from duplicating shared inputs to multiplication gates is an acceptable trade-off.

4.2.3 Slot-Independent Affine Gates

Because the B-laned payload circuit has no cross-slot operations, each of the B slots executes the same arithmetic circuit independently. This endows the affine gates with a strong structural property: the coefficients of a affine gate depend only on the *ciphertext indices*, not on the *slot index*.

Concretely, for an even layer $2i$, the coefficient function lin_{2i} that defines the linear combination has the factored form

$$\mathsf{lin}_{2i}(\vec{z}_c, \vec{z}_s, \vec{x}_c, \vec{x}_s) = \mathsf{lin}_{2i}(\vec{z}_c, \vec{x}_c) \cdot \mathbf{1}[\vec{z}_s = \vec{x}_s], \tag{4}$$

where $\mathsf{lin}_{2i}(\vec{z}_c, \vec{x}_c) : \{0,1\}^{s_{2i}} \times \{0,1\}^{s_{2i+1}} \to \mathbb{F}_q$ is a *scalar coefficient function* that captures the circuit wiring, and the indicator $\mathbf{1}[\vec{z}_s = \vec{x}_s]$ enforces that each slot's output depends only on the same slot's inputs. We overload notation: the 2-argument form $\mathsf{lin}_{2i}(\vec{z}_c, \vec{x}_c)$ denotes the scalar coefficient function, while the 4-argument form includes the slot-equality factor.

MLE factorization. The multilinear extension of the 4-argument coefficient function factors as:

$$\tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{Z}_S, \vec{X}_c, \vec{X}_s) = \tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{X}_c) \cdot \tilde{\mathsf{eq}}(\vec{Z}_S, \vec{X}_s), \tag{5}$$

where $\tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{X}_c)$ is the MLE of the 2-argument scalar coefficient function. An evaluation claim about $\tilde{\mathsf{lin}}_{2i}(\vec{\Gamma}_C, \vec{\Gamma}_S, \vec{R}_c, \vec{R}_s)$ reduces immediately to computing the $\tilde{\mathsf{eq}}(\vec{\Gamma}_S, \vec{R}_s)$ factor from public randomness (in $O(\log B)$ field operations) and verifying the opening of $\tilde{\mathsf{lin}}_{2i}(\vec{\Gamma}_C, \vec{R}_c)$ against the committed scalar coefficient polynomial.

4.3 Witness Packing

After evaluating the B-laned payload circuit, the server holds intermediate ciphertexts from each layer of the dense alternating arithmetization, each containing B meaningful values replicated N/B times.[23] Before proof generation, we pack each layer's ciphertexts into fewer, fully utilized N-slot ciphertexts via a *witness packing* step.

4.3.1 Packing Procedure

Fix a layer i with S_i intermediate ciphertexts, each storing B useful values. The total number of meaningful wire values in this layer is $S_i \cdot B$. We pack these into

$$S'_i := \left\lceil \frac{S_i \cdot B}{N} \right\rceil$$

[23] Technically, the server needs one more step to attain intermediate ciphertexts for the evaluation of the payload circuit in its dense alternating form. However, each of these intermediate ciphertexts are already computed when evaluating the payload circuit in its original form (and in particular the slot allocation is consistent between the two circuit representations, because of the laned structure), so this requires no further FHE operations.

packed ciphertexts, each utilizing all N slots.[24]

The packing groups the S_i original ciphertexts into S'_i groups, each containing N/B ciphertexts (the last group may be smaller). For each group, the jth ciphertext (for $j = 0, \ldots, N/B - 1$) is multiplied by a plaintext mask $\mathsf{pt}[\mathbf{1}_j]$ that preserves the jth chunk of B slots, positions jB through $(j+1)B - 1$, and zeroes out all other slots. Since the original ciphertexts have B-periodic structure (all N/B chunks store identical values), the target chunk already contains the correct values; no rotations are needed. The N/B masked ciphertexts are then summed to produce one packed ciphertext whose N slots are fully utilized.

4.3.2 Data Layout After Packing

After packing, each packed ciphertext stores N wire values organized in N/B contiguous *chunks* of B slots each. Chunk j (for $j \in \{0, \ldots, N/B - 1\}$) of a packed ciphertext contains the B values from the jth original ciphertext in the packing group. This gives a three-level address for each wire value:

- The *packed ciphertext index*: $s'_i := \log_2 S'_i$ bits.
- The *chunk index* within the packed ciphertext: $\log(N/B)$ bits, specifying which of the N/B original ciphertexts within this group the value came from.
- The *intra-chunk slot index*: $\log B$ bits, specifying which of the B independent computations this value belongs to.

The chunk index and intra-chunk slot index together form the $\log N$-bit slot index within the packed ciphertext. The total wire index is $t'_i := s'_i + \log N$ bits.

Remark 4.4 (Relationship between packed and original indices)**.** The packed ciphertext index (s'_i bits) and the chunk index ($\log(N/B)$ bits) together form the original ciphertext index (s_i bits). Indeed, $s'_i + \log(N/B) = \log\lceil S_i B/N \rceil + \log(N/B) = s_i$ (assuming exact divisibility for simplicity). Thus, the original ciphertext index is the concatenation of the packed ciphertext index and the chunk index.

Remark 4.5 (Packed layout and the dense arithmetization)**.** The dense wiring constraint ($S_{2i} = 2S_{2i-1}$) interacts cleanly with packing. For the multiplication layer $2i - 1$, the two inputs to gate j, wires j and $j + S_{2i-1}$ in layer $2i$, are packed into the *same* chunk in two packed ciphertexts. Concretely, if we view the s'_{2i}-bit packed ciphertext index of layer $2i$ as a pair $(\vec{b}, \vec{x}_c)$ where $\vec{b} \in \{0,1\}$ is the leading bit and $\vec{x}_c \in \{0,1\}^{s'_{2i}-1}$, then the two inputs to multiplication gate $(\vec{x}_c, \vec{q}, \vec{\ell})$ are at packed addresses $(0, \vec{x}_c, \vec{q}, \vec{\ell})$ and $(1, \vec{x}_c, \vec{q}, \vec{\ell})$. Here $\vec{q}$ is the chunk index and $\vec{\ell}$ is the intra-chunk slot index. In other words, the leading bit of the packed ciphertext index selects between the left and right inputs.

4.3.3 Witness Packing Cost

The packing procedure requires one PT-CT multiplication (masking) and one CT-CT addition per original ciphertext. Across all layers, the total cost is $3m$ PT-CT multiplications and $3m$ additions.[25] This is $O(m)$ FHE operations,the same order as the payload computation itself. Moreover, since

[24] We pad with zeros if $S_i \cdot B$ is not a multiple of N. For notational convenience, we assume S'_i is a power of two; otherwise, we round up to the next power of two and zero-pad.

[25] plus S_d for the input layer, which we ignore for convenient bookkeeping since it is small in practice.

payload computation requires relinearizing (and witness packing does not), witness packing will be concretely faster than payload computation.[26]

Remark 4.6 (Multiplicative depth of witness packing)**.** The masking step consumes one multiplicative level. For layers that serve as input to an affine layer reduction (odd-indexed layers), this masking can be *combined* with the $\widetilde{\mathsf{eq}}$ partial evaluation required during proof generation (Remark 4.7), so that both operations share a single PT-CT multiplication and hence a single multiplicative level. See Section 4.4.2 for details. The ciphertexts entering the proof generation phase have therefore consumed at most $d_\times + 1$ levels: $d_\times$ from the payload computation and 1 from packing.

4.4 BhIP for Laned Payload Circuits

We now present a BhIP for B-laned payload circuits, operating on the packed ciphertexts produced by witness packing (Section 4.3). Fix a dense alternating layered payload circuit with $d = 2d_\times + 1$ layers. After packing, layer i has S'_i packed ciphertexts, each with N utilized slots. The GKR protocol performs d layer reductions, alternating between two types:

- For each even layer $2i$ ($i \in \{0, \ldots, d_\times\}$): an affine layer reduction.
- For each odd layer $2i - 1$ ($i \in \{1, \ldots, d_\times\}$): a multiplication layer reduction.

As discussed in Remark 3.8, the layered structure assumption is made for clarity rather than necessity. To handle general (non-layered) circuits, one would extend the affine layer reduction so that each gate in layer $2i$ draws inputs from *all* layers below it (not just layer $2i + 1$), using the techniques of [85]. The resulting sumcheck structure is more involved but remains efficient; the multiplication layer reduction requires no modification, as multiplication gates already take inputs from the immediately preceding even layer. We present the layered version here because it admits a cleaner cost analysis and suffices for many circuits encountered in practice.

Notation recap. Let $s'_i := \log_2 S'_i$ and $t'_i := s'_i + \log N$. We use $s'_{\max} := \max_i\{s'_i\}$. The packed wire values in layer i are addressed by (t'_i)-bit indices $(\vec{x}_c, \vec{x}_s)$ where $\vec{x}_c \in \{0,1\}^{s'_i}$ is the packed ciphertext index and $\vec{x}_s \in \{0,1\}^{\log N}$ is the slot index within the packed ciphertext.

We decompose the slot index $\vec{x}_s = (\vec{x}_q, \vec{x}_\ell)$ into a *chunk index* $\vec{x}_q \in \{0,1\}^{\log(N/B)}$ and an *intra-chunk slot index* $\vec{x}_\ell \in \{0,1\}^{\log B}$, reflecting the packed data layout from Section 4.3.2. By Remark 4.4, the *original ciphertext index* is the concatenation $(\vec{x}_c, \vec{x}_q) \in \{0,1\}^{s_i}$.

4.4.1 Multiplication Layer Reduction

Fix an odd layer $2i - 1$ for $i \in \{1, \ldots, d_\times\}$. By the dense wiring constraint (Section 4.2.2), multiplication gate j in layer $2i - 1$ computes the slot-wise product of wires j and $j + S_{2i-1}$ from layer $2i$. In the packed representation (Remark 4.5), the leading bit $\vec{b}$ of the packed ciphertext index of layer $2i$ selects between the left ($\vec{b} = 0$) and right ($\vec{b} = 1$) inputs. Denoting the remaining bits of the layer-$2i$ packed ciphertext index as $\vec{x}_c \in \{0,1\}^{s'_{2i}-1} = \{0,1\}^{s'_{2i-1}}$, the assignment identity is:

$$L_{2i-1}(\vec{x}_c, \vec{x}_s) = L_{2i}(0, \vec{x}_c, \vec{x}_s) \cdot L_{2i}(1, \vec{x}_c, \vec{x}_s) \tag{6}$$

for all $\vec{x}_c \in \{0,1\}^{s'_{2i-1}}$ and $\vec{x}_s \in \{0,1\}^{\log N}$.

[26] Technically, witness packing will be concretely faster than the *variant of payload computation* where input ciphertexts are given more noise budget, to be able to compute a proof on the computation trace.

Multiplication Layer Reduction (Laned Payload)

1. **Full Sumcheck:** P and V engage in a sumcheck protocol for $s'_{2i-1} + \log N$ rounds to prove a claim of the form given in Eq. (7). At the end, V has challenges $\vec{r}_c \in \mathbb{F}_{q^R}^{s'_{2i-1}}$ and $\vec{r}_s \in \mathbb{F}_{q^R}^{\log N}$, and a final claim $c \in \mathbb{F}_{q^R}$:

$$c \stackrel{?}{=} \tilde{\mathsf{eq}}((\vec{r}_c, \vec{r}_s), (\vec{\gamma}_c, \vec{\gamma}_s)) \cdot \tilde{L}_{2i}(0, \vec{r}_c, \vec{r}_s) \cdot \tilde{L}_{2i}(1, \vec{r}_c, \vec{r}_s) \tag{8}$$

2. **Evaluation Claims:** P sends the claimed evaluations of the next layer:

$$v_0 \stackrel{?}{=} \tilde{L}_{2i}(0, \vec{r}_c, \vec{r}_s), \quad v_1 \stackrel{?}{=} \tilde{L}_{2i}(1, \vec{r}_c, \vec{r}_s)$$

3. **Consistency Check:** V checks if c is consistent with v_0, v_1 using Eq. (8). The $\tilde{\mathsf{eq}}$ factor is computed by V from public randomness.

4. **Batching:** V samples a random challenge $\alpha \in \mathbb{F}_{q^R}$. The reduction is complete, and the new claim for layer $2i$ is:

$$(1-\alpha)v_0 + \alpha v_1 \stackrel{?}{=} \tilde{L}_{2i}(\alpha, \vec{r}_c, \vec{r}_s) \tag{9}$$

Figure 4: Multiplication Layer $2i-1 \to 2i$ reduction for the hIP in the laned setting.

Multilinear evaluation for $\tilde{L}_{2i-1}$. Taking the MLE of Eq. (6) and summing over the Boolean hypercube:

$$\tilde{L}_{2i-1}(\vec{\gamma}_c, \vec{\gamma}_s) = \sum_{\vec{x}_c \in \{0,1\}^{s'_{2i-1}}} \sum_{\vec{x}_s \in \{0,1\}^{\log N}} \tilde{\mathsf{eq}}((\vec{x}_c, \vec{x}_s), (\vec{\gamma}_c, \vec{\gamma}_s)) \cdot \tilde{L}_{2i}(0, \vec{x}_c, \vec{x}_s) \cdot \tilde{L}_{2i}(1, \vec{x}_c, \vec{x}_s) \tag{7}$$

The prover and verifier engage in a sumcheck protocol for $s'_{2i-1} + \log N$ variables. The complete protocol is specified in Fig. 4.

4.4.2 Affine Layer Reduction

Fix an even layer $2i$ for $i \in \{0, \ldots, d_\times\}$. Each affine gate in layer $2i$ produces one output as an affine map (with scalar coefficients) of wires in layer $2i+1$. That is, every wire value in layer $2i$ is a linear combination of wire values in layer $2i+1$, plus an additive scalar constant.

Linear coefficients in the packed representation. By the slot-independence property (Eq. (4)), the linear coefficient function factors into a scalar coefficient and a slot-equality indicator. In the packed representation, this factorization carries over cleanly via the index correspondence from Remark 4.4. Recall that the original ciphertext index is the concatenation of the packed ciphertext index $\vec{x}_c$ and the chunk index $\vec{x}_q$. The packed linear coefficient function is:

$$\mathsf{lin}'_{2i}(\vec{z}_c, \vec{z}_q, \vec{z}_\ell, \vec{x}_c, \vec{x}_q, \vec{x}_\ell) = \mathsf{lin}_{2i}\big((\vec{z}_c, \vec{z}_q), (\vec{x}_c, \vec{x}_q)\big) \cdot \mathbf{1}[\vec{z}_\ell = \vec{x}_\ell], \tag{10}$$

where $\mathsf{lin}_{2i}((\vec{z}_c, \vec{z}_q), (\vec{x}_c, \vec{x}_q))$ is the original 2-argument scalar coefficient function evaluated at the original ciphertext indices, and the indicator enforces that only same-slot values interact.

MLE factorization in the packed representation. Taking the MLE of Eq. (10):

$$\tilde{\mathsf{lin}}'_{2i}(\vec{Z}_c, \vec{Z}_q, \vec{Z}_\ell, \vec{X}_c, \vec{X}_q, \vec{X}_\ell) = \tilde{\mathsf{lin}}_{2i}\big((\vec{Z}_c, \vec{Z}_q), (\vec{X}_c, \vec{X}_q)\big) \cdot \tilde{\mathsf{eq}}(\vec{Z}_\ell, \vec{X}_\ell). \tag{11}$$

The first factor is the MLE of the original scalar coefficient function (over the original ciphertext index variables), and the second is the standard $\tilde{\mathsf{eq}}$ over the $\log B$-bit intra-chunk indices. The

verifier can evaluate the $\tilde{\mathsf{eq}}(\vec{Z}_\ell, \vec{X}_\ell)$ factor from public randomness in $O(\log B)$ field operations. An evaluation claim about $\tilde{\mathsf{lin}}'_{2i}$ thus reduces to a single opening query against the committed scalar coefficient MLE $\tilde{\mathsf{lin}}_{2i}$, with no additional sumcheck rounds.

MLE evaluation for $\tilde{L}_{2i}$. For $(\vec{\gamma}_c, \vec{\gamma}_s) \in \mathbb{F}_{q^R}^{s'_{2i}+\log N}$, writing $\vec{\gamma}_s = (\vec{\gamma}_q, \vec{\gamma}_\ell)$, the MLE of the $2i$th packed layer evaluates as:

$$\tilde{L}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q, \vec{\gamma}_\ell) = \tilde{\mathsf{add}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q) + \sum_{\vec{x}_c \in \{0,1\}^{s'_{2i+1}}} \sum_{\vec{x}_q \in \{0,1\}^{\log(N/B)}} \sum_{\vec{x}_\ell \in \{0,1\}^{\log B}} g(x_c, x_q, x_\ell) \tag{12}$$

$$g(x_c, x_q, x_\ell) := \tilde{\mathsf{lin}}_{2i}\big((\vec{\gamma}_c, \vec{\gamma}_q), (\vec{x}_c, \vec{x}_q)\big) \cdot \tilde{\mathsf{eq}}(\vec{\gamma}_\ell, \vec{x}_\ell) \cdot \tilde{L}_{2i+1}(\vec{x}_c, \vec{x}_q, \vec{x}_\ell) \tag{13}$$

$$\tag{14}$$

where $\tilde{\mathsf{add}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q)$ is the MLE of the scalar-CT addition function, evaluated at the original ciphertext index $(\vec{\gamma}_c, \vec{\gamma}_q)$. It captures the additive scalar constant contributed to each output wire by scalar-CT additions in layer $2i$; since these scalars are slot-independent, the $\vec{x}_\ell$ summation separates out cleanly via $\sum_{\vec{x}_\ell} \tilde{\mathsf{eq}}(\vec{x}_\ell, \vec{\gamma}_\ell) = 1$, leaving $\tilde{\mathsf{add}}_{2i}$ depending only on the original ciphertext index. The remaining sum is expanded using $\tilde{\mathsf{lin}}'_{2i}$ and the factorization from Eq. (11), and is over $s'_{2i+1} + \log(N/B) + \log B = s'_{2i+1} + \log N$ variables. Note that $\tilde{\mathsf{lin}}_{2i}$ does not depend on $\vec{x}_\ell$, and $\tilde{\mathsf{eq}}(\vec{\gamma}_\ell, \vec{x}_\ell)$ does not depend on $(\vec{x}_c, \vec{x}_q)$.

Remark 4.7 (Combined packing and $\tilde{\mathsf{eq}}$ weighting)**.** Before entering the sumcheck, the prover precomputes the product $\tilde{\mathsf{eq}}(\vec{x}_\ell, \vec{\gamma}_\ell) \cdot L_{2i+1}(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$ for all Boolean $(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$, merging this with the witness packing masking (Section 4.3.1). For each of the S_{2i+1} original ciphertexts in layer $2i+1$ (which store B-periodic data), the prover multiplies by a single plaintext $\mathsf{pt}[\mathbf{w}_j]$ whose slot at position $jB+\ell$ is $\tilde{\mathsf{eq}}(\vec{\ell}, \vec{\gamma}_\ell)$ (where j is the ciphertext's assigned chunk index within its packing group) and whose remaining slots are zero. This single PT-CT multiplication simultaneously (i) selects the appropriate chunk, zeroing out duplicates (masking), and (ii) weights each intra-chunk slot ℓ by $\tilde{\mathsf{eq}}(\vec{\ell}, \vec{\gamma}_\ell)$. The N/B masked-and-weighted ciphertexts in each packing group are then summed to produce S'_{2i+1} packed ciphertexts, each storing N weighted values organized in N/B chunks of B slots. The intra-chunk slot variables $\vec{x}_\ell$ are *not* collapsed at this stage; they remain as sumcheck variables in Eq. (12).

Since the challenge $\vec{\gamma}_\ell$ is not known until proof generation, the packing of odd-indexed layers is performed *lazily* as the first step of each affine layer reduction, rather than during a separate witness packing phase. By combining masking and $\tilde{\mathsf{eq}}$ weighting into a single PT-CT multiplication, this precomputation consumes only one multiplicative level, the same level that standard packing would use, rather than the two levels that separate masking and $\tilde{\mathsf{eq}}$ weighting would require.

The prover and verifier engage in $s'_{2i+1} + \log N$ sumcheck rounds over the variables $(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$ of layer $2i+1$. After the precomputation (Remark 4.7), the sumcheck operates on packed ciphertexts storing $\tilde{\mathsf{eq}}$-weighted values across all N slots. The protocol is specified in Fig. 5.

Remark 4.8 (Virtual polynomial consistency check)**.** In both protocols, the verifier's consistency check requires evaluating the packed linear coefficient MLE $\tilde{\mathsf{lin}}'_{2i}$ at challenge points. By the factorization in Eq. (11), this reduces to: (i) evaluating $\tilde{\mathsf{eq}}(\vec{\gamma}_\ell, \vec{R}_\ell)$ from public randomness in $O(\log B)$ field operations, and (ii) verifying $\tilde{\mathsf{lin}}_{2i}((\vec{\gamma}_c, \vec{\gamma}_q), (\vec{R}_c, \vec{R}_q))$ against the committed scalar coefficient polynomial. No additional sumcheck is needed for this reduction.

Affine Layer Reduction (Laned Payload)

1. **Precomputation:** P computes S'_{2i+1} packed ciphertexts encoding $\tilde{\mathsf{eq}}(\vec{x}_\ell, \vec{\gamma}_\ell) \cdot L_{2i+1}(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$ for all Boolean $(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$ via the combined packing and $\tilde{\mathsf{eq}}$ weighting described in Remark 4.7.

2. **Full Sumcheck:** V computes $\tilde{\mathsf{add}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q)$ from the public circuit description and subtracts it from the current claim. P and V then engage in a sumcheck protocol for $s'_{2i+1} + \log N$ rounds proving the adjusted claim $\tilde{L}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q, \vec{\gamma}_\ell) - \tilde{\mathsf{add}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q)$ equals the $\tilde{\mathsf{lin}}_{2i}$ sum in Eq. (12). At the end, V has challenges $\vec{r}_c \in \mathbb{F}_{q^R}^{s'_{2i+1}}$, $\vec{r}_q \in \mathbb{F}_{q^R}^{\log(N/B)}$, and $\vec{r}_\ell \in \mathbb{F}_{q^R}^{\log B}$, and a final claim $c \in \mathbb{F}_{q^R}$:

$$c \stackrel{?}{=} \tilde{\mathsf{lin}}_{2i}\big((\vec{\gamma}_c, \vec{\gamma}_q), (\vec{r}_c, \vec{r}_q)\big) \cdot \tilde{\mathsf{eq}}(\vec{\gamma}_\ell, \vec{r}_\ell) \cdot \tilde{L}_{2i+1}(\vec{r}_c, \vec{r}_q, \vec{r}_\ell) \tag{15}$$

3. **Evaluation Claim:** P sends the claimed evaluation of the next layer:

$$v_x \stackrel{?}{=} \tilde{L}_{2i+1}(\vec{r}_c, \vec{r}_q, \vec{r}_\ell)$$

4. **Consistency Check:** V checks if c is consistent with v_x using Eq. (15). The $\tilde{\mathsf{lin}}_{2i}$ factor is verified against the committed scalar coefficient polynomial (see Eq. (11)). The $\tilde{\mathsf{eq}}(\vec{\gamma}_\ell, \vec{r}_\ell)$ factor is computed from public randomness.

5. **Reduction:** Setting $\vec{r}_s := (\vec{r}_q, \vec{r}_\ell)$, the new claim for layer $2i+1$ is $v_x \stackrel{?}{=} \tilde{L}_{2i+1}(\vec{r}_c, \vec{r}_s)$.

Figure 5: Affine Layer $2i \to 2i+1$ reduction for the hIP in the laned setting.

4.4.3 Operation Counts

We now analyze the FHE operation counts of the laned BhIP provers. The packed circuit parameters are $S'_i = \lceil S_i B/N \rceil$, $s'_i = \log_2 S'_i$, and $t'_i = s'_i + \log N$. Let $m' := \sum_{i=1}^{d_\times} S'_{2i-1}$ and $p' := \sum_{i=0}^{d_\times} S'_{2i}$ denote the packed gate counts. By $p = 2m$ and the packing relation, $p' \le 2m' + 1$, and the total packed gate count satisfies $m' + p' \le 3mB/N + O(d)$. Since $m \le n$, the packed circuit has at most $O(nB/N)$ gates (ignoring lower-order terms in d).

The detailed derivation of FHE operation counts for both layer reduction types is given in Appendix E.3. The multiplicative depth analysis is presented in Lemma E.13.

Theorem 4.9. *Suppose plaintext field order q and cryptographic extension degree R are such that there exists a multiplicative depth 1 $\mathbb{F}_q$-circuit to compute $\mathbb{F}_{q^R}$ multiplication.*[27] *Then the BhIP prover for dense alternating layered laned payload circuits with m multiplication gates takes as input the intermediate ciphertexts arising from payload computation, requiring 2.5 levels of additional noise budget, and has homomorphic operation count upper bounded by Table 6.*

4.5 Summary

End-to-end workflow. For a B-laned payload circuit with n FHE operations and $m \le n$ CT-CT multiplications (so that the dense alternating arithmetization has $3m$ gates):

1. **Payload evaluation:** The prover evaluates the payload circuit using B-laned ciphertexts (each with N/B replicas). Cost: n FHE operations, $d_\times$ multiplicative levels.

2. **Witness packing:** The prover packs the intermediate ciphertexts into $O(mB/N)$ packed N-slot ciphertexts. For even-indexed layers (multiplication layer inputs), this is standard masking

[27] See Remark 7.1 for a discussion about this assumption.

Operation	Level	Total Count Bound
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$2.5 \to 2.0$	$3m' \cdot s'_{\max}$
$\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$	$2.0 \to 1.0$	$2m'$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$1.5 \to 1.0$	$1.5m' \cdot s'_{\max}$
$\mathsf{ept} \cdot \mathsf{ect} \to \mathsf{ect}$	$1.0 \to 0.0$	$4m'$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	1.0	$2m'$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	0.0	$2m'$
FOLD	0.0	$2d(s'_{\max} + \log N)$
Total FHE Depth		**2.5 Levels**

Table 6: Total FHE Cost for the Laned BhIP Prover on dense alternating layered B-laned payload circuits. Here $m' = \sum S'_{2i-1} \le mB/N$ is the packed multiplication gate count, $s'_{\max} = \max_i\{s'_i\}$, and $d = 2d_\times + 1$. We bound the packed odd-layer count by m' (ignoring the negligible packed input layer size); see Appendix E.3.5 for details. Witness packing costs ($3m$ PT-CT multiplications and $3m$ additions) are not included. The FOLD operator introduces no additional multiplicative depth.

and accumulation. For odd-indexed layers (affine layer inputs), the masking is combined with $\tilde{\mathsf{eq}}$ partial evaluation during proof generation (Remark 4.7).

3. **Proof generation:** The prover runs the BhIP on the packed ciphertexts.

The $B = 1$ special case and relation to prior work. When $B = 1$, the laned setting reduces to verifying a single-slot arithmetic circuit evaluated under FHE, precisely the setting considered by prior works such as Blind zkSNARKs [40] and Phalanx [86]. These works also separate the cost of payload evaluation from the cost of proof generation, typically omitting witness packing costs from their accounting. Our framework identifies two distinct witness packing costs: (i) emulating non-native field arithmetic (which is not an issue for us, since the GKR protocol is field-agnostic and operates natively over $\mathbb{F}_q$), and (ii) packing the intermediate ciphertexts so that the prover can exploit the full SIMD width. For small values of B, the cost of our BhIP's witness packing and proof generation is a small fraction of the cost to evaluate the payload.

5 General Case: BhIPs for Wide Payload Circuits

5.1 Overview and Prerequisites

We now present BhIPs for *wide* payload circuits. The common BhIP machinery (ciphertext/plaintext notation, the FOLD operator, and the convention on conservative operation count bounds) was established in Section 4.1.

Definition 5.1 (Wide Payload Circuit)**.** A payload circuit is *wide* if it operates on all N SIMD slots of each ciphertext. Wide payload circuits may include arbitrary cross-slot operations such as rotations; no structural restrictions on the interaction pattern between slots are imposed.[28]

Remark 5.2. Every payload circuit can be viewed as a wide circuit by treating unused slots as carrying dummy zero values. In particular, a B-laned payload circuit (Definition 4.3) is a special case of a wide circuit in which the N slots decompose into B independent lanes with no cross-slot interactions. The BhIPs presented in this section apply to the general wide setting; for laned payloads, the specialized (and more efficient) constructions in Section 4 exploit the additional structure.

[28]The term "wide" reflects the fact that the payload computation spans the full width of the ciphertext (all N slots), but not necessarily in a laned fashion, as cross-slot interactions are permitted.

The base vs. folded paradigm. For the wide setting, we distinguish between two strategies for the prover, which trade off homomorphic noise growth against communication bandwidth:

- **Base hIP:** The prover minimizes noise consumption by avoiding homomorphic rotations and multiplications where possible. Instead of aggregating sumcheck polynomials across SIMD slots homomorphically, the prover sends distinct polynomials for every slot. The verifier aggregates these plaintext polynomials locally. This results in lower prover noise but higher communication ($N\times$).
- **Folded hIP:** The prover prioritizes succinctness. It uses homomorphic rotations (via the FOLD operator) to aggregate sumcheck polynomials across slots into a single slot. This minimizes communication to standard GKR levels but consumes additional noise budget (approx. 1 level + $\log N$ rotations).

We present a table of BGV operation counts for both of the BhIP provers whose full derivations are detailed in Appendix E. We begin with prerequisites specific to the wide setting: the class of supported payload circuits, data layout, and the rotational symmetry property of affine gates.

5.1.1 Class of Supported Payload Circuits

The payload circuit is computed using six types of FHE operations: *CT-CT addition*, *PT-CT addition*, *PT-CT multiplication*, *scalar-CT multiplication*, *CT rotation*, and *CT-CT multiplication*.

Our arithmetization of such a circuit, used by the proving stage, represents it as an alternating layered circuit with $d = 2d_\times + 1$ layers, where $d_\times$ is the multiplicative depth. This arithmetization has two gate types: *affine gates* (even layers) and *SIMD multiplication gates* (odd layers). Layer 0 (the output layer) and all even-indexed layers consist of affine gates; all odd-indexed layers consist solely of SIMD multiplication gates.

Affine gates. We generalize the affine gates from Section 4.2.3 to support all affine maps of input wires that exhibit a certain kind of *rotational symmetry* (see Section 5.1.3): if linear combination coefficients for slot 0 of the output are known, then the coefficients for all other slots are determined. These generalized affine gates can express rotations (as well as CT-CT additions, PT-CT additions, scalar-CT multiplications), i.e., all FHE operations except CT-CT multiplication and PT-CT multiplication.

SIMD multiplication gates. SIMD multiplication gates are also generalized from Section 4.2.2, and now also support *PT-CT multiplication gates* (as well as CT-CT multiplication gates). A CT-CT multiplication gate takes as input two ciphertexts and produces one output ciphertext by performing a slot-wise (Hadamard) CT-CT multiplication. A PT-CT multiplication gate takes as input one ciphertext and produces one output ciphertext by performing a slot-wise (Hadamard) PT-CT multiplication with a fixed plaintext.

As discussed in Remark 3.8, the layered structure assumption is made for clarity rather than necessity. To handle general (non-layered) circuits, one would extend both the affine layer reduction and the multiplication layer reduction so that gates in each layer draw inputs from *all* layers below, using the techniques of [85]. The resulting sumcheck structure involves extended wiring predicates that sum over cross-layer connections, but remains efficient and uses only standard sumcheck techniques. We present the layered version here because it admits a cleaner cost analysis and suffices for the circuits encountered in practice.

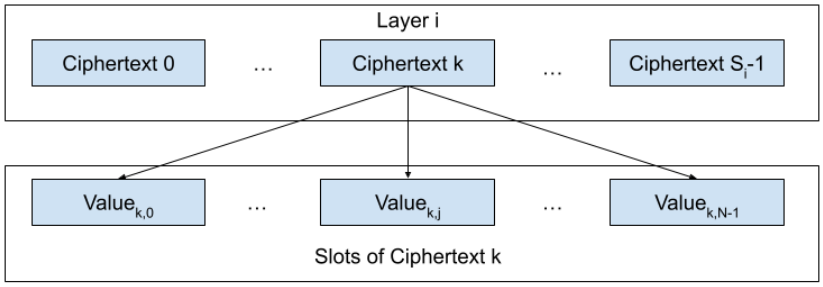

Figure 7: Data layout for the ith layer of a payload circuit. The first s_i bits determine which ciphertext ct holds the value, while the last $\log N$ bits determine the SIMD slot.

5.1.2 Data Layout and Notation

The payload circuit is represented in d layers, with L_0 denoting the output layer, and L_d denoting the input layer. Each layer has S_i gates. The supported gate types are **Input**, **Affine** (for even layers), and **SIMD Multiplication** (for odd layers). We assume that S_i is a power of two and $s_i := \log_2(S_i)$. We use $T_i = S_i \cdot N$ to denote the number of wires in the global $\mathbb{F}_q$-arithmetic circuit and $t_i := \log_2(T_i) = s_i + \log N$. Furthermore, we let n denote the number of FHE operations required to compute the payload circuit. In our alternating layered arithmetization, we let $m := \sum_{i=1}^{d_\times} S_{2i-1}$ denote the total number of SIMD multiplication gates and $p := \sum_{i=0}^{d_\times} S_{2i}$ denote the total number of affine gates (including the output layer). Some useful bounds are $p \leq 2m$, and $m + p \leq n$ (see Section 5.2). When obvious from context, we drop subscripts on S_i, T_i, s_i, t_i.

After homomorphic payload evaluation, the server has $m+p+S_d$ ciphertexts (from the alternating layered arithmetization) that arose as intermediate assignments when calculating the output.[29] In a particular layer whose assignment has S ciphertexts, the assignment has T wire values. We index the wires with length t bitstrings according to the following convention:

- The most significant s bits describe the *ciphertext index*.
- The least significant $\log N$ bits describe the *slot index*
- The $t = s + \log N$ bits together describe the overall *index*.

This data layout is depicted below in Fig. 7.

We use the ct, ect, pt, ept, mask, and emask notation introduced in Section 4.1.1. In the context of the data layout above, $\mathsf{ct}[x]$ denotes the ciphertext at ciphertext index x, with its N slots indexed by $\{0,1\}^{\log N}$.

5.1.3 SIMD Rotational Symmetry of Affine Gates

We now describe a key structural property of affine gates that arises from the SIMD structure of the payload circuit.

[29] Technically, the server needs one more step to attain intermediate ciphertexts for the evaluation of the payload circuit in its alternating form. However, these intermediate ciphertexts are a subset of those already computed when evaluating the payload circuit in its original form. In particular, the intermediate ciphertexts of the alternating form are precisely those that are inputs or outputs to SIMD multiplication gates in the original form. So, this step requires no further FHE operations.

Recall that each affine gate z_c in an even layer $2i$ produces one output ciphertext whose slot values are determined by a linear combination of all wires in layer $2i+1$, with scalar coefficients, plus a constant term (accounting for addition with a plaintext). We define the coefficient function $\mathsf{lin}_{2i} : \{0,1\}^{s_{2i}} \times \{0,1\}^{\log N} \times \{0,1\}^{s_{2i+1}} \times \{0,1\}^{\log N} \to \mathbb{F}_q$ as follows:

$$\mathsf{lin}_{2i}(\vec{z}_c, \vec{z}_s, \vec{x}_c, \vec{x}_s) := \text{coefficient of wire } (\vec{x}_c, \vec{x}_s) \text{ in the linear combination defining wire } L_{2i}(\vec{z}_c, \vec{z}_s).$$

Because the payload circuit has SIMD structure, the coefficient function exhibits a *rotational symmetry* across SIMD slots:

$$\mathsf{lin}_{2i}(\vec{z}_c, \vec{z}_s + \vec{o} \bmod N, \vec{x}_c, \vec{x}_s + \vec{o} \bmod N) = \mathsf{lin}_{2i}(\vec{z}_c, \vec{z}_s, \vec{x}_c, \vec{x}_s) \tag{16}$$

for all offsets $\vec{o} \in \{0,1\}^{\log N}$, where addition is performed modulo N on the integer representations of the bit-vectors. Intuitively, the linear combination that computes slot z_s of output ciphertext z_c uses the same coefficients as the one for slot 0, but applied to cyclically shifted input slots.

This symmetry means that each affine gate is fully specified by its coefficients for a single reference slot. Setting $\vec{z}_s = \vec{0}$ in Eq. (16), we define the *reduced coefficient function*:

$$\mathsf{lin}_{2i}(\vec{z}_c, \vec{x}_c, \vec{x}_s) := \mathsf{lin}_{2i}(\vec{z}_c, \vec{0}, \vec{x}_c, \vec{x}_s),$$

with domain $\{0,1\}^{s_{2i}} \times \{0,1\}^{s_{2i+1}} \times \{0,1\}^{\log N} \to \mathbb{F}_q$. We overload lin by arity: the 4-argument form is the full coefficient function, while the 3-argument form is the reduced (slot-0) version. The full function is recovered via $\mathsf{lin}_{2i}(\vec{z}_c, \vec{z}_s, \vec{x}_c, \vec{x}_s) = \mathsf{lin}_{2i}(\vec{z}_c, \vec{x}_c, (\vec{x}_s - \vec{z}_s) \bmod N)$.

Virtual polynomial interpretation. We treat the multilinear extension $\tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{Z}_S, \vec{X}_c, \vec{X}_s)$ of the full 4-argument coefficient function as a *virtual polynomial*: rather than committing to it directly, we commit only to $\tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{X}_c, \vec{X}_s)$, the MLE of the reduced 3-argument function. Thus, during preprocessing, it suffices to commit to the reduced coefficient functions alone. Since the total support across all affine gates is at most $n - m$, a suitable polynomial commitment scheme (e.g. Sona [75]) allows the prover to produce opening proofs in $\tilde{O}(n)$ field operations outside FHE. This cost is a negligible fraction of the VCoED server's overall work, which requires $\Omega(n)$ FHE operations, each costing $\Omega(N)$ CPU operations.

The two are related by the following decomposition:

$$\tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{Z}_S, \vec{X}_c, \vec{X}_s) = \sum_{\vec{d} \in \{0,1\}^{\log N}} \tilde{\mathsf{shift}}_N(\vec{Z}_S, \vec{X}_s, \vec{d}) \cdot \tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{X}_c, \vec{d}) \tag{17}$$

where $\tilde{\mathsf{shift}}_N$ is the multilinear extension of the function $\mathsf{shift}_N(\vec{z}_s, \vec{x}_s, \vec{d}) = \mathbf{1}[\vec{x}_s = (\vec{z}_s + \vec{d}) \bmod N]$ encoding modular addition on $\log N$-bit integers.

An evaluation claim about the full $\tilde{\mathsf{lin}}_{2i}$ at a point $(\vec{\Gamma}_C, \vec{\Gamma}_S, \vec{R}_c, \vec{R}_s)$ can therefore be reduced to an evaluation claim about the reduced $\tilde{\mathsf{lin}}_{2i}$ via an additional $\log N$-round sumcheck (outside FHE) on Eq. (17). The verifier computes the shift coefficients $\tilde{\mathsf{shift}}_N(\vec{\Gamma}_S, \vec{R}_s, \vec{d})$ from public randomness, and the sumcheck reduces to a single evaluation query $\tilde{\mathsf{lin}}_{2i}(\vec{\Gamma}_C, \vec{X}_c, \vec{r}_d)$ for a challenge $\vec{r}_d \in \mathbb{F}_{q^R}^{\log N}$.

MLE-structured evaluation of $\tilde{\mathsf{shift}}_N$.

Theorem 5.3. *The function* $\mathsf{shift}_N(\vec{z}, \vec{x}, \vec{o}) = \mathbf{1}[\{z\} = \{x\} + \{o\} \bmod N]$ *is* MLE-structured *in the sense of [75]: its multilinear extension* $\tilde{\mathsf{shift}}_N(\vec{Z}, \vec{X}, \vec{O})$ *can be evaluated at any point in* $O(\log N)$ *field operations, rather than the* $O(N^2)$ *cost of naive Lagrange interpolation.*

The proof, given in Appendix B, decomposes modular addition via a binary carry chain and evaluates the resulting multilinear extension through a simple $O(\log N)$-step dynamic program over a 2-element state vector. Consequently, the verifier can evaluate $\tilde{\mathsf{shift}}_N$ at random challenge points in $O(\log N)$ time, making both the virtual polynomial consistency check and the $\log N$-round sumcheck reduction efficient.

5.2 BhIPs for Wide Payload Circuits

We now present the BhIPs for wide payload circuits. Recall from Section 5.1.1 that we assume the payload circuit has $d = 2d_\times + 1$ layers with an alternating structure: even layers consist of affine gates, and odd layers consist of SIMD multiplication gates.

This structure is natural. For any circuit with multiplicative depth $d_\times$, there is an equivalent alternating layered circuit with $d = 2d_\times + 1$ layers: the $d_\times$ odd layers handle all multiplications, while the $d_\times + 1$ even layers handle all linear operations (CT-CT additions, CT-PT additions, CT-PT multiplications, CT rotations, and identity forwarding) via affine gates. The SIMD multiplication gates in the alternating circuit correspond exactly to the SIMD multiplications performed during the original payload computation, so the number of multiplication gates m satisfies $m \leq n$. Furthermore, each affine gate below the output layer serves as an input to a multiplication gate in the layer above, so $p \leq 2m$. Since each gate in the alternating circuit accounts for at least one FHE operation in the original payload computation, we also have $m + p \leq n$. For payload circuits intended to be evaluated under FHE *without bootstrapping*, we expect $d_\times$ to be bounded above by some reasonable constant, and not growing with circuit size n.

Notation recap We let T_i denote the number of wires in the ith layer of Ckt, and let $S_i := \frac{T_i}{N}$ denote the number of ciphertexts required to represent the ith layer. As usual, we let $s_i := \log_2(S_i)$ and $t_i := \log_2(T_i) = s_i + \log N$. Following Section 5.1.2, n denotes the number of FHE operations to compute the payload, $m := \sum_{i=1}^{d_\times} S_{2i-1}$ the total number of multiplication gates, and $p := \sum_{i=0}^{d_\times} S_{2i}$ the total number of affine gates. We let $s_{\max} := \max_i\{s_i\}$.

5.2.1 Alternating GKR hIP Description

Fix an alternating layered payload circuit Ckt with $d = 2d_\times + 1$ layers. The GKR protocol performs d layer reductions, reducing a claim about L_0 (the output layer) to L_d (the input layer). The layer reductions alternate between two types:

- For each even layer $2i$ ($i \in \{0, \ldots, d_\times\}$), we use an affine layer reduction.
- For each odd layer $2i - 1$ ($i \in \{1, \ldots, d_\times\}$), we use a multiplication layer reduction.

We now describe the layer reduction protocols, specifying both the base and folded hIP variants.

Affine layer $2i \to 2i+1$ reduction. Fix an even layer $2i$ for $i \in \{0, \ldots, d_\times\}$. Each affine gate z_c in layer $2i$ produces one output ciphertext as a linear combination (with scalar coefficients) of wires in layer $2i+1$ plus a constant term. The coefficient function $\mathsf{lin}_{2i}(\vec{z}_c, \vec{z}_s, \vec{x}_c, \vec{x}_s)$ and its reduced form $\mathsf{lin}_{2i}(\vec{z}_c, \vec{x}_c, \vec{x}_s)$ are defined in Section 5.1.3; we use $\tilde{\mathsf{lin}}_{2i}$ to denote the MLE of the full 4-argument function (treated as a virtual polynomial).[30] For $(\vec{\gamma}_c, \vec{\gamma}_s) \in \mathbb{F}_{q^R}^{s_{2i} + \log N}$, evaluation of the $2i$th layer MLE can be written as the following sum with respect to layer $2i+1$:

$$\tilde{L}_{2i}(\vec{\gamma}_c, \vec{\gamma}_s) - \tilde{\mathsf{add}}(\vec{\gamma}_c, \vec{\gamma}_s) = \sum_{\vec{x}_c \in \{0,1\}^{s_{2i+1}}} \sum_{\vec{x}_s \in \{0,1\}^{\log N}} \tilde{\mathsf{lin}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_s, \vec{x}_c, \vec{x}_s) \cdot \tilde{L}_{2i+1}(\vec{x}_c, \vec{x}_s) \tag{18}$$

Folded hIP. In the folded hIP, the prover and verifier engage in all $t_{2i+1} = s_{2i+1} + \log N$ sumcheck rounds. This process fully reduces the claim about layer $2i$ to a single evaluation claim about layer $2i+1$. The complete protocol is specified in Figure 8.

[30] A virtual polynomial is one is not directly committed to. For each virtual polynomial, the prover and verifier agree on a (possibly interactive) procedure to reduce evaluation claims about virtual polynomials to evaluation claims about committed or known polynomials. This technique is defined formally in [33].

Folded Affine Layer Reduction

1. **Full Sumcheck:** P and V engage in a sumcheck protocol for t_{2i+1} rounds to prove a claim of the form given in Eq. (18). At the end, V has challenges $\vec{r}_x \in \mathbb{F}_{q^R}^{s_{2i+1}}$ and $\vec{r}_l \in \mathbb{F}_{q^R}^{\log N}$, and a final claim $c \in \mathbb{F}_{q^R}$:

$$c \stackrel{?}{=} \tilde{\mathsf{lin}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_s, \vec{r}_x, \vec{r}_l) \cdot \tilde{L}_{2i+1}(\vec{r}_x, \vec{r}_l) \tag{19}$$

2. **Evaluation Claim:** P sends the claimed evaluation of the next layer:

$$v_x \stackrel{?}{=} \tilde{L}_{2i+1}(\vec{r}_x, \vec{r}_l)$$

3. **Consistency Check:** V checks if c is consistent with v_x using Eq. (19).
4. **Reduction:** The reduction is complete. The verifier now waits to be convinced of the claim $v_x \stackrel{?}{=} \tilde{L}_{2i+1}(\vec{r}_x, \vec{r}_l)$ in the next step.

Figure 8: Full specification of a Affine Layer $2i \to 2i+1$ reduction for the folded hIP.

Base hIP. In the base hIP, the prover and verifier engage in only s_{2i+1} rounds of sumcheck, leaving the sum over $\log N$ slot bits for the verifier to handle. Additionally, the prover sends per-slot round polynomials rather than aggregating them. The detailed protocol is specified in Figure 9.

Remark 5.4 (Virtual polynomial consistency check)**.** In both protocols, the verifier's consistency check requires evaluating the virtual polynomial $\tilde{\mathsf{lin}}_{2i}$ at challenge points involving non-Boolean slot coordinates $\vec{\gamma}_s$. These evaluations are reduced to queries against the committed reduced MLE $\tilde{\mathsf{lin}}_{2i}(\vec{Z}_C, \vec{X}_c, \vec{X}_s)$ via the decomposition in Eq. (17), using an additional $\log N$-round sumcheck outside of FHE. In this sumcheck, the verifier needs to evaluate $\tilde{\mathsf{shift}}_N$ at random challenge points; this can be done in $O(\log N)$ field operations using the carry-chain state iteration (Eq. (35)).

Multiplication layer $2i-1 \to 2i$ reduction. Fix an odd layer $2i-1$ for $i \in \{1, \ldots, d_\times\}$. We are promised that the layer $2i \to 2i-1$ computation consists only of SIMD multiplication operations, which may be either CT-CT or PT-CT multiplications. We therefore restrict the domain of the indicator functions to take labels that indicate *ciphertext indices* rather than wire indices (which would include a ciphertext index and a slot index). See Section 5.1.2.

We define two indicator functions for this layer:

- $\mathsf{mul}_{2i-1} : \{0,1\}^{s_{2i-1}} \times \{0,1\}^{s_{2i}} \times \{0,1\}^{s_{2i}} \to \{0,1\}$ specifies whether a particular ciphertext in layer $2i-1$ is the result of a *CT-CT* SIMD-multiplication of two particular ciphertexts in layer $2i$.
- $\mathsf{pmul}_{2i-1} : \{0,1\}^{s_{2i-1}} \times \{0,1\}^{\log N} \times \{0,1\}^{s_{2i}} \times \{0,1\}^{\log N} \to \{0,1\}$ specifies whether a particular *wire* (z_c, z_s) in layer $2i-1$ is the result of a *PT-CT* multiplication, with coefficient given by $\mathsf{pmul}_{2i-1}(z_c, z_s, x_c, x_s)$ for wire (x_c, x_s) in layer $2i$. For any (z_c, z_s) in the hypercube, the support of $\mathsf{pmul}_{2i-1}(z_c, z_s, \cdot, \cdot)$ is exactly one wire.

Multilinear evaluation for $\tilde{L}_{2i-1}$. Exploiting this, evaluation of the $(2i-1)$th layer MLE is

Base Affine Layer Reduction

1. **Sumcheck (Early Termination):** P and V engage in sumcheck for s_{2i+1} rounds to prove a claim of the form given in Eq. (18). In every round j, P sends a vector of round polynomials $\mathsf{ept}[R_j]$ (i.e., N polynomials $\{R_{j,\vec{\ell}}(X)\}_{\vec{\ell}\in\{0,1\}^{\log N}}$). Fixing a specific slot index $\vec{\ell}\in\{0,1\}^{\log N}$, the round polynomial $R_{j,\vec{\ell}}(X)$ is equivalent to the jth round polynomial of the following sumcheck instance:

$$\tilde{L}_{2i}(\vec{\gamma}_c,\vec{\ell}\,)-\tilde{\mathsf{add}}(\vec{\gamma}_c,\vec{\ell})=\sum_{\vec{x}_c\in\{0,1\}^{s_{2i+1}}}\tilde{\mathsf{lin}}_{2i}(\vec{\gamma}_c,\vec{\gamma}_s,\vec{x}_c,\vec{\ell})\cdot\tilde{L}_{2i+1}(\vec{x}_c,\vec{\ell})$$

   To learn $R_j(X)$, V aggregates the slot polynomials (note that unlike the multiplication layer, there is no $\tilde{\mathsf{eq}}$ term here):

$$R_j(X)=\sum_{\vec{\ell}\in\{0,1\}^{\log N}}R_{j,\vec{\ell}}(X)$$

2. **Final Sumcheck Claim:** After s_{2i+1} rounds, with challenges $\vec{r}_x\in\mathbb{F}_{q^R}^{s_{2i+1}}$, V holds a claim $c\in\mathbb{F}_{q^R}$ that still sums over the slot indices $\vec{\ell}$:

$$c\stackrel{?}{=}\sum_{\vec{\ell}\in\{0,1\}^{\log N}}\tilde{\mathsf{lin}}_{2i}(\vec{\gamma}_c,\vec{\gamma}_s,\vec{r}_x,\vec{\ell})\cdot\tilde{L}_{2i+1}(\vec{r}_x,\vec{\ell}) \tag{20}$$

3. **Vector Evaluation Claim:** P sends a vector of N scalar evaluation claims $\mathsf{ept}[w_x]$ such that for all $\vec{\ell}\in\{0,1\}^{\log N}$:

$$w_{x,\vec{\ell}}\stackrel{?}{=}\tilde{L}_{2i+1}(\vec{r}_x,\vec{\ell})$$

4. **Consistency Check:** V uses the vector $\mathsf{ept}[w_x]$ to evaluate the RHS of Eq. (20) and checks if it equals c.

5. **Sampling and Reduction:** V samples slot challenges $\vec{r}_l\sim\mathbb{F}_{q^R}^{\log N}$. V evaluates the multilinear extension of the claim vector to get a single point:

$$v_x\leftarrow\tilde{w}_x(\vec{r}_l)$$

   The verifier now waits to be convinced of the claim $v_x\stackrel{?}{=}\tilde{L}_{2i+1}(\vec{r}_x,\vec{r}_l)$.

Figure 9: Full specification of a Affine Layer $2i\to 2i+1$ reduction for the base hIP.

given by the following equation:

$$\tilde{L}_{2i-1}(\vec{\gamma}_c,\vec{\gamma}_s)=\sum_{\vec{x}_c,\vec{y}_c\in\{0,1\}^{s_{2i}}}\sum_{\vec{\ell}\in\{0,1\}^{\log N}}\Big(\tilde{\mathsf{eq}}(\vec{\gamma}_s,\vec{\ell})\cdot\tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}_c,\vec{x}_c,\vec{y}_c)\cdot\tilde{L}_{2i}(\vec{x}_c,\vec{\ell})\cdot\tilde{L}_{2i}(\vec{y}_c,\vec{\ell})$$
$$+\tilde{\mathsf{eq}}(\vec{1},\vec{x}_c)\cdot\tilde{\mathsf{pmul}}_{2i-1}(\vec{\gamma}_c,\vec{\gamma}_s,\vec{y}_c,\vec{\ell})\cdot\tilde{L}_{2i}(\vec{y}_c,\vec{\ell})\Big) \tag{21}$$

The first term handles CT-CT multiplications and the second handles PT-CT multiplications. The factor $\tilde{\mathsf{eq}}(\vec{1},\vec{x}_c)$ ensures the PT-CT contribution is counted exactly once in the sum over $\vec{x}_c$, since it evaluates to 1 only when $\vec{x}_c=\vec{1}$ and 0 otherwise.

Folded hIP. In the folded hIP, the prover and verifier engage in a standard sumcheck protocol for all $2s_{2i}+\log N$ variables. This process fully reduces the claim about layer $2i-1$ to a claim about layer $2i$ evaluated at specific challenges $\vec{r}_x,\vec{r}_y\in\mathbb{F}_{q^R}^{s_{2i}}$ and $\vec{r}_l\in\mathbb{F}_{q^R}^{\log N}$. The complete protocol is specified in Figure 10.

Folded Multiplication Layer Reduction

1. **Full Sumcheck:** P and V engage in a sumcheck protocol for $2s_{2i} + \log N$ rounds to prove a claim of the form given in Eq. (21). At the end, V has challenges $\vec{r}_x, \vec{r}_y \in \mathbb{F}_{q^R}^{s_{2i}}$ and $\vec{r}_l \in \mathbb{F}_{q^R}^{\log N}$, and a final claim $c \in \mathbb{F}_{q^R}$:

$$\begin{aligned} c \stackrel{?}{=} & \tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}_c, \vec{r}_x, \vec{r}_y) \cdot \tilde{L}_{2i}(\vec{r}_x, \vec{r}_l) \cdot \tilde{L}_{2i}(\vec{r}_y, \vec{r}_l) \\ & + \tilde{\mathsf{eq}}(\vec{1}, \vec{r}_x) \cdot \tilde{\mathsf{pmul}}_{2i-1}(\vec{\gamma}_c, \vec{\gamma}_s, \vec{r}_y, \vec{r}_l) \cdot \tilde{L}_{2i}(\vec{r}_y, \vec{r}_l) \end{aligned} \tag{22}$$

2. **Evaluation Claims:** P sends the claimed evaluations of the next layer:

$$v_x \stackrel{?}{=} \tilde{L}_{2i}(\vec{r}_x, \vec{r}_l), \quad v_y \stackrel{?}{=} \tilde{L}_{2i}(\vec{r}_y, \vec{r}_l)$$

3. **Consistency Check:** V checks if c is consistent with v_x, v_y using Eq. (22).

4. **Line Interpolation:** V defines the unique line $\ell : \mathbb{F}_{q^R} \to \mathbb{F}_{q^R}^{s_{2i}}$ such that $\ell(0) = \vec{r}_x$ and $\ell(1) = \vec{r}_y$. Explicitly, $\ell(t) = (1-t)\vec{r}_x + t\vec{r}_y$. P sends a univariate polynomial $q(t)$ of degree at most s_{2i}, claimed to be $q(t) = \tilde{L}_{2i}(\ell(t), \vec{r}_l)$.

5. **Reduction to Single Point:** V checks that $q(0) = v_x$ and $q(1) = v_y$. V then samples a random challenge $r^* \in \mathbb{F}_{q^R}$. The reduction is complete, and the new claim for layer $2i$ is:

$$q(r^*) \stackrel{?}{=} \tilde{L}_{2i}(\ell(r^*), \vec{r}_l) \tag{23}$$

Figure 10: Specification of a Multiplication Layer $2i-1 \to 2i$ reduction using line interpolation for the folded hIP.

Base hIP. The base protocol modifies the standard approach in two ways to reduce prover noise at the cost of verifier work. First, the sumcheck is terminated early (after $2s_{2i}$ rounds), leaving the sum over the SIMD slots ($\log N$ variables) to be handled directly by the verifier. Second, the prover is lazy: instead of sending aggregated round polynomials, it sends separate round polynomials for each SIMD slot, which the verifier must aggregate. The detailed protocol is specified in Figure 11.

5.2.2 Operation Counts

This section introduces the operation counts of our schemes via the following theorems, with proofs of these counts provided in Appendix E.1 and Appendix E.2.

Theorem 5.5. *Suppose plaintext field order q and cryptographic extension degree R are such that there exists a multiplicative depth 1 $\mathbb{F}_q$-circuit to compute $\mathbb{F}_{q^R}$ multiplication.*[31] *Then the Base BhIP prover for alternating layered payload circuits with m multiplication gates and p affine gates takes as input the intermediate ciphertexts arising from payload computation, requiring 1 level of additional noise budget, and has homomorphic operation count upper bounded by Table 12.*

Theorem 5.6. *Suppose plaintext field order q and cryptographic extension degree R are such that there exists a multiplicative depth 1 $\mathbb{F}_q$-circuit to compute $\mathbb{F}_{q^R}$ multiplication.*[32] *Then the Folded BhIP prover for alternating layered payload circuits with m multiplication gates and p affine gates takes as input the intermediate ciphertexts arising from payload computation, requiring 1.5 levels of additional multiplicative depth and has homomorphic operation count upper bounded by Table 13.*

[31] See Remark 7.1 for a discussion about this assumption.
[32] See Remark 7.1 for a discussion about this assumption.

Base Multiplication Layer Reduction

1. **Sumcheck (Early Termination):** P and V engage in sumcheck for $2s_{2i}$ rounds to prove a claim of the form given in Eq. (21). In every round j, P sends a vector of round polynomials $\mathsf{ept}[R_j]$ (i.e., N polynomials $\{R_{j,\vec{\ell}}(X)\}_{\vec{\ell}\in\{0,1\}^{\log N}}$). Fixing a specific slot index $\vec{\ell}\in\{0,1\}^{\log N}$, the round polynomial $R_{j,\vec{\ell}}(X)$ is equivalent to the jth round polynomial of the following sumcheck instance:

$$\tilde{L}_{2i-1}(\vec{\gamma}_c,\vec{\ell}\,) = \sum_{\vec{x}_c,\vec{y}_c\in\{0,1\}^{s_{2i}}} \Big(\tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}_c,\vec{x}_c,\vec{y}_c)\cdot\tilde{L}_{2i}(\vec{x}_c,\vec{\ell})\cdot\tilde{L}_{2i}(\vec{y}_c,\vec{\ell}) + \tilde{\mathsf{eq}}(\vec{1},\vec{x}_c)\cdot\tilde{\mathsf{pmul}}_{2i-1}(\vec{\gamma}_c,\vec{\ell},\vec{y}_c,\vec{\ell})\cdot\tilde{L}_{2i}(\vec{y}_c,\vec{\ell})\Big)$$

   V aggregates these to recover the standard round polynomial $R_j(X)$ via multilinear evaluation:

$$R_j(X) := \tilde{R}_j(X,\vec{\gamma}_s) = \sum_{\ldots\in\{0,1\}^{\log N}} \tilde{\mathsf{eq}}(\vec{\gamma}_s,\vec{\ell})\cdot R_{j,\vec{\ell}}(X)$$

2. **Final Sumcheck Claim:** After $2s_{2i}$ rounds, with challenges $\vec{r}_x,\vec{r}_y\in\mathbb{F}_{q^R}^{s_{2i}}$, V holds a claim $c\in\mathbb{F}_{q^R}$ that still sums over the slot indices $\vec{\ell}$:

$$c \stackrel{?}{=} \sum_{\vec{\ell}\in\{0,1\}^{\log N}} \tilde{\mathsf{eq}}(\vec{\gamma}_s,\vec{\ell})\cdot\Big(\tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}_c,\vec{r}_x,\vec{r}_y)\cdot\tilde{L}_{2i}(\vec{r}_x,\vec{\ell})\cdot\tilde{L}_{2i}(\vec{r}_y,\vec{\ell}) + \tilde{\mathsf{eq}}(\vec{1},\vec{r}_x)\cdot\tilde{\mathsf{pmul}}_{2i-1}(\vec{\gamma}_c,\vec{\ell},\vec{r}_y,\vec{\ell})\cdot\tilde{L}_{2i}(\vec{r}_y,\vec{\ell})\Big) \tag{24}$$

3. **Vector Evaluation Claims:** P sends $2N$ evaluation claims. Using the plaintext vector notation from Section 5.1.2, these are sent as $\mathsf{ept}[w_x]$ and $\mathsf{ept}[w_y]$ such that for all $\vec{\ell}\in\{0,1\}^{\log N}$:

$$w_{x,\vec{\ell}} \stackrel{?}{=} \tilde{L}_{2i}(\vec{r}_x,\vec{\ell}),\quad w_{y,\vec{\ell}} \stackrel{?}{=} \tilde{L}_{2i}(\vec{r}_y,\vec{\ell})$$

4. **Consistency Check:** V uses the vectors $\mathsf{ept}[w_x],\mathsf{ept}[w_y]$ to evaluate the RHS of Eq. (24) and checks if it equals c.

5. **Slot Reduction:** V samples slot challenges $\vec{r}_l\in\mathbb{F}_{q^R}^{\log N}$. V computes the single-point evaluations:

$$v_x \leftarrow \tilde{w}_x(\vec{r}_l),\quad v_y \leftarrow \tilde{w}_y(\vec{r}_l)$$

   Note that these are claims for $\tilde{L}_{2i}(\vec{r}_x,\vec{r}_l)$ and $\tilde{L}_{2i}(\vec{r}_y,\vec{r}_l)$.

6. **Line Interpolation:** V defines the line $\ell:\mathbb{F}_{q^R}\to\mathbb{F}_{q^R}^{s_{2i}}$ where $\ell(t)=(1-t)\vec{r}_x+t\vec{r}_y$. P sends a univariate polynomial $q(t)$ of degree at most s_{2i} claimed to be $\tilde{L}_{2i}(\ell(t),\vec{r}_l)$.

7. **Reduction to Single Point:** V checks $q(0)=v_x$ and $q(1)=v_y$. V samples $r^*\in\mathbb{F}_{q^R}$. The new claim for layer $2i$ is:

$$q(r^*) \stackrel{?}{=} \tilde{L}_{2i}(\ell(r^*),\vec{r}_l) \tag{25}$$

Figure 11: Full specification of a Multiplication Layer $2i-1\to 2i$ reduction for the base hIP using line interpolation.

5.3 Summary: Base vs Folded BhIP variants

We presented two BhIPs: one *base* and one *folded* variant. The base variant has the following property. In each BhIP round rd, the base prover sends a list of m_{rd} ciphertexts to the verifier. The base verifier decrypts these ciphertexts and uses all $m_{\mathsf{rd}}\cdot N$ of the decrypted $\mathbb{F}_q$ values to make a verification decision. On the other hand, the folded verifier decrypts received ciphertexts and uses

Operation	Level	Total Count Bound
$\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$	$1.0 \to 0.0$	$2p$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$1.5 \to 1.0$	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$	1.0	$2m \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	1.0	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	0.0	$2m \cdot s_{\max}$
Total FHE Depth		**1.5 Levels**

Table 12: Total FHE Cost for the Base BhIP Prover on alternating layered payload circuits with m multiplication gates and p affine gates. Recall $m + p \leq n$ and $p \leq 2m$; in particular $2m + 3p \leq 3n$, where n is the total number of FHE operations in the original payload computation.

Operation	Level	Total Count Bound
$\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$	$2.0 \to 1.0$	$2p$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$2.5 \to 1.5$	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$1.5 \to 1.0$	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$	1.0	$2m \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	2.0	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	1.0	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	0.0	$2m \cdot s_{\max}$
FOLD	0.0	$4dR(s_{\max} + \log N)$
Total FHE Depth		**2.5 Levels**

Table 13: Total FHE Cost for the Folded BhIP Prover on alternating layered payload circuits with m multiplication gates and p affine gates. Recall $m + p \leq n$ and $p \leq 2m$; in particular $2m + 3p \leq 3n$, where n is the total number of FHE operations in the original payload computation. The FOLD operator introduces no additional multiplicative depth.

only m_{rd} of the $m_{\mathsf{rd}} \cdot N$ decrypted $\mathbb{F}_q$ values to make a verification decision.[33] As a visual aid, the difference between these protocols for common sumcheck rounds[34] is depicted below in Fig. 14.
A priori, the purpose of the folded protocol is unclear. To list a few drawbacks:

- For payload circuits with $d = 2d_\times + 1$ layers, the folded BhIP has $d \log N$ additional rounds. This happens when the base BhIP prover terminates each layer reduction sumcheck $\log N$ rounds early.
- A direct consequence is that the folded BhIP prover computes and sends more ciphertexts, and the folded BhIP verifier needs (one slot of) information from all of the decrypted plaintexts.
- The folded BhIP prover consumes more noise than its base counterpart, specifically one additional level of multiplication and $\log(N)$ rotations (which, as discussed in Section 3.2, is concretely < 10 bits of noise).

Despite the drawback of the folded prover doing more work, when compiled to SNARGs in a particular way, the resulting proof size and verifier time of the folded protocol are much less than their base counterparts. We discuss this in Section 6.

6 Achieving Efficient VCoED

In Sections 4 and 5, we defined three public-coin[35] BhIP protocols (base, folded, and laned). In this section, we describe how to use these to build efficient end-to-end VCoED protocols.

[33] The folded BhIP prover has done additional processing so that the folded BhIP verifier need only care about the value stored in the first slot of each decrypted plaintext.
[34] Recall that the base prover skips some rounds of sumcheck entirely.
[35] It should be assumed henceforth that all discussed protocols are public-coin.

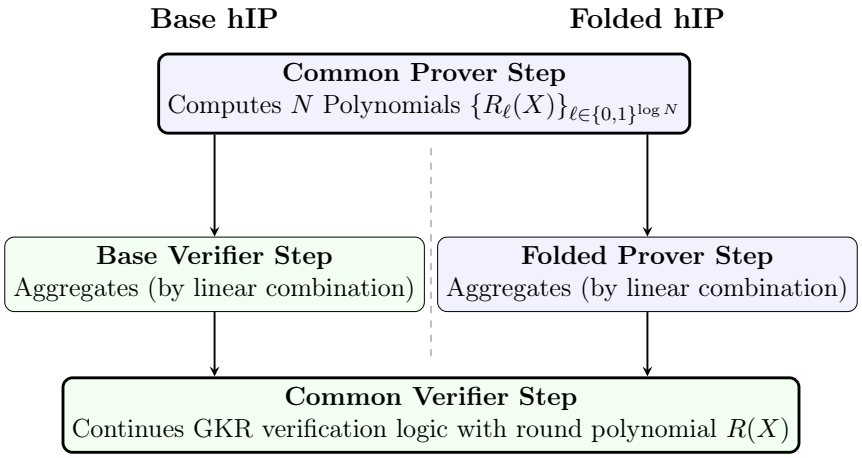

Figure 14: Comparison of protocol flow. Top: Both variants begin with the Prover computing raw polynomials. Middle: In **Base** (left), the Verifier aggregates; in **Folded** (right), the Prover aggregates. Bottom: Both converge on the same verification logic.

6.1 Communication Overhead of Blind Interactive Proofs

To obtain a non-interactive argument, we apply the Fiat-Shamir transform to the BhIP, optionally preceded by the transcript packing compiler introduced below. However, the BhIP's communication cost is a bottleneck that we must first address.

In each round rd, the BhIP prover sends M_{rd} ciphertexts to the verifier. Each ciphertext encrypts N field elements, but in the laned and folded BhIP variants, only M_{rd} of the $M_{\mathsf{rd}}\cdot N$ communicated $\mathbb{F}_q$ elements (those in the first slot of each ciphertext) influence the verification decision. Letting $M = \sum_{\mathsf{rd}} M_{\mathsf{rd}}$ denote the total number of ciphertexts sent across all rounds, the BhIP communicates $M\cdot N$ field elements to convey only M useful values. Ideally, the prover could pack all useful information into $\lceil M/N\rceil$ "packed ciphertexts", but doing so naively would break the temporal ordering required for IP soundness.

6.2 Transcript Packing Compiler

In this subsection, we introduce a **transcript packing** compiler that transforms a BhIP into a new (designated-verifier) interactive proof system for the same language with dramatically reduced communication. The key observation is that in each round, it suffices for the prover to send a succinct *commitment* to its ciphertext messages (rather than the ciphertexts themselves). At the end of the protocol, the prover sends a small number of "packed" ciphertexts encoding the combined messages from all rounds, together with an outer SNARK proof that these packed ciphertexts are consistent with the per-round commitments.

Theorem 6.1. *Let $H : \{0,1\}^* \to \{0,1\}^{\mathrm{poly}(\lambda)}$ be a hash function modeled as a random oracle. Let $\mathcal{E}[\pi] = (\mathsf{Setup}, \mathsf{P}, \mathsf{V})$ be an r-round public-coin BhIP defined over BGV scheme $\mathcal{E}$, with state-restoration soundness error $\varepsilon^{sr}(t,n)$ for indexed language $\mathcal{L}$. Let $M := \sum_{\mathsf{rd}=1}^{r} M_{\mathsf{rd}}$ denote the total number of raw ciphertexts sent in $\mathcal{E}[\pi]$ across all r rounds. Fix parameter $k \in [N]$ such that the verification decision of $\mathcal{E}[\pi].\mathsf{V}$ is unaffected by changes to the contents of slots $\{k+1,\ldots,N\}$ in any of these M ciphertexts. Let $z := M\cdot k$ be the total number of useful $\mathbb{F}_q$ elements. Let $Z := \lceil z/N\rceil$ denote the number of N-tuples required to represent z elements. Let $F := \sum_{i=1}^{r}\lceil M_{\mathsf{rd}}\cdot k/N\rceil$ denote the total number of flattened ciphertexts across all rounds.*

Then, the compiler described in Fig. 15 creates an r-round designated-verifier interactive proof $\pi' = (\mathsf{Setup}', \mathsf{P}', \mathsf{V}')$ for $\mathcal{L}$ with the following properties:

- ***Completeness:*** *If $\mathcal{E}[\pi]$ is complete, then π' is complete.*
- ***State-restoration soundness (ROM):*** *In the random oracle model, π' has state-restoration soundness error at most $\varepsilon^{sr}(t, n) + \kappa_{\mathsf{SNARK}}(\lambda) + \mathrm{negl}(\lambda)$, where κ_{SNARK} is the knowledge error of the outer SNARK and the $\mathrm{negl}(\lambda)$ term accounts for hash collision probability.*
- ***Prover efficiency:*** *Homomorphically computing all $\pi'.\mathsf{P}'$ messages requires two additional levels compared to computing $\mathcal{E}[\pi].\mathsf{P}$ messages. Specifically, intra-round flattening performs M plaintext-ciphertext multiplications (selection masks) consuming one level, $M + F$ rotations, and $M+F$ ciphertext additions. Inter-round packing (Eq. (26)) performs F plaintext-ciphertext multiplications (position masks) consuming a second level, and at most F ciphertext additions.*
- ***Communication:*** *The communication of $\pi'.\mathsf{P}'$ messages across all rounds consists of r hash digests, Z ciphertexts, and one SNARK proof.*

The proof of Theorem 6.1 can be found in Appendix C.

6.2.1 Compiler Description

Each raw ciphertext produced by the BhIP prover $\mathcal{E}[\pi].\mathsf{P}$ carries useful data only in its first k slots; the remaining $N - k$ slots may contain arbitrary values (e.g., partial sums left by the FOLD operator). To pack the useful data from all rounds into a small number of dense ciphertexts, we employ a two-stage packing strategy.

Stage 1: intra-round flattening. In round $i \in [r]$, the BhIP prover $\mathcal{E}[\pi].\mathsf{P}_i$ produces M_i raw ciphertexts, each with k active slots. The compiled prover P'_i consolidates these into $f_i := \lceil (M_i \cdot k)/N \rceil$ dense ciphertexts as follows. For each raw ciphertext $j \in [M_i]$:

1. Multiply by the selection mask $[1, 0, \ldots, 0]^{\otimes k}$ (a plaintext that is 1 in slots $0, \ldots, k-1$ and 0 elsewhere) to zero out the non-active slots. This is a **plaintext-ciphertext multiplication**, consuming one multiplicative level.
2. Rotate by an intra-round offset so that the k active values land in the correct positions within one of the f_i flattened ciphertexts.

The M_i masked-and-rotated ciphertexts are then summed (via ciphertext addition) into f_i dense flattened ciphertexts $\mathsf{ct}_{\mathrm{flat}}^{(i,\cdot)}$.

Stage 2: inter-round packing. Each flattened ciphertext is further rotated to a global offset so that its contents align with the appropriate positions in the final packed vector. Let $\mathsf{ct}_{\mathrm{rot}}^{(i,u)}$ denote the result of rotating $\mathsf{ct}_{\mathrm{flat}}^{(i,u)}$ by global offset $\rho_{i,u}$. Each aligned ciphertext is then multiplied by a **position mask** $\mathsf{pt}_{\mathrm{mask}}^{(i,u)}$ that selects the slot range destined for a specific packed ciphertext. This is a second **plaintext-ciphertext multiplication**, consuming one additional multiplicative level. The final packed ciphertexts are sums of the masked aligned ciphertexts:

$$\mathsf{ct}_{\mathrm{final},\ell} = \sum_{(i,u) \in S_\ell} \mathsf{pt}_{\mathrm{mask}}^{(i,u)} \cdot \mathsf{ct}_{\mathrm{rot}}^{(i,u)} \tag{26}$$

where S_ℓ indexes the (round, flattened CT) pairs that contribute to packed ciphertext ℓ. Since each masked ciphertext occupies a unique set of slots, the addition introduces no collisions.

Commit-and-Pack Compiler

Setup: Let N be the ciphertext slot count. Let M_i be the number of raw ciphertexts in round i, each with k active slots. The prover will pack these into $Z = \lceil (\sum M_i \cdot k)/N \rceil$ final ciphertexts.

1. **For each round $i = 1$ to r:**
   - P'_{i} runs the BhIP prover $\mathcal{E}[\pi].\mathsf{P}_{\mathsf{i}}$ to obtain M_i raw ciphertexts: $\mathsf{ct}_{\mathrm{raw}}^{(i,1)}, \ldots, \mathsf{ct}_{\mathrm{raw}}^{(i,M_i)}$.
   - **Intra-round Flattening:** For each raw ciphertext $j \in [M_i]$, P'_{i} multiplies by the selection mask to zero out non-active slots and rotates to the appropriate intra-round position. The M_i results are summed into $f_i = \lceil (M_i \cdot k)/N \rceil$ dense flattened ciphertexts $\mathsf{ct}_{\mathrm{flat}}^{(i,\cdot)}$. This consumes one multiplicative level.
   - **Inter-round Alignment:** For each flattened ciphertext $u \in [f_i]$, P'_{i} computes a global offset $\rho_{i,u}$ and rotates:
   $$\mathsf{ct}_{\mathrm{rot}}^{(i,u)} \leftarrow \mathsf{Rot}(\mathsf{ct}_{\mathrm{flat}}^{(i,u)}, \rho_{i,u})$$
   - P'_{i} sends a hash commitment to the aligned set:
   $$C_i := H\left(\{\mathsf{ct}_{\mathrm{rot}}^{(i,u)}\}_{u=1}^{f_i}\right)$$
   - The verifier V' observes C_i and (if $i < r$) sends challenge c_i.
2. **Finalization (after round r):**
   - P' computes Z final packed ciphertexts by applying position masks and summing aligned ciphertexts across rounds:
   $$\mathsf{ct}_{\mathrm{final},\ell} = \sum_{(i,u)\in S_\ell} \mathsf{pt}_{\mathrm{mask}}^{(i,u)} \cdot \mathsf{ct}_{\mathrm{rot}}^{(i,u)}$$
   where each position mask $\mathsf{pt}_{\mathrm{mask}}^{(i,u)}$ selects the slots of aligned ciphertext (i,u) that belong to packed ciphertext ℓ. This consumes one additional multiplicative level.
   - P' sends $\{\mathsf{ct}_{\mathrm{final},\ell}\}_{\ell=1}^{Z}$ and a SNARK proof π_{outer} attesting that there exist aligned ciphertexts consistent with the commitments $\{C_i\}_{i=1}^{r}$ that produce the packed ciphertexts via Eq. (26).
3. **Verifier Decision:**
   - V' verifies π_{outer} against public inputs $(\{C_i\}_{i=1}^{r}, \{\mathsf{ct}_{\mathrm{final},\ell}\}_{\ell=1}^{Z})$.
   - **Transcript Reconstruction:** V' reconstructs the per-round ciphertext messages of $\mathcal{E}[\pi]$ from the packed ciphertexts using the known packing structure. Concretely, for each round i and raw message index $j \in [M_i]$, V' extracts the corresponding k ciphertext slots from the appropriate packed ciphertext(s) to form a reconstructed ciphertext message.
   - V' invokes the BhIP verifier's decision algorithm $\mathcal{E}[\pi].\mathsf{V}$ on the reconstructed ciphertext transcript and the challenges $\{c_i\}_{i=1}^{r}$.

Figure 15: Commit-and-Pack Compiler. Two-stage packing: intra-round flattening (mask, rotate, sum) consolidates raw ciphertexts into dense ones, then inter-round packing (rotate, mask, sum) produces the final packed ciphertexts.

Per-round commitment. After computing the f_i aligned ciphertexts in round i, the compiled prover P'_{i} sends a hash commitment:

$$C_i := H\left(\{\mathsf{ct}_{\mathrm{rot}}^{(i,u)}\}_{u=1}^{f_i}\right)$$

Committing to the f_i *flattened* ciphertexts (rather than the M_i raw ciphertexts) significantly reduces the hash input size for the outer SNARK.

We formally describe the compiled protocol π' in Fig. 15.

Remark 6.2 (Depth of transcript packing)**.** The transcript packing compiler adds two multiplicative levels on top of the BhIP prover. The first level is consumed by the intra-round flattening

step (the selection mask, a plaintext-ciphertext multiplication that zeros out non-active slots). The second level is consumed by the inter-round packing step (the position masks in Eq. (26), a plaintext-ciphertext multiplication that selects the slot range destined for each packed ciphertext). All other operations (rotations and ciphertext additions) consume no multiplicative depth.

6.2.2 Cost Analysis of Computing Outer SNARK π_{outer}

The R1CS arithmetization. Let Q denote the final BGV ciphertext modulus. For simplicity, we assume that Q is a prime. Our R1CS will be defined natively over the scalar field $\mathbb{F}_Q$, so as to avoid expensive non-native field arithmetic.

- **Public inputs:**
  - The r commitment hashes $C_1, \ldots, C_r$.
  - The L final packed ciphertexts $\{\mathsf{ct}_{\text{final},\ell}\}_{\ell=1}^{L}$. Since the packing operation involves only plaintext-ciphertext multiplication and addition, these ciphertexts remain degree-1 (consisting of 2 polynomials: c_0, c_1). Thus, this input contributes $2NL$ field elements.
- **Witness:**
  - The $f = \sum f_i$ flattened ciphertexts $\{\mathsf{ct}_{\text{rot}}^{(i,u)}\}_{i,u}$. These are also degree-1 ciphertexts, contributing $2Nf$ field elements.

Constraint 1: hashing (commitment verification). The circuit must verify that the witness elements correspond to the public commitments C_i. Let C_{hash} be the (amortized) number of R1CS constraints required to hash a single $\mathbb{F}_Q$ element. The total number of field elements to be hashed is $2Nf$. Thus, the hashing cost is:

$$\mathcal{N}_{\text{hash}} = 2Nf \cdot C_{\text{hash}} \tag{27}$$

Constraint 2: linear packing consistency. The circuit must verify the relationship in Eq. (26):

$$\mathsf{ct}_{\text{final},\ell} = \sum_{(i,u)\in S_\ell} \left(\mathsf{ct}_{\text{rot}}^{(i,u)} \cdot \mathsf{pt}_{\text{mask}}^{(i,u)}\right)$$

In this equation, the masks $\mathsf{pt}_{\text{mask}}^{(i,u)}$ are public constants. We emphasize that this operation is a **plaintext-ciphertext** multiplication, so no relinearization is required; the result remains a pair of polynomials (c_0, c_1). Furthermore, we assume the modulus Q is sufficient to accommodate the noise growth from this operation.

In R1CS, this equation represents a fixed linear combination of witness variables. Enforcing that a linear combination of witnesses equals a public output requires **1 constraint per output element**. Since there are L output ciphertexts, each consisting of $2N$ elements, the cost is:

$$\mathcal{N}_{\text{linear}} = 2NL \tag{28}$$

Total complexity. Summing these components, the total constraint count is:

$$\begin{aligned}
\mathcal{N}_{\text{total}} &= \mathcal{N}_{\text{hash}} + \mathcal{N}_{\text{linear}} && (29)\\
&= 2Nf \cdot C_{\text{hash}} + 2NL && (30)\\
&\leq 2Nf \cdot (C_{\text{hash}} + 1) && (31)
\end{aligned}$$

Prover runtime estimates. We are free to instantiate the outer SNARK with any field-agnostic scheme. For concreteness, we select:

- The Spartan [74] field-agnostic PIOP with the BaseFold [84] field-agnostic PCS.
- Wrapped in a Groth16 proof [48].

This combination yields $O(1)$ proof size and verification time. It remains to verify that the prover runtime is acceptable.

Cost of the outer Groth16 proof. The Groth16 prover proves the Spartan verification circuit. As this is a succinct instance significantly smaller than the primary circuit, it is not the bottleneck; we therefore ignore this cost.

Figure 7 of [74] indicates that the single-threaded Spartan PIOP can prove 2^{20} R1CS constraints in approximately 36 seconds. However, our R1CS instance is highly uniform; as the author notes in [73], the Spartan prover is roughly $10\times$ faster for uniform circuits. Consequently, we estimate the single-threaded Spartan PIOP can prove 2^{20} constraints in 3.6 seconds.

The Spartan prover must also commit to a witness vector z of length 2^{20}. According to Figure 5 of [84], committing to a 2^{20}-length vector of 64-bit field elements takes less than 0.4 seconds. Summing these components ($3.6+0.4$) and assuming quasi-linear asymptotics for both Spartan and BaseFold, we extrapolate the time to compute π_{outer} for a circuit of size $\mathcal{N}$ as:

$$4 \cdot \frac{\mathcal{N}}{2^{20}} \cdot \frac{\log(\mathcal{N})}{20} \text{ seconds.}$$

Concrete circuits. Assume a plaintext modulus of $q = 2^{16} + 1$ and extension degree $R = 8$ (providing ≈ 100 bits of security). We select a final ciphertext modulus $Q \leq 2^{32}$, which fits within 4 bytes. Using an arithmetization-friendly hash function like Poseidon [34] at security level $\lambda = 128$, the cost amortizes to approximately two constraints per byte of input. Since elements of $\mathbb{F}_Q$ are 4 bytes, we set $C_{\text{hash}} \leq 8$.

Suppose our payload circuit has $n = 2^{20}$ gates, $d = 32$ layers, and SIMD-batching parameter $N = 2^{14}$. Following an argument similar to Claim 3.5, our folded prover requires at most:

$$\begin{aligned} r &= d(2\log(n/d) + \log N + 1) \\ &= 32(30 + 14 + 1) \\ &= 1440 \end{aligned}$$

rounds. Moreover, since each round contains $2R \ll N$ slots of useful data, we have $f = r$. Substituting these values into our bound for total constraints $\mathcal{N}_{\text{total}}$:

$$\mathcal{N}_{\text{total}} \leq 2Nf \cdot (C_{\text{hash}} + 1) \leq 2^{15} \cdot 1440 \cdot 9 \leq 2^{15} \cdot 2^{14} = 2^{29}.$$

Finally, we estimate the prover runtime:

$$\begin{aligned} 4 \cdot \frac{2^{29}}{2^{20}} \cdot \frac{29}{20} &= 4 \cdot 2^9 \cdot 1.45 \\ &\leq 3600 \text{ seconds.} \end{aligned}$$

This is a negligible fraction of the total VCoED server runtime (see Table 20). We conclude that the inclusion of an outer SNARK impacts neither proof size nor server runtime meaningfully.

End-to-End VCoED Protocol

1. **Setup:** The client runs $\Pi.\mathsf{Setup}(1^\lambda, \mathsf{Ckt})$. This generates the HE keys $(\mathsf{sk}, \mathsf{evk})$ and the SNARG prover/verifier keys $(\mathsf{pk}, \mathsf{vk})$. The client keeps vk (which contains sk) and sends $(\mathsf{pk}, \mathsf{evk})$ to the server.
2. **Client Encryption:** The client encrypts its inputs with a modulus chain supporting θ levels of multiplicative depth.
3. **Server Computation:** The server homomorphically evaluates the payload circuit:
$$\{\mathsf{ct}[\mathsf{out}]\} \leftarrow \mathcal{E}.\mathsf{Eval}_{\mathsf{evk}}(\mathsf{Ckt}, \{\mathsf{ct}[\mathsf{in}]\})$$
4. **Server Witness Cleanup:** To minimize the cost of the subsequent proof generation, the server performs modulus switching on each intermediate ciphertext so that it retains *exactly* θ_π levels of noise budget. This reduces the bit-width of the witness ciphertexts before they enter the SNARG prover.
5. **Server Proof Generation:** Let $\mathcal{W}$ denote the cleaned-up intermediate ciphertexts. The server computes the SNARG proof:
$$\pi \leftarrow \Pi.\mathsf{P}^{[\![\rho]\!]}\big(\mathsf{pk},\ (\mathsf{ct}[\mathsf{in}], \mathsf{ct}[\mathsf{out}]),\ \mathcal{W}\big)$$
6. **Server Response:** The server sends $\{\mathsf{ct}[\mathsf{out}]\}$ and proof π to the client.
7. **Client Verification:** The client uses its verifier key vk (which includes sk) to verify the decrypted claim:
$$\{\mathsf{acc}/\mathsf{rej}\} \leftarrow \Pi.\mathsf{V}^{[\![\rho]\!]}\big(\mathsf{vk},\ (\mathsf{in}, \mathsf{out}),\ \pi\big)$$

Figure 16: End-to-End VCoED Protocol

6.3 The Laminate VCoED Protocols

An end-to-end VCoED protocol is fully determined by the choice of BhIP and whether the transcript packing compiler is applied before Fiat-Shamir compilation. We present three concrete instantiations: two derived from the wide BhIPs in Section 5 and one from the laned BhIP in Section 4. A high-level comparison against prior works is given in Table 1.

All instantiations follow the same end-to-end structure. Let Ckt be a payload circuit with multiplicative depth $d_\times$, and let $\Pi = (\mathsf{Setup}, \mathsf{P}, \mathsf{V})$ be a designated-verifier SNARG for the indexed language
$$\mathcal{L}_{\mathsf{Ckt}} = \big\{(\mathsf{in}, \mathsf{out}) \;\big|\; \mathsf{Ckt}(\mathsf{in}) = \mathsf{out}\big\},$$
constructed by compiling a BhIP as described in the preceding sections. Let θ_π denote the number of multiplicative levels consumed by the SNARG prover, so the total required depth is $\theta \le d_\times + \theta_\pi$. The SNARG prover takes as input the intermediate ciphertexts that arise from homomorphically evaluating the payload circuit, and produces a proof that the server computed the circuit correctly. The end-to-end protocol is described in Fig. 16.

6.3.1 Efficiency Parameters and Notation

We analyze the server efficiency and proof size of the three Laminate protocols with respect to a payload circuit over the plaintext field $\mathbb{F}_q$ (as defined in Section 5.2).

Following earlier conventions, we let m denote the number of SIMD multiplication gates, p the number of affine gates, and n the total number of gates in the payload circuit. Furthermore, we let d be the number of layers, $s_{\max}$ the logarithm of the maximum layer width ($\max\{\log S_i\}$), and R the extension degree of $\mathbb{F}_{q^R}$ required for soundness. We assume throughout that q and R are such that there exists a multiplicative depth 1 $\mathbb{F}_q$-circuit to compute $\mathbb{F}_{q^R}$ multiplication, as discuss in Remark 7.1.

The operation counts and proof sizes presented below use the following parameters:

- r**: The number of BhIP rounds.**
  - Base BhIP: $r < d(2\log(n/d) + 1)$.[36]
  - Folded BhIP: $r < d(2\log(n/d) + \log N + 1)$.
  - B-Laned BhIP: $r \leq d\log\left(\frac{2nB}{d} + N\right)$.[37]
- M**: The total number of "raw" ciphertexts in the BhIP transcript.**
  - Base BhIP: $M = 2Rr$.
  - Folded BhIP: $M = 2Rr$.
  - Laned BhIP: $M = 2Rr$.
- F**: The number of "flattened" ciphertexts (before inter-round packing).** Relevant for protocols that use transcript packing ($\mathsf{Laminate}_{\mathsf{fold}}$ and $\mathsf{Laminate}_{\mathsf{laned}}$).
  - Folded BhIP: $F \leq r\lceil 2R/N \rceil$.
  - Laned BhIP: $F \leq r\lceil 2R/N \rceil$.
- Z**: The final number of "packed" ciphertexts sent to the verifier (when using transcript packing).** Relevant for $\mathsf{Laminate}_{\mathsf{fold}}$ and $\mathsf{Laminate}_{\mathsf{laned}}$.
  - Folded BhIP: $Z = \lceil M/N \rceil$.
  - Laned BhIP: $Z = \lceil M/N \rceil$.

6.3.2 $\mathsf{Laminate}_{\mathsf{base}}$: Base BhIP

This scheme applies the Fiat-Shamir transform directly to the Base BhIP (see Table 12) without transcript packing. It requires 1 level of additional multiplicative depth (see Theorem 5.5). Since no packing is performed, the proof consists of M raw ciphertexts.

- **Prover Efficiency:** The server's cost is dominated by the BGV operations performed by the Base BhIP prover.
- **Use Case:** This scheme is optimal for high-throughput applications where the payload fully utilizes all available SIMD slots and bandwidth is not the bottleneck.

Proof Size: M ciphertexts.

Total server FHE operations are presented in Table 17.

[36] This is the bound from Claim 3.5. This is a loose upper bound for our arithmetization and GKR layer reduction subroutines.

[37] Each of the d layer reductions is an $(s'_j + \log N)$-round sumcheck, where $s'_j = \log_2 S'_j$ is the packed width of the relevant odd layer. The dense wiring structure (Section 4.2.2) ensures that each reduction sums over input variables *once* (yielding s'_j, not $2s'_j$, ciphertext-index rounds). Since d of the odd layers appear in two reductions each, $r = \sum_{i=1}^{d}(s'_{j(i)} + \log N)$ where $T := \sum_{i=1}^{d} S'_{j(i)} = 2m' + S'_d$. Applying Jensen's inequality: $\sum \log S'_{j(i)} \leq d\log(T/d)$, giving $r \leq d\log(TN/d)$. Bounding $T \leq (2m + S_d)B/N + d \leq 2nB/N + d$ completes the claim.

Operation	Level	Total Count Bound
$\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$	$1.0 \to 0.0$	$2p$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$1.5 \to 1.0$	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$	1.0	$2m \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	1.0	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	0.0	$2m \cdot s_{\max}$
Total FHE Depth		**1.5 Levels**

Table 17: Total Additional FHE Cost for the $\mathsf{Laminate}_{\mathsf{base}}$ Server after Payload Evaluation. This table is identical to Table 12 (the Base BhIP Prover), since the base variant does not use transcript packing. Recall $m + p \leq n$ and $p \leq 2m$; in particular $2m + 3p \leq 3n$, where n is the total number of FHE operations in the original payload computation.

Operation	Level	Total Count Bound
$\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$	$4.0 \to 3.0$	$2p$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$4.5 \to 3.5$	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$3.5 \to 3.0$	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$	3.0	$2m \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	4.0	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	3.0	$(2m + 3p) \cdot s_{\max}$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	2.0	$2m \cdot s_{\max}$
FOLD	2.0	$4dR(s_{\max} + \log N)$
Total FHE Depth		**4.5 Levels**

Table 18: Total Additional FHE Cost for the $\mathsf{Laminate}_{\mathsf{fold}}$ Server after Payload Evaluation. This table is derived from Table 13 (the Folded BhIP Prover) with +2 levels added to account for the transcript packing compiler (Section 6.2). The FOLD operator introduces no additional multiplicative depth. Recall $m + p \leq n$ and $p \leq 2m$; in particular $2m + 3p \leq 3n$, where n is the total number of FHE operations in the original payload computation.

6.3.3 $\mathsf{Laminate}_{\mathsf{fold}}$: Folded BhIP with Transcript Packing

This scheme applies the transcript packing compiler (Section 6.2) to the Folded BhIP (see Table 13), followed by Fiat-Shamir compilation. It requires 3.5 levels of additional multiplicative depth (1.5 from the Folded BhIP prover by Theorem 5.6 plus 2 from the transcript packing compiler). The active slots of the BhIP transcript are packed into $Z = \lceil M/N \rceil$ ciphertexts (since the folded prover outputs data only in slot 0).

- **Prover Efficiency:** The server's cost is dominated by the BGV operations performed by the Folded BhIP prover The server has an additional cost of proving the outer SNARK for the transcript packing consistency check.
- **Use Case:** This protocol offers the smallest possible proof size ($Z \ll M$). It is ideal for bandwidth-constrained environments where the additional depth is acceptable. This scheme is intended for payloads that fully utilize all available SIMD slots.

Proof Size: $\lceil M/N \rceil$ ciphertexts + 1 SNARK Proof + r hash outputs.

Total server FHE operations are presented in Table 18.

6.3.4 $\mathsf{Laminate}_{\mathsf{laned}}$: Laned BhIP with Transcript Packing

This scheme targets B-laned payload circuits ($B < N$, see Section 4) and applies the transcript packing compiler (Section 6.2) to the laned BhIP (see Table 6), followed by Fiat-Shamir compilation. The total depth overhead is 4.5 levels: 2.5 from the laned BhIP prover (Lemma E.13, which includes 1 level for witness packing) plus 2 from the transcript packing compiler (Remark 6.2).

Operation	Level	Total Count Bound
BhIP Proof Generation (excl. witness packing)		
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$4.5 \to 4.0$	$3m' \cdot s'_{\max}$
$\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$	$4.0 \to 3.0$	$2m'$
$\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$	$3.5 \to 3.0$	$1.5m' \cdot s'_{\max}$
$\mathsf{ept} \cdot \mathsf{ect} \to \mathsf{ect}$	$3.0 \to 2.0$	$4m'$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	3.0	$2m'$
$\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$	2.0	$2m'$
FOLD	2.0	$2d(s'_{\max} + \log N)$
Transcript Packing		
$\mathsf{pt} \cdot \mathsf{ct} \to \mathsf{ct}$	$2.0 \to 1.0$	M
rotation	1.0	$M + F$
$\mathsf{ct} + \mathsf{ct} \to \mathsf{ct}$	1.0	M
$\mathsf{pt} \cdot \mathsf{ct} \to \mathsf{ct}$	$1.0 \to 0.0$	F
$\mathsf{ct} + \mathsf{ct} \to \mathsf{ct}$	0.0	F
Total FHE Depth		**4.5 Levels**

Table 19: Total Additional FHE Cost for the $\mathsf{Laminate}_{\mathsf{laned}}$ Server after Payload Evaluation, **excluding witness packing**. The BhIP proof generation rows are derived from Table 6 with +2 levels added to account for the transcript packing compiler (Section 6.2). Witness packing adds $3m$ PT-CT multiplications and $3m$ additions (see Appendix E.3.2); these consume one multiplicative level that is already accounted for in the 4.5-level depth budget. Here $m' = \sum S'_{2i-1} \leq mB/N$ is the packed multiplication gate count, $s'_{\max} = \max_i\{s'_i\}$, $d = 2d_\times + 1$, and M, F are the transcript packing parameters defined in Section 6.3.1. The FOLD operator introduces no additional multiplicative depth.

Since the laned BhIP prover uses the FOLD operator to aggregate round polynomial evaluations into slot 0, the useful slot count is $k = 1$. Consequently, transcript packing achieves maximum compression: the $M = 2Rr$ raw ciphertexts are packed into $Z = \lceil 2Rr/N \rceil$ dense ciphertexts.

- **Prover Efficiency:** The server first evaluates the B-laned payload circuit (n FHE operations at $\Omega(N)$ CPU cost each), then performs witness packing ($O(n)$ additional FHE operations to pack intermediate ciphertexts), followed by BhIP proof generation on the packed circuit ($O(m's'_{\max})$ FHE operations), and finally transcript packing ($O(M)$ FHE operations). In the regime $B \ll N$, the payload and witness packing costs dominate; proof generation and transcript packing are comparatively inexpensive.
- **Use Case:** This protocol is designed for applications where the payload circuit uses only $B < N$ SIMD slots with no cross-slot interactions (i.e., laned payloads). It delivers compact proofs ($Z \ll M$) while operating on the packed representation, amortizing proof generation across the full SIMD width.

Proof Size: $\lceil M/N \rceil$ ciphertexts + 1 SNARK Proof + r hash outputs.

Total server FHE operations (excluding witness packing) are presented in Table 19.

6.3.5 Protocol Properties

All three Laminate instantiations satisfy the following properties.

Completeness. Completeness holds provided the client initializes the HE scheme with a sufficiently large modulus chain. Given enough noise budget, every homomorphic operation in the payload evaluation and subsequent proof generation decrypts correctly, so an honest server always produces a proof that the client accepts.

Client privacy. Up to the one-bit leakage inherent in any verification protocol (the accept/reject decision), client privacy follows directly from the IND-CPA security of the underlying FHE scheme $\mathcal{E}$. The server's view throughout the protocol consists entirely of ciphertexts and evaluation keys; no additional information about the client's plaintext inputs is revealed.

Soundness. We argue soundness via a sequence of reductions, where all interactive soundness notions below refer to *state-restoration* soundness (as defined in Section 2).

1. **Interactive GKR has negligible soundness error.** The GKR interactive proof (Section 5) for circuit evaluation over $\mathbb{F}_{q^R}$ has state-restoration soundness error $\varepsilon^{\mathrm{sr}}(t, n) \leq \mathrm{negl}(\lambda)$ for polynomially bounded move budgets t.

2. **Running the prover under FHE preserves soundness.** Evaluating the GKR prover's messages homomorphically (i.e., forming the BhIP $\mathcal{E}[\pi]$) does not increase the soundness error as per Theorem 2.3. Intuitively, the verifier can always decrypt to recover the underlying interactive proof transcript.

3. **Transcript packing adds $\kappa_{\mathsf{SNARK}}(\lambda) + \mathrm{negl}(\lambda)$ soundness error.** By Theorem 6.1, the transcript packing compiler transforms the BhIP into a designated-verifier interactive proof with state-restoration soundness error at most $\varepsilon^{\mathrm{sr}}(t, n) + \kappa_{\mathsf{SNARK}}(\lambda) + \mathrm{negl}(\lambda)$, where κ_{SNARK} is the knowledge error of the outer SNARK and the $\mathrm{negl}(\lambda)$ term accounts for hash collision probability.

4. **Fiat-Shamir compilation in the ROM.** Since the interactive protocol has negligible state-restoration soundness error, the Fiat-Shamir compilation bound of [26] (recalled in Section 2) implies that the resulting non-interactive argument is sound in the random oracle model.

For $\mathsf{Laminate}_{\mathsf{base}}$, which does not use transcript packing, step 3 is unnecessary and the argument simplifies: the BhIP is compiled directly via Fiat-Shamir, yielding a non-interactive argument with soundness error $\varepsilon^{\mathrm{sr}}(t, n) + t^2/2^{\lambda}$. For $\mathsf{Laminate}_{\mathsf{laned}}$, the same four-step argument applies: the laned BhIP (Section 4) provides the interactive soundness (steps 1–2), transcript packing adds the SNARK error (step 3), and Fiat-Shamir compilation completes the reduction (step 4).

Remark 6.3 (Verifiable CKKS)**.** Another important line of work in FHE with SIMD support is the CKKS homomorphic encryption scheme [25]. The main distinction from BGV/BFV is that CKKS supports fixed-point (i.e., approximate real-number) arithmetic rather than exact finite-field operations. Consequently, our current method does not immediately apply, since the GKR protocol is inherently defined over finite fields.

However, a very recent work [10] introduces a method for performing sumcheck over approximate computations. Because sumcheck is the core algebraic component underlying GKR, this result provides an intriguing first step toward extending GKR to the approximate setting. As the authors note, additional analysis is required to fully instantiate a GKR protocol over approximate arithmetic, but we view this development as a promising direction. An approximate GKR protocol may be combined with the techniques of this paper to attain *verifiable CKKS*. We leave this exploration to future work.

7 Evaluation

This section discusses implementation considerations and how we compare to prior works.

7.1 Implementation Considerations

Low-depth extension field multiplications. Recall that BGV/BFV uses $\mathbb{F}_p$ as its plaintext field, but for GKR evaluation, we need to work over $\mathbb{F}_{p^R}$ for some $R > 1$ (chosen so that $p^R \geq 2^\lambda$ for a security parameter λ). Fortunately, this extension is straightforward. Define $\mathbb{F}_{p^R} := \mathbb{F}_p[X]/f(X)$ where $f(X)$ is an irreducible polynomial of degree R. Then each element $x \in \mathbb{F}_{p^R}$ is represented as a vector in $\mathbb{F}_p^R$. To multiply $x, y \in \mathbb{F}_{p^R}$, we first compute $z \leftarrow x \otimes y \in \mathbb{F}_p^{2R-1}$ where $\otimes$ denotes polynomial multiplication (from two degree-$(R-1)$ polynomials to a degree-$(2R-2)$ polynomial).

Note that this polynomial must then be reduced modulo $f(X)$. More formally, for $i \geq R$ we use the relation $X^i \equiv X^{i-R} \cdot (-f(X))$ to reduce its degree. However, computing this naively requires $i - R + 1$ multiplicative levels: since $X^R \equiv -f(X)$, multiplying any coefficient $c_R \cdot X^R$ becomes $c_R \cdot (-f(X))$, which costs one multiplicative level. Then, reducing $c_{R+1} \cdot X^{R+1}$ first similarly requires multiplying $-X \cdot c_{R+1} \cdot f(X)$, again costing a multiplicative level. However, now, the resulting polynomial still contains a degree-R term that requires an additional reduction. Thus, modding X^i naively takes $i - R + 1$ multiplication levels, if the modulo operation is done in such a recursive way.

To avoid this depth blowup, we precompute $f_i(X) = X^i \bmod f(X)$ for all $i \geq R$ and store the results. Now, each reduction requires only a single multiplication, independent of i. Moreover, by choosing $f(X)$ so that all coefficients are ± 1, the reduction step involves only additions and subtractions, with no multiplications at all. Such a polynomial does not necessarily exist for arbitrary (p, R), but for the parameters we use (see Section 7), we identified such a choice. In this case, each extension-field multiplication costs exactly one multiplication level, matching the cost of a base-field multiplication.

Remark 7.1. For many prime powers p and required extension degrees R, there are sparse polynomials of degree R with low norm coefficients (e.g. $-1, 0, 1$) that are irreducible over $\mathbb{F}_p$.[38] Even when such a polynomial cannot be found, $\mathbb{F}_{p^R}$ field multiplication can be computed by a $\mathbb{F}_p$-arithmetic circuit with multiplicative depth 1.5 (since we only require an addition level of scalar multiplication).

Delayed Inverse NTT and Relinearization. In BGV/BFV, a ciphertext multiplication between ct_1 and ct_2 (each containing two ring elements) consists of four steps: (1) applying the Number Theoretic Transform (NTT) to both ciphertexts, (2) performing tensor products to obtain a ciphertext with three ring elements, (3) applying the Inverse NTT, and (4) relinearizing to reduce back to two ring elements. When evaluating $\sum_i \mathsf{ct}_{1,i} \times \mathsf{ct}_{2,i}$, we can keep all the ciphertexts in NTT form in advance (such that step (1) is not necessary), and then we only need to perform step (2) for each pair to compute $\mathsf{ct}'_i \leftarrow \mathsf{ct}_{1,i} \times \mathsf{ct}_{2,i}$, and then accumulate $\mathsf{ct}' \leftarrow \sum_i \mathsf{ct}'_i$. Lastly, we apply steps (3) and (4) once to ct'. This reduces both runtime and noise growth.

7.2 Methodology

We estimate the server runtime in our scheme $\mathsf{Laminate}_{\mathsf{base}}, \mathsf{Laminate}_{\mathsf{fold}}, \mathsf{Laminate}_{\mathsf{laned}}$ (cf. Table 1) with the operation counts in Section 6.3, using the same methodology as the closest prior works,

[38] A "folklore" conjecture stipulates that we can find irreducible polynomials of any degree that have at most 5 nonzero coefficients, though it does not assert the coefficients are low norm. See Section 3.4.3 of [71].

	1 lane		128 lane		16384 lane		Noise budget overhead	Multiplicative Depth
	Total proving runtime (hours)	Total proof size (MB)	Total proving runtime (hours)	Total proof size (MB)	Total proving runtime (hours)	Total proof size (MB)		
$\mathsf{Laminate}_{\mathsf{base}}$	13.9	2604	13.9	2604	13.9	2604	32	1
$\mathsf{Laminate}_{\mathsf{fold}}$	43.9	0.25	43.9	0.25	43.9	0.25	120	3.5
$\mathsf{Laminate}_{\mathsf{laned}}$	1.1	0.13	1.2	0.13	15.8	0.13	150	4.5
Phalanx [86]	$\mathsf{W}(1)+2.3$	58.10	$\mathsf{W}(128)+294.4$	7437	$\mathsf{W}(16384)+37683.2$	951910	360	6
Blind Fractal [40]	$\mathsf{W}(1)+9.3$	116.0	$\mathsf{W}(128)+1190.4$	14848	$\mathsf{W}(16384)+152371.2$	1900544	250	12

Table 20: Comparisons of concrete costs with prior works for the parameters in Section 7.2 with parameters in Section 7.2 (except for varying $B = 1, 128, 16384$). $\mathsf{W}(k)$ stands for the runtime of witness reduction and witness packing with respect to k disjoint lanes, which is overlooked by prior works as discussed in Section 7. Noise budget overhead refers to the bits of the noise budget that must be added to the original FHE payload evaluation in order to compute the proof. It assumes witness reduction introduces no levels, though this may not be true for prior works. Laminate is separated into the three variants: a base scheme (Table 17), a folded plus packed scheme (Table 18), and a scheme for B-laned payload circuits (Table 19). Runtimes are all single-threaded.

Phalanx [86] and Blind Fractal [40]. Specifically, we rely on operation counts and microbenchmarks, and we adopt similar parameter choices to ensure a fair comparison.

We consider B-laned payload circuits as defined in Definition 4.3 for varying B. Thus, the resulting payload circuit uses all N SIMD slots (one per lane) and contains only SIMD arithmetic (not utilizing our rotation gates), with $n = 2^{20}$ gates and $d \leq 32$ layers (where the multiplicative depth is ≤ 16), with $m = p = 0.5n$ (i.e., half being multiplications and half being additions). We use only B-laned payload circuits (Definition 4.3) instead of wide payload circuits (i.e., circuits with cross-slot operations such as rotations, Definition 5.1) since wide payload circuits are not supported by prior works. We assume each layer i contains the same number of gates $S_i = n/d$.[39] The resulting operation counts are shown in Section 6.3. Note that during computation, we assume an upper bound of 1 hour added for transcript packing for all cases for $\mathsf{Laminate}_{\mathsf{fold}}$ and $\mathsf{Laminate}_{\mathsf{laned}}$ as per the analysis in Section 6.2.2.

Microbenchmarks of BGV. We use the same VM instance type as [86] (Google Compute Cloud `N4` with 16GB RAM) and the same ring dimension $N = 2^{14}$. We perform microbenchmarks on the BGV scheme implemented in SEAL [72], as shown in Table 22. We choose BGV instead of BFV (as in [86]) because our evaluation involves substantially more ciphertext multiplications. However, these multiplications are aggregated, allowing us to use *delayed relinearization*. At a high level, relinearization is the most expensive component of ciphertext multiplication, but since it can be deferred until after aggregation (see Section 7.1), the amortized cost is significantly smaller. We therefore benchmark the primitive operations listed below. We use a ciphertext modulus of $\sim 2^{50}$ (such that HEXL acceleration can be best utilized [11]) and plaintext modulus $2^{16}+1$ with extension degree $R = 8$ (to achieve at least 100 bits of security in the GKR). For $x > 1$ RNS limbs, the runtime is multiplied by x. Table 20 is based on 2 RNS limbs for $\mathsf{Laminate}_{\mathsf{base}}$, 3 RNS limbs for $\mathsf{Laminate}_{\mathsf{fold}}$, and 4 RNS limbs for for $\mathsf{Laminate}_{\mathsf{laned}}$ (each with ≤ 50 bits, as discussed in Section 3.2).[40]

More specifically, one level of scalar by ciphertext multiplication takes $\sim$17 bits of noise budget; one level of plaintext by ciphertext and ciphertext by ciphertext multiplication takes $\sim$30 bits of noise budget; and one FOLD operation, which takes $\log(N)$ rotations, takes < 10 bits of noise budget[41] (tested using the SEAL library [72]). By using this (and leaving 17 bits for the plaintext

[39] As shown in Lemma 3.6, assuming that each layer has the same number of wires upper bounds the resulting proof size for a circuit with n gates and d layers.

[40] Note that if only counting the overhead, we only need $1, 3, 3$ RNS limbs respectively. However, we need to count for plaintext space and additional noise budget to ensure correct decryption. Therefore, we need $2, 3, 4$ instead.

[41] Note that this is the default noise consumption of rotations by SEAL [72]. However, rotation or key switching in general is highly flexible in terms of noise budget consumption and one may use a different way to realize key switching or rotation that takes a smaller or larger amount of noise budget in the underlying application. We use the

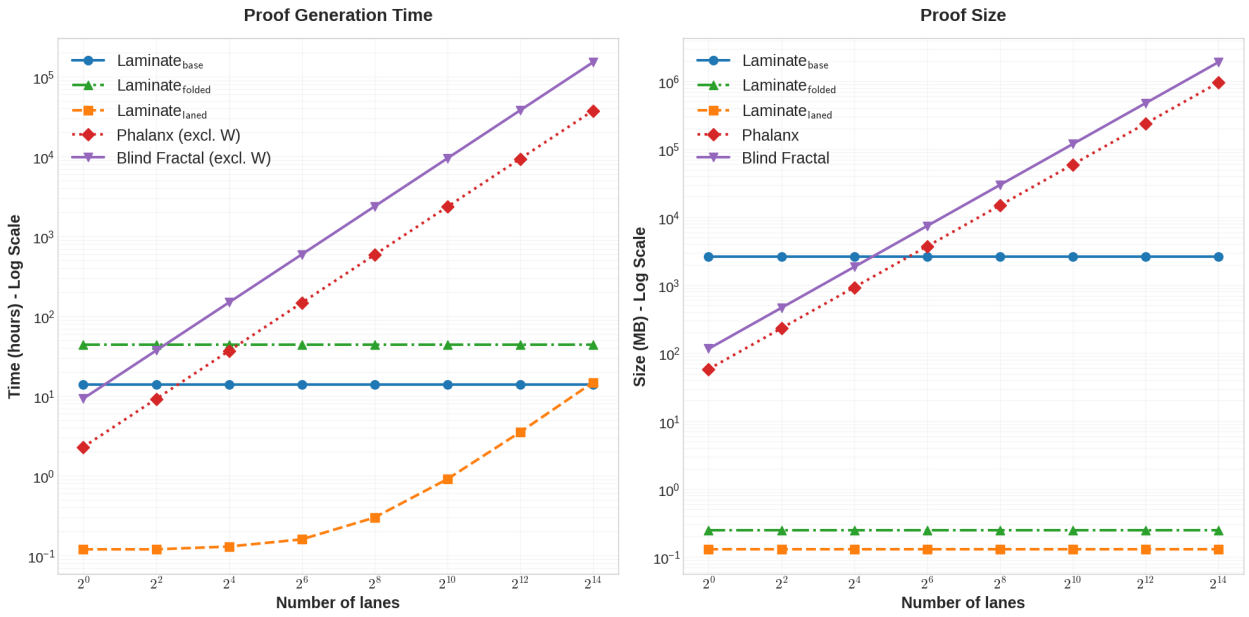

Figure 21: Proof generation time and proof size with respect to the number of lanes for Laminate (all three variants as in Table 20) compared to prior works Phalanx [86] and Blind Fractal [40] *excluding* their runtime for witness reduction, i.e., W in Table 20. Note that these efficiency metrics overlook some of our additional advantages, such as our smaller noise budget overhead and its implications for the payload evaluation (again, with the parameters in Section 7.2 except for varying B). We discuss this in more detail in Section 7.3.

Add	Scalar-by-ciphertext multiplication	Ciphertext-by-ciphertext multiplication (w/o relinearization)	Relinearization or rotation
0.022ms	0.027ms	0.064ms	1.331ms

Table 22: Microbenchmarks using SEAL library for our $N = 2^{14}$ and 1 RNS limb with ∼50 bit.

space), we obtain the amount of noise budget needed for each version of Laminate, and we transfer this to the number of RNS limbs by having each limb be at most 50 bits.

7.3 Comparisons

Performance. Using these microbenchmarks and delayed relinearization, we estimate our server runtime and proof size in Table 20 and Fig. 21. For Laminate$_{laned}$ and Laminate$_{fold}$, we additionally account for the SNARK runtime to compress the proof (see Section 6.2.2 for details).

As one can see, Laminate is strictly better than prior works in various settings: for the case where only one lane is evaluated (which is the main setting tested by prior works), Laminate$_{laned}$ is at least 2× faster than prior works [86, 40]. On the other hand, once the underlying BGV scheme leverages SIMD batching, our advantages become more significant. With 128 lanes, our runtime (for Laminate$_{laned}$) becomes $> 245\times$ faster. With 16384 lanes (full SIMD utilization), our runtime is $> 2385\times$ faster (for Laminate$_{laned}$). For proof size, Laminate$_{laned}$ is 446× for 1-laned and $7,322,384\times$ smaller for 2^{14}-laned payloads compared to [86]. In general, as shown in Fig. 21, Laminate$_{laned}$ strictly outperforms prior works for all settings for runtime and proof size (and smaller multiplicative depth overhead compared to honest payload evaluation).

For Laminate$_{base}$, the runtime becomes better than Laminate$_{laned}$ when the number of lanes is

default setting in [72] to demonstrate that rotation in most cases does not take much noise budget.

large and when the number of multiplications is larger (e.g., $m = 0.99n$ instead of $m = 0.5n$). On the other hand, $\mathsf{Laminate}_{\mathsf{fold}}$ is more general and can handle rotations while maintaining low proof sizes, with smaller multiplicative depth overhead.

Limitations of prior evaluation. While the comparison above already illustrates the superior performance of our construction, we highlight several further advantages. (1) We can match the base field directly, avoiding additional overhead in the underlying application. (2) We can generate the proof almost directly from the intermediate ciphertexts produced during FHE evaluation, without expensive reformatting. (3) Our depth overhead is significantly smaller. The first advantage is difficult to quantify, and we elaborate on the latter two below.

Overlooked costs for proof generation (witness reduction). Prior works fixes a plaintext field for their proof system, which may not match the plaintext field of the payload computation. In such cases, the SNARG prover expects different inputs than those that arise as intermediate states of payload computation, e.g. to emulate non-native arithmetic. As discussed in Section 1.1.1, these costs that are unaccounted for in prior work may be enormous. On the other hand, none of our schemes require a witness reduction step.

Overlooked costs for proof generation (witness packing). Prior works require a preprocessing step to convert intermediate FHE values into the witness format their proof systems expect.

In [86], if the FHE evaluation uses B SIMD slots per ciphertext (i.e., B-laned payloads), then matching the Phalanx input format requires $B \cdot n$ plaintext multiplications and $\log(B) \cdot B \cdot n$ rotations to produce $B \cdot n$ ciphertexts, each containing the same value replicated in all slots. With our parameters and $B = 128$ for $N = 16384$, the cost exceeds 10 hours. For $B = 1$, this preprocessing disappears, but such a configuration is unrealistic for BFV/BGV-based workloads.

In [40] and $\mathsf{Laminate}_{\mathsf{laned}}$, witness packing is also needed, in a similar way. It only costs $O(n)$ plaintext by ciphertext multiplications (in NTT form), concretely minutes, and these costs are added to our total proving runtime in Table 20 and Fig. 21 for $\mathsf{Laminate}_{\mathsf{laned}}$ but not [40].

Once these overlooked costs are included, our performance advantage becomes even larger.

Additional gain from low-depth overhead. A further benefit of our construction is its near-minimal multiplicative depth overhead. Concretely, our variants require only $32, 120, 150$ bits of noise budget (and asymptotically a constant multiplicative depth of $1, 3.5, 4.5$ respectively), corresponding to $1, 3$ and 3 additional RNS limbs; see Table 20.[42] In contrast, prior works require either sacrificing a quasi-linear server circuit or introducing $\omega(1)$ additional levels asymptotically, and concretely for the parameters chosen in prior works, at least four additional levels concretely and ≥ 250 bits of noise budget, corresponding to ≥ 5 RNS limbs.

Note that multiplicative depth does not directly translate into noise budget, as it depends on the underlying parameter choices. For example, [40] adopts specialized parameters for GBGV rather than standard BGV, which reduce the noise consumption per level and therefore incur a smaller noise budget cost than [86]. In terms of concrete performance, the noise budget—implemented via RNS limbs, as discussed in Section 3.2—is the primary cost driver. Accordingly, we focus on the noise budget in our comparisons below.

For a circuit requiring 20 RNS limbs (see Section 3.2), our proof requires only one to three extra limbs, while prior works require at least four. Thus, the underlying FHE computation incurs significantly greater overhead when using prior works. If each level has roughly the same number of homomorphic operations, then the payload evaluation overhead is $\sim 1.16\times, 1.29\times, 1.29\times$

[42] Note that during proof generation, we use $2, 4, 3$ RNS limbs to calculate proof generation time since we need to account for the plaintext space and additional noise budget to ensure correct decryption. However, here, the additional overhead is only $1, 3, 3$ instead.

Circuit Type	8 Levels			20 Levels		
	Honest Eval (hrs)	Laminate Payload	Laminate Proving	Honest Eval (hrs)	Laminate Payload	Laminate Proving
Gates distributed uniformly over all multiplicative depths						
Half Adds/Mults	0.93	1.67×	47.2×	2.17	1.29×	20.2×
90% Mults	1.65	1.67×	23.3×	3.85	1.29×	10.0×
90% Adds	0.21	1.67×	66.3×	0.49	1.29×	28.4×
90% of gates at the highest multiplicative depth						
Half Adds/Mults	1.57	1.40×	27.9×	3.92	1.16×	11.2×
90% Mults	2.78	1.40×	13.8×	6.96	1.16×	5.5×
90% Adds	0.35	1.40×	39.7×	0.88	1.16×	15.9×

Table 23: Server evaluation times with $\mathsf{Laminate}_{\mathsf{fold}}$. The top section shows circuits with gates distributed uniformly over layers; the bottom section shows circuits where 90% of the gates are at the input layer. "Honest Eval" denotes the honest evaluation time of the payload without integrity. "VCoED Payload" denotes the VCoED evaluation of the payload relative to honest execution. "VCoED Proving" denotes the VCoED evaluation of the prover relative to honest payload execution.

for $\mathsf{Laminate}_{\mathsf{base}}, \mathsf{Laminate}_{\mathsf{fold}}, \mathsf{Laminate}_{\mathsf{laned}}$ respectively, while it is ∼1.48× for Blind Fractal [40] and ∼1.76× for Phalanx [86].

This depth overhead also increases ciphertext sizes and evaluation-key sizes. For example, suppose a circuit computation requires a 1200-bit ciphertext modulus. Under our scheme, the communication overhead from increasing the noise budget to $1232, 1350, 1320$ bits is $\sim 2.7\%, 12.5\%, 10\%$ respectively for our three variants. For prior works, the overhead is $\sim 20.9\%, 30\%$ respectively. For smaller-depth circuits, such as using 600 bits, the overhead doubles. For applications with large inputs, this difference is significant: for 10GB of ciphertext input or evaluation keys, prior works introduce 1–2GB of extra communication compared to our scheme for 1200-bit noise budget and doubles for 600-bit noise budget.

More generally, the communication overhead is approximately L/X, where L is the proof-system noise-budget overhead and X is the budget required by the original circuit. The computation overhead is roughly $\sum_{i\in[d]} C_i \cdot L/(X \cdot (d-i)/d)$, where C_i is the number of multiplicative operations at depth i in a circuit of depth d.

7.4 Overhead Compared to Honest Evaluation

Lastly, we discuss the overhead of our construction compared to payload evaluation via FHE *without* integrity, i.e., assuming an honest-but-curious server.

Parameters. We first discuss the parameters we compare against. Again, we use $n = 2^{20}$ gates (of either ciphertext additions or multiplications).

We start with the base case. Since $N = 2^{14}$ is used as ring dimension, the maximum noise budget is ∼440 bits to guarantee 128-bit computational security [72]. Excluding the 120 bits needed to generate the proof and 17 bits of plaintext, the noise budget remaining is ∼300, which is sufficient for ≈ 8 multiplicative depth, so we set the number of RNS limbs to 8. We then assume that all these gates are uniformly distributed across each multiplicative depth, and half of them are additions and half of them are multiplications (for simplicity, assuming only ciphertext multiplications).

Then, we can extend the parameters to two additional settings: (1) gates are mostly distributed

with the highest multiplicative depth, instead of uniformly distributed over all the depths[43]; and (2) most gates (e.g., 90%) of the gates are multiplications or are additions against half additions and half multiplications.

Lastly, we also explore the case where there are 20 multiplicative depth instead of 8. This is possible if we use $N = 2^{15}$ instead of 2^{14}, and since the change of ring dimension affects the payload computation and our proof generation with the same ratio, the overhead ratio remains the same. Thus, we still use $N = 2^{14}$ to perform the comparisons for simplicity. These parameters are summarized in Table 23.

Proof generation overhead. With these parameters, we show in Table 23 the payload generation runtime (using our microbenchmarks in Section 7.2) and the proof generation overhead of $\mathsf{Laminate}_{\mathsf{fold}}$ compared to the proof generation time. As shown, our proof generation creates 5.5×—66.3× over overhead compared to the payload generation. The scenario where our proof generation has the lowest overhead (∼5.5×) is when most gates are multiplications and when most gates are calculated with the highest depth. The scenario where our proof generation has the input overhead (∼66.3×) is when the most gates are additions and the gates are uniformly distributed. This is as expected since our proof generation runtime is essentially independent of these scenarios.

Payload generation overhead from additional multiplicative depth. As mentioned above, there is one additional overhead: since the proof requires additional multiplicative depth, the payload generation induces some additional overhead. Each operation has an overhead of $\frac{i+x}{i}$ at level i, for x being the number of extra RNS limbs for the VCoED proof generation. For $\mathsf{Laminate}_{\mathsf{fold}}$, this overhead is at most 89% (when all the gates are uniformly distributed for 8 levels). Nonetheless, this overhead is relatively small compared to the proof generation overhead. However, recall that this still overlooks the potential overhead from prior works on witness reduction and witness packing discussed before.

Acknowledgments

Zeyu Liu is supported by Yale CADMY.

[43]There is another scenario where most gates have the lowest multiplicative depth. The payload generation time of this case is close to when the multiplicative depth is only 1, so we omit this case.

References

[1] A. Alexandru, A. A. Badawi, D. Micciancio, and Y. Polyakov. Application-aware approximate homomorphic encryption: Configuring FHE for practical use. Cryptology ePrint Archive, Paper 2024/203, 2024.

[2] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam. Ligero: lightweight sublinear arguments without a trusted setup. *Des. Codes Cryptogr.*, 91(11):3379–3424, 2023.

[3] S. Angel, H. Chen, K. Laine, and S. T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2018.

[4] D. F. Aranha, A. Costache, A. Guimarães, and E. Soria-Vazquez. HELIOPOLIS: verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. *Asiacrypt 2024*, 2024.

[5] S. Atapoor, K. Baghery, H. V. L. Pereira, and J. Spiessens. Verifiable FHE via lattice-based snarks. *IACR Commun. Cryptol.*, (1), 2024.

[6] L. Babai, L. Fortnow, L. A. Levin, and M. Szegedy. Checking computations in polylogarithmic time. In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing*. ACM, 1991.

[7] A. A. Badawi, A. Alexandru, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, C. Pascoe, Y. Polyakov, I. Quah, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca. OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915, 2022. https://eprint.iacr.org/2022/915.

[8] E. Ben-Sasson, A. Chiesa, and N. Spooner. Interactive oracle proofs. *IACR Cryptol. ePrint Arch.*, 2016.

[9] O. Bernard, M. Joye, N. P. Smart, and M. Walter. Drifting towards better error probabilities in fully homomorphic encryption schemes. In S. Fehr and P.-A. Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 181–211. Springer, Cham, May 2025.

[10] D. Bitan, Z. DeStefano, S. Goldwasser, Y. Ishai, Y. T. Kalai, and J. Thaler. Sum-check protocol for approximate computations. Cryptology ePrint Archive, Paper 2025/2152, 2025.

[11] F. Boemer, S. Kim, G. Seifu, F. D.M. de Souza, and V. Gopal. Intel hexl: Accelerating homomorphic encryption with intel avx512-ifma52. In *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC '21, page 57–62, New York, NY, USA, 2021. Association for Computing Machinery.

[12] A. Bois, I. Cascudo, D. Fiore, and D. Kim. Flexible and efficient verifiable computation on encrypted data. In J. A. Garay, editor, *Public-Key Cryptography - PKC 2021*.

[13] D. Boneh, S. Eskandarian, L. Hanzlik, and N. Greco. Single secret leader election. In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, AFT '20, page 12–24, New York, NY, USA, 2020. Association for Computing Machinery.

[14] J. Bootle, A. Chiesa, and J. Groth. Linear-time arguments with sublinear verification from tensor codes. In R. Pass and K. Pietrzak, editors, *Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 19–46. Springer, 2020.

[15] Z. Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In *CRYPTO 2012*. Springer-Verlag, 2012.

[16] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.

[17] Z. Brakerski and V. Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In R. Ostrovsky, editor, *52nd FOCS*, pages 97–106. IEEE Computer Society Press, Oct. 2011.

[18] B. Bünz, B. Fisch, and A. Szepieniec. Transparent snarks from DARK compilers. In A. Canteaut and Y. Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706. Springer, 2020.

[19] G. Cao, S. Chatel, and C. Knabenhans. HE-based on-the-fly MPC, revisited: Universal composability, approximate and imperfect computation, circuit privacy. Cryptology ePrint Archive, Paper 2025/1845, 2025.

[20] I. Cascudo, A. Costache, D. Cozzo, D. Fiore, A. Guimarães, and E. Soria-Vazquez. Verifiable computation for approximate homomorphic encryption schemes. In Y. T. Kalai and S. F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VII*, Lecture Notes in Computer Science, pages 643–677. Springer, 2025.

[21] M. Checri, R. Sirdey, A. Boudguiga, and J.-P. Bultel. On the practical cpad security of "exact" and threshold fhe schemes and libraries. In *Advances in Cryptology – CRYPTO 2024: 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part III*, page 3–33, Berlin, Heidelberg, 2024. Springer-Verlag.

[22] H. Chen, Z. Huang, K. Laine, and P. Rindal. Labeled PSI from fully homomorphic encryption with malicious security. In *ACM CCS 2018*. ACM Press, Oct. 2018.

[23] H. Chen, K. Laine, and P. Rindal. Fast private set intersection from homomorphic encryption. In *ACM CCS 2017*. ACM Press, Oct. / Nov. 2017.

[24] J. H. Cheon, H. Choe, A. Passelègue, D. Stehlé, and E. Suvanto. Attacks against the IND-CPA-d security of exact FHE schemes. CCS 2024.

[25] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 409–437. Springer, 2017.

[26] A. Chiesa, M. Dall'Agnol, Z. Di, Z. Guan, and N. Spooner. Quantum rewinding for iop-based succinct arguments. In B. Applebaum and H. R. Lin, editors, *Theory of Cryptography - 23rd International Conference, TCC 2025, Aarhus, Denmark, December 1-5, 2025, Proceedings, Part III*, volume 16270 of *Lecture Notes in Computer Science*, pages 460–479. Springer, 2025.

[27] A. Chiesa, D. Ojha, and N. Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. In *Advances in Cryptology - EUROCRYPT 2020*. Springer, 2020.

[28] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *Advances in Cryptology – ASIACRYPT 2016*, pages 3–33. Springer, 2016.

[29] K. Cong, R. C. Moreno, M. B. da Gama, W. Dai, I. Iliashenko, K. Laine, and M. Rosenberg. Labeled PSI from homomorphic encryption with reduced computation and communication. In *ACM CCS 2021*. ACM Press, Nov. 2021.

[30] G. Cormode, M. Mitzenmacher, and J. Thaler. Practical verified computation with streaming interactive proofs. In *Innovations in Theoretical Computer Science 2012*. ACM.

[31] G. Cormode, J. Thaler, and K. Yi. Verifying computations with streaming interactive proofs. *Proc. VLDB Endow.*, (1), 2011.

[32] A. Dalvi, A. Jain, S. Moradiya, R. Nirmal, J. Sanghavi, and I. Siddavatam. Securing neural networks using homomorphic encryption. In *2021 International Conference on Intelligent Technologies (CONIT)*, pages 1–7, 2021.

[33] B. E. Diamond and J. Posen. Succinct arguments over towers of binary fields. *IACR Cryptol. ePrint Arch.*, 2023.

[34] C. Dobraunig, L. Grassi, A. Guinet, and D. Kuijsters. Ciminion: Symmetric encryption based on Toffoli-gates over large finite fields. In A. Canteaut and F. Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021*, volume 12697 of *Lecture Notes in Computer Science*, pages 3–34. Springer, 2021.

[35] L. Ducas and D. Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *Advances in Cryptology – EUROCRYPT 2015*, pages 617–640. Springer, 2015.

[36] J. Fan and F. Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012. https://ia.cr/2012/144.

[37] A. Fiat and A. Shamir. How to prove yourself: practical solutions to identification and signature problems. In *CRYPTO '86*. Springer-Verlag, 1987.

[38] D. Fiore, A. Nitulescu, and D. Pointcheval. Boosting verifiable computation on encrypted data. In A. Kiayias, M. Kohlweiss, P. Wallden, and V. Zikas, editors, *Public-Key Cryptography - PKC 2020*.

[39] B. Fisch, A. Lazzaretti, Z. Liu, and C. Papamanthou. Thorpir: Single server pir via homomorphic thorp shuffles. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2024.

[40] M. Gama, E. H. Beni, J. Kang, J. Spiessens, and F. Vercauteren. Blind zkSNARKs for private proof delegation and verifiable computation over encrypted data. *IACR Cryptol. ePrint Arch.*, 2024.

[41] C. Ganesh, A. Nitulescu, and E. Soria-Vazquez. Rinocchio: Snarks for ring arithmetic. *J. Cryptol.*, (4), 2023.

[42] S. Garg, A. Goel, and M. Wang. How to prove statements obliviously? In *Advances in Cryptology - CRYPTO 2024*. Springer, 2024.

[43] R. Geelen and F. Vercauteren. Bootstrapping for BGV and BFV revisited. Cryptology ePrint Archive, Paper 2022/1363, 2022.

[44] R. Gennaro, C. Gentry, and B. Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In *Advances in Cryptology - CRYPTO 2010*. Springer, 2010.

[45] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009*. ACM, 2009.

[46] S. Goldwasser, G. N. Rothblum, and Y. T. Kalai. Delegating computation: Interactive proofs for muggles. *Electron. Colloquium Comput. Complex.*, TR17-108, 2017.

[47] A. Golovnev, J. Lee, S. T. V. Setty, J. Thaler, and R. S. Wahby. Brakedown: Linear-time and field-agnostic snarks for R1CS. In H. Handschuh and A. Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023*, volume 14082 of *Lecture Notes in Computer Science*, pages 193–226. Springer, 2023.

[48] J. Groth. On the size of pairing-based non-interactive arguments. In M. Fischlin and J. Coron, editors, *Advances in Cryptology - EUROCRYPT 2016*.

[49] A. Gruen. Some improvements for the PIOP for zerocheck. *IACR Cryptol. ePrint Arch.*, page 108, 2024.

[50] A. Henzinger, E. Dauterman, H. Corrigan-Gibbs, and N. Zeldovich. Private web search with tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 396–416, New York, NY, USA, 2023. Association for Computing Machinery.

[51] W. jie Lu, Z. Huang, C. Hong, Y. Ma, and H. Qu. PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021.

[52] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *USENIX Security 2018*. USENIX Association, Aug. 2018.

[53] A. Kim, Y. Polyakov, and V. Zucca. Revisiting homomorphic encryption schemes for finite fields. In *ASIACRYPT 2021*. Springer, 2021.

[54] J. Kim, J. Seo, and Y. Song. Simpler and faster BFV bootstrapping for arbitrary plaintext modulus from CKKS. Cryptology ePrint Archive, Paper 2024/109, 2024.

[55] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim, and J.-S. No. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access*, 10:30039–30054, 2022.

[56] J.-W. Lee, E. Lee, Y.-S. Kim, and J.-S. No. Rotation key reduction for client-server systems of deep neural network on fully homomorphic encryption. In *Advances in Cryptology – ASIACRYPT 2023*, pages 36–68. Springer Nature Singapore, 2023.

[57] K. Lee and Y. Yeo. SophOMR: Improved oblivious message retrieval from SIMD-aware homomorphic compression. Cryptology ePrint Archive, Paper 2024/1814, 2024.

[58] B. Li and D. Micciancio. On the security of homomorphic encryption on approximate numbers. In A. Canteaut and F.-X. Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 648–677. Springer, Cham, Oct. 2021.

[59] B. Li, D. Micciancio, M. Raykova, and M. Schultz. Hintless single-server private information retrieval. In L. Reyzin and D. Stebila, editors, *CRYPTO 2024, Part IX*, volume 14928 of *LNCS*, pages 183–217. Springer, Cham, Aug. 2024.

[60] F. Liu, H. Liang, T. Zhang, Y. Hu, X. Xie, H. Tan, and Y. Yu. Hasteboots: Proving FHE bootstrapping in seconds. *IACR Cryptol. ePrint Arch.*, 2025.

[61] Z. Liu, K. Sotiraki, E. Tromer, and Y. Wang. Snake-eye resistant PKE from LWE for oblivious message retrieval and robust encryption. In S. Fehr and P.-A. Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 126–156. Springer, Cham, May 2025.

[62] Z. Liu and E. Tromer. Oblivious message retrieval. In *CRYPTO 2022, Part I*, Aug. 2022.

[63] Z. Liu, E. Tromer, and Y. Wang. Group oblivious message retrieval. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4367–4385, 2024.

[64] Z. Liu, E. Tromer, and Y. Wang. PerfOMR: Oblivious message retrieval with reduced communication and computation. Usenix Security'24, 2024.

[65] Z. Liu and Y. Wang. Amortized functional bootstrapping in less than 7 ms, with $\tilde{\mathrm{O}}(1)$ polynomial multiplications. In J. Guo and R. Steinfeld, editors, *ASIACRYPT 2023, Part VI*, volume 14443 of *LNCS*, pages 101–132. Springer, Singapore, Dec. 2023.

[66] Z. Liu and Y. Wang. Relaxed functional bootstrapping: A new perspective on BGV/BFV bootstrapping. In K.-M. Chung and Y. Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 208–240. Springer, Singapore, Dec. 2024.

[67] Z. Liu, Y. Wang, and B. Fisch. IND-CPA-d of relaxed functional bootstrapping: A new attack, a general fix, and a stronger model. ACM CCS 2025, 2025.

[68] C. Lund, L. Fortnow, H. J. Karloff, and N. Nisan. Algebraic methods for interactive proof systems. *J. ACM*, (4), 1992.

[69] S. J. Menon and D. J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *2022 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2022.

[70] D. Micciancio and J. Sorrell. Ring packing and amortized FHEW bootstrapping. In I. Chatzigiannakis, C. Kaklamanis, D. Marx, and D. Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 100:1–100:14. Schloss Dagstuhl, July 2018.

[71] G. L. Mullen and D. Panario. *Handbook of Finite Fields*. Chapman & Hall/CRC, 1st edition, 2013.

[72] Microsoft SEAL (release 4.1). https://github.com/Microsoft/SEAL, Jan. 2023. Microsoft Research, Redmond, WA.

[73] S. Setty. Spartan, 2025. Accessed: 2025-12-05.

[74] S. T. V. Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Advances in Cryptology - CRYPTO 2020*. Springer, 2020.

[75] S. T. V. Setty, J. Thaler, and R. S. Wahby. Unlocking the lookup singularity with lasso. In M. Joye and G. Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI*, volume 14656 of *Lecture Notes in Computer Science*, pages 180–209. Springer, 2024.

[76] N. P. Smart and F. Vercauteren. Fully homomorphic SIMD operations. *DCC*, 71(1):57–81, 2014.

[77] J. Thaler. Time-optimal interactive proofs for circuit evaluation. In *Advances in Cryptology - CRYPTO 2013*. Springer, 2013.

[78] J. Thaler. A note on the gkr protocol. Technical report, Georgetown University, 2015.

[79] J. Thaler. Proofs, arguments, and zero-knowledge. *Found. Trends Priv. Secur.*, (2-4), 2022.

[80] L. T. Thibault and M. Walter. Towards verifiable FHE in practice: Proving correct execution of tfhe's bootstrapping using plonky2. *IACR Cryptol. ePrint Arch.*, 2024.

[81] A. Viand, C. Knabenhans, and A. Hithnawi. Verifiable fully homomorphic encryption. *WAHC'24*, 2023.

[82] Y. Wang and F. Zhang. Qelect: Lattice-based single secret leader election made practical. Usenix Security 2025, 2025.

[83] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In A. Boldyreva and D. Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 733–764. Springer, 2019.

[84] H. Zeilberger, B. Chen, and B. Fisch. Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In *Advances in Cryptology - CRYPTO 2024*. Springer, 2024.

[85] J. Zhang, W. Wang, Y. Zhang, and Y. Zhang. Doubly efficient interactive proofs for general arithmetic circuits with linear prover time. *IACR Cryptol. ePrint Arch.*, 2020.

[86] X. Zhang, R. Wang, Z. Liu, B. Xiang, Y. Deng, B. Fisch, and X. Lu. Phalanx: An FHE-friendly SNARK for verifiable computation on encrypted data. CCS'25, 2025.

[87] M. Zhou, W.-K. Lin, Y. Tselekounis, and E. Shi. Optimal single-server private information retrieval. In C. Hazay and M. Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 395–425. Springer, Cham, Apr. 2023.

[88] Z. Zhou, Y. Li, Y. Wang, Z. Yang, B. Zhang, C. Hong, T. Wei, and W. Chen. ZHE: Efficient zero-knowledge proofs for HE evaluations. In *2025 IEEE SP*, 2025. Extended version at https://eprint.iacr.org/2025/770.

A Proof of Lemma 3.7

In this section, we give a full proof of the following lemma.

Lemma A.1. *Fix a layer reduction $i \in \{0, \ldots, d-1\}$. The prover can compute (sparse) tables $\hat{A}_j$ (and $\hat{M}_j$) at the beginning of the jth round of sumcheck such that for layer challenge $\vec{r} \in \mathbb{F}_{q^R}^{s_i}$, sumcheck challenges $\vec{\gamma} = (\gamma_0, \ldots, \gamma_{j-1}) \in \mathbb{F}_{q^R}^{j}$, $X \in \{0, 2\}$, and $w \in \{0,1\}^{2s_{i+1}-j-1}$*

$$\hat{A}_j[X][w] = \tilde{\mathsf{add}}_i(\vec{r}, \vec{\gamma}, X, \vec{w})$$
$$\hat{M}_j[X][w] = \tilde{\mathsf{mul}}_i(\vec{r}, \vec{\gamma}, X, \vec{w})$$

Moreover, the prover requires $O(S_i \cdot (s_i + s_{i+1}))$ $\mathbb{F}_q$-ops in total across all sumcheck rounds.

We start with some machinery. Fix a layer $i \in \{0, \ldots, d-1\}$. Suppose for simplicity, that the statement we wish to prove is the following claim about layer i.

$$v_i = \tilde{L}_i(r) = \sum_{\vec{x} \in \{0,1\}^{s_{i+1}}} \sum_{y \in \{0,1\}^{s_{i+1}}} g^{(i)}(\vec{r}, \vec{x}, \vec{y})$$

for some evaluation point $r \in \mathbb{F}_{q^R}^{s_i}$ and evaluation claim $v_i \in \mathbb{F}_{q^R}$.

Remark A.2. Indeed, as we presented our protocol, we will actually be proving a claim about $\tilde{L}_i(r_a) + \alpha \tilde{L}_i(r_b)$, but we ignore this for ease of readability.

At each round $j \in \{0, \ldots, 2s_{i+1} - 1\}$ of the sumcheck, given challenges $\vec{\gamma} = (\gamma_0, \ldots, \gamma_{j-1})$, the prover needs to efficiently access evaluations of the gate functions. We define (sparse) tables A_j and M_j that store these values:

$$A_j[w] = \tilde{\mathsf{add}}_i(\vec{r}, \vec{\gamma}, \vec{w})$$
$$M_j[w] = \tilde{\mathsf{mul}}_i(\vec{r}, \vec{\gamma}, \vec{w})$$

for all $w \in \{0,1\}^{2s_{i+1}-j}$, where $\vec{r} \in \mathbb{F}_{q^R}^{s_i}$ is the layer i challenge. Because $\tilde{\mathsf{add}}_i$ is sparse, these tables will also be sparse, containing at most S_i non-zero entries. The prover computes them efficiently in two phases:

Initial table computation (A_0, M_0). The prover first computes the initial tables A_0 and M_0. We show this for A_0. From the definition of an MLE (c.f. Section 2.5):

$$\begin{aligned} A_0[x][y] &= \tilde{\mathsf{add}}_i(\vec{r}, \vec{x}, \vec{y}) \\ &= \sum_{z \in \{0,1\}^{s_i}} \mathsf{add}_i(z, x, y) \cdot \tilde{\mathsf{eq}}(\vec{z}, \vec{r}) \\ &= \sum_{(z,x,y) \in \mathsf{add}_i^{-1}(1)} \tilde{\mathsf{eq}}(\vec{z}, \vec{r}) \end{aligned}$$

The prover only needs to iterate over the (at most) S_i non-zero entries of add_i. Each $\tilde{\mathsf{eq}}(\vec{z}, \vec{r})$ evaluation (an inner product) takes $O(s_i)$ $\mathbb{F}_q$-ops. Therefore, computing the sparse tables A_0 and M_0 takes $O(s_i \cdot S_i)$ $\mathbb{F}_q$-ops.

Updating the tables ($A_j \to A_{j+1}$). Given the tables A_j, M_j from round j and a new challenge γ_j, the prover computes A_{j+1}, M_{j+1}. This is a simple linear combination:

$$\begin{aligned} A_{j+1}[w] &= \tilde{\mathsf{add}}_i(\vec{r}, \gamma_0, \ldots, \gamma_j, \vec{w}) \\ &= (1-\gamma_j) \cdot \tilde{\mathsf{add}}_i(\vec{r}, \gamma_0, \ldots, \gamma_{j-1}, 0, \vec{w}) + \gamma_j \cdot \tilde{\mathsf{add}}_i(\vec{r}, \gamma_0, \ldots, \gamma_{j-1}, 1, \vec{w}) \\ &= (1-\gamma_j) \cdot A_j[0||w] + \gamma_j \cdot A_j[1||w] \end{aligned}$$

The prover only iterates over the w's for which $A_j[0||w]$ or $A_j[1||w]$ is non-zero. Since A_j has at most S_i non-zero entries, this update step takes only $O(S_i)$ $\mathbb{F}_q$-ops.

Finally, we are ready to prove the main lemma.

Proof. This follows from the two phases described above.

1. **Initial computation:** Computing A_0 and M_0 requires iterating over S_i non-zero entries and performing an $O(s_i)$ operation for each. The total cost is $O(S_i \cdot s_i)$.
2. **Per-round computation:** For each of the $j \in \{0, \ldots, 2s_{i+1} - 1\}$ rounds, the prover updates the tables (e.g., $A_j \to A_{j+1}$) and computes the necessary round polynomial evaluations (e.g., $\hat{A}_j[X][\vec{w}]$).
   - The update $A_j \to A_{j+1}$ requires $O(S_i)$ ops, as it's a linear combination over at most S_i non-zero entries.
   - Computing the round polynomial evaluations (e.g., for $X \in \{0, 2\}$) also involves a linear extrapolation from $A_j[0][\vec{w}]$ and $A_j[1][\vec{w}]$, which again takes $O(S_i)$ ops for all w.

   The total cost across all $2s_{i+1}$ sumcheck rounds is $O(s_{i+1} \cdot S_i)$.

Combining both phases, the total work is $O(S_i \cdot s_i + S_i \cdot s_{i+1}) = O(S_i \cdot (s_i + s_{i+1}))$ $\mathbb{F}_q$-ops. □

B Proof of Theorem 5.3

We show that $\tilde{\mathsf{shift}}_N(\vec{Z}, \vec{X}, \vec{O})$ can be evaluated at any point in $O(\log N)$ field operations.

Carry-chain decomposition. The key observation is that the modular addition $\{z\} = \{x\} + \{o\}$ mod N decomposes bit-by-bit via the standard binary addition carry chain. For Boolean inputs $(x_i, o_i, c_i) \in \{0,1\}^3$, the ith sum bit is $s_i = x_i \oplus o_i \oplus c_i$ and the next carry is $c_{i+1} = \mathsf{MAJ}(x_i, o_i, c_i)$, with initial carry $c_0 = 0$. The function shift_N checks that $z_i = s_i$ for every bit position $i \in \{0, \ldots, \log N - 1\}$, while the final carry $c_{\log N}$ is unconstrained (since we compute modulo $N = 2^{\log N}$).

We define a *per-bit factor* φ_i that checks whether bit position i is consistent with a pair of incoming and outgoing carries $(c_{\text{in}}, c_{\text{out}})$:

$$\varphi_i(z_i, x_i, o_i;\, c_{\text{in}}, c_{\text{out}}) := \mathbf{1}\big[z_i = x_i \oplus o_i \oplus c_{\text{in}}\big] \cdot \mathbf{1}\big[c_{\text{out}} = \mathsf{MAJ}(x_i, o_i, c_{\text{in}})\big]. \tag{32}$$

Summing over all $2^{\log N}$ Boolean carry sequences $(c_1, \ldots, c_{\log N}) \in \{0,1\}^{\log N}$ yields:

$$\mathsf{shift}_N(\vec{z}, \vec{x}, \vec{o}) = \sum_{c_1, \ldots, c_{\log N} \in \{0,1\}} \prod_{i=0}^{\log N - 1} \varphi_i(z_i, x_i, o_i;\, c_i, c_{i+1}). \tag{33}$$

MLE via multilinear extension of per-bit factors. For each fixed pair $(c_{\text{in}}, c_{\text{out}}) \in \{0,1\}^2$, the factor $\varphi_i(z_i, x_i, o_i;\, c_{\text{in}}, c_{\text{out}})$ selects the valid (z_i, x_i, o_i) triples. Its multilinear extension $\tilde{\varphi}_i(Z_i, X_i, O_i;\, c_{\text{in}}, c_{\text{out}})$ in the variables (Z_i, X_i, O_i) is a degree-3 polynomial computable in $O(1)$ field operations. The multilinear extension of shift_N is obtained by replacing each φ_i in Eq. (33) with $\tilde{\varphi}_i$:

$$\tilde{\mathsf{shift}}_N(\vec{Z}, \vec{X}, \vec{O}) = \sum_{c_1,\ldots,c_{\log N} \in \{0,1\}} \prod_{i=0}^{\log N - 1} \tilde{\varphi}_i(Z_i, X_i, O_i;\, c_i, c_{i+1}). \tag{34}$$

This holds because the carry variables $(c_1, \ldots, c_{\log N})$ are summed over $\{0,1\}^{\log N}$ (not extended to non-Boolean values), so the multilinear extension acts only on the (Z_i, X_i, O_i) variables.

Dynamic programming evaluation. The sum in Eq. (34) has $2^{\log N}$ terms (one per carry sequence), but its structure as a chain of local factors, each involving only adjacent carries c_i, c_{i+1}, admits efficient evaluation via dynamic programming. We maintain a 2-element state vector $\vec{v} \in \mathbb{F}^2$ indexed by carry value $c \in \{0,1\}$, initialized to $\vec{v} = (1, 0)$ (reflecting $c_0 = 0$). At each bit position $i = 0, \ldots, \log N - 1$, we update:

$$v'[c_{\text{out}}] \leftarrow \sum_{c_{\text{in}} \in \{0,1\}} \tilde{\varphi}_i(Z_i, X_i, O_i;\, c_{\text{in}},\, c_{\text{out}}) \cdot v[c_{\text{in}}], \quad \text{for } c_{\text{out}} \in \{0,1\}. \tag{35}$$

After processing all $\log N$ bits, the result is $\tilde{\mathsf{shift}}_N(\vec{Z}, \vec{X}, \vec{O}) = v[0] + v[1]$, where the sum over both final carry values accounts for the unconstrained carry $c_{\log N}$. Each iteration performs $O(1)$ field operations (four evaluations of $\tilde{\varphi}_i$ and two additions), giving $O(\log N)$ total. □

C Proof of Theorem 6.1

Completeness, efficiency, and communication complexity follow directly from the construction. We establish state-restoration soundness in the random oracle model via a hybrid argument.

Proof overview. We proceed in two steps. First, we define a "trivial" variant of the compiled protocol in which the prover sends the unpacked ciphertexts directly (instead of a SNARK proof) in the final round. We show that this trivial variant preserves state-restoration soundness up to hash collision probability. Then, we argue that replacing the explicit consistency check with a SNARK proof-of-packing introduces at most the knowledge error $\kappa_{\mathsf{SNARK}}(\lambda)$.

Step 1: trivial packing variant. Consider a modified protocol π'_{triv} that proceeds identically to π' except in the final round: instead of sending a SNARK proof π_{outer}, the prover sends all of the aligned ciphertexts $\{\mathsf{ct}_{\text{rot}}^{(i,u)}\}_{i\in[r],u\in[f_i]}$ together with the packed ciphertexts $\{\mathsf{ct}_{\text{final},\ell}\}_{\ell=1}^{Z}$. The verifier $\mathsf{V}'_{\mathsf{triv}}$ checks the following:

1. For each round $i \in [r]$: the aligned ciphertexts $\{\mathsf{ct}_{\text{rot}}^{(i,u)}\}_{u=1}^{f_i}$ are consistent with the commitment C_i, i.e., $H(\{\mathsf{ct}_{\text{rot}}^{(i,u)}\}_{u=1}^{f_i}) = C_i$.
2. The packed ciphertexts satisfy the packing relation Eq. (26).
3. After reconstructing the per-round ciphertext messages from the packed ciphertexts, the BhIP verifier $\mathcal{E}[\pi].\mathsf{V}$ accepts.

We claim π'_{triv} has state-restoration soundness error at most $\varepsilon^{\text{sr}}_{\mathcal{E}[\pi]}(t, n) + \text{negl}(\lambda)$.

Let $\mathcal{A}'$ be a state-restoration adversary against π'_{triv} with move budget t. We construct a state-restoration adversary $\mathcal{A}$ against the BhIP $\mathcal{E}[\pi]$ with the same move budget. $\mathcal{A}$ simulates the compiled protocol for $\mathcal{A}'$: whenever $\mathcal{A}'$ sends a commitment C_i and the state-restoration game provides a challenge c_i, $\mathcal{A}$ forwards c_i to $\mathcal{A}'$. When $\mathcal{A}'$ rewinds to a prior round and sends a new commitment,

$\mathcal{A}$ performs the corresponding rewind in the BhIP state-restoration game and forwards the fresh challenge. Each move by $\mathcal{A}'$ corresponds to exactly one move by $\mathcal{A}$, so the move budgets are identical.

When $\mathcal{A}'$ outputs a complete accepting transcript, $\mathcal{A}$ extracts the per-round aligned ciphertexts from the final message, reconstructs the raw ciphertexts for each round (via the deterministic reconstruction procedure in Fig. 15), and outputs this as its BhIP transcript.

Since H is modeled as a random oracle, $\mathcal{A}$ can observe $\mathcal{A}'$'s queries to H. The simulation fails only if $\mathcal{A}'$ outputs a commitment C_i in some round without having queried H on the corresponding preimage (probability $\mathrm{negl}(\lambda)$ per round), or if $\mathcal{A}'$ finds a hash collision (probability at most $t^2/|\mathrm{Image}(H)| \leq \mathrm{negl}(\lambda)$). Conditioned on neither event occurring, the aligned ciphertexts are uniquely determined by the commitments, and each commitment C_i was fixed *before* $\mathcal{A}'$ received challenge c_i. Therefore, the reconstructed BhIP messages were determined before the corresponding challenges, which is the validity condition in the state-restoration game. If the trivial verifier accepts, then the packing relation is satisfied and the BhIP verifier $\mathcal{E}[\pi].\mathsf{V}$ accepts the reconstructed transcript.

We conclude:

$$\varepsilon^{\mathrm{sr}}_{\pi'_{\mathsf{triv}}}(t,n) \leq \varepsilon^{\mathrm{sr}}_{\mathcal{E}[\pi]}(t,n) + \mathrm{negl}(\lambda).$$

Step 2: replacing explicit check with SNARK. In the actual compiled protocol π', the prover sends a SNARK proof π_{outer} instead of the aligned ciphertexts. The SNARK attests that there exist aligned ciphertexts consistent with the commitments $\{C_i\}_{i=1}^r$ that pack to the given packed ciphertexts via Eq. (26). The verifier reconstructs the per-round ciphertext messages from the packed ciphertexts and invokes the BhIP verifier's decision algorithm $\mathcal{E}[\pi].\mathsf{V}$.

Let $\mathcal{A}'$ be a state-restoration adversary against π'. Given $\mathcal{A}'$'s accepting output (commitments, challenges, packed ciphertexts, and π_{outer}), we invoke the SNARK knowledge extractor $\mathcal{K}$ to recover the aligned ciphertexts. If extraction succeeds, the extracted witness satisfies the same consistency conditions checked explicitly by $\mathsf{V}'_{\mathsf{triv}}$, so we can apply the reduction from Step 1. Extraction fails with probability at most $\kappa_{\mathsf{SNARK}}(\lambda)$. Therefore:

$$\varepsilon^{\mathrm{sr}}_{\pi'}(t,n) \leq \varepsilon^{\mathrm{sr}}_{\mathcal{E}[\pi]}(t,n) + \kappa_{\mathsf{SNARK}}(\lambda) + \mathrm{negl}(\lambda).$$

End-to-end composition. By Theorem 2.3, the BhIP $\mathcal{E}[\pi]$ inherits the state-restoration soundness error of the underlying hIP π. Composing with the Fiat-Shamir compilation bound from Section 2.4, applying Fiat-Shamir to π' yields a non-interactive argument with soundness error at most $\varepsilon^{\mathrm{sr}}_{\pi}(t,n) + \kappa_{\mathsf{SNARK}}(\lambda) + t^2/2^\lambda + \mathrm{negl}(\lambda)$, which is negligible for polynomial t.

D Gruen Section 3 Optimization

We first need to describe the optimization from Section 3 [49] whose most important consequence is to reduce the degree of the univariate round polynomials the prover must send.

Setting Suppose the multivariate polynomial $f(X_0, \ldots, X_{n-1})$ in our sumcheck instance takes the form $f(X_0, \ldots, X_{n-1}) = g(X_0, \ldots, X_{n-1}) \cdot \widetilde{\mathsf{eq}}(X_0, \ldots, X_{n-1}, \gamma_0, \ldots, \gamma_{n-1})$ where g has maximum individual degree $d-1$.

Regular sumcheck protocol In round $i \in \{0, \ldots, n-1\}$ of the regular sumcheck protocol, the current claim the prover needs to show is of the form:

$$s_i = \sum_{\vec{x} \in \{0,1\}^{n-i}} f(\vec{r}, \vec{x}),$$

where $\vec{r} = (r_0, \ldots, r_{i-1})$ are the i verifier challenges received thus far. To do so, the honest prover sends

$$\begin{aligned} R_i(X) &= \sum_{\vec{x} \in \{0,1\}^{n-i-1}} f(\vec{r}, X, \vec{x}) \\ &= \sum_{\vec{x} \in \{0,1\}^{n-i-1}} \tilde{\mathsf{eq}}(\vec{r}, X, \vec{x}, \vec{\gamma}) \cdot g(\vec{r}, X, \vec{x}) \\ &= \tilde{\mathsf{eq}}(\vec{r}, \gamma_0, \ldots, \gamma_{i-1}) \cdot \tilde{\mathsf{eq}}(X, \gamma_i) \cdot \sum_{\vec{x} \in \{0,1\}^{n-i-1}} \tilde{\mathsf{eq}}(\vec{x}, \gamma_{i+1}, \ldots, \gamma_{n-1}) \cdot g(\vec{r}, X, \vec{x}), \end{aligned}$$

Finally, the verifier checks whether $R_i(0) + R_i(1) \stackrel{?}{=} s_i$. If this check passes, the verifier samples a fresh challenge r_i, evaluates $s_{i+1} := R_i(r_i)$, and believes the original claim is true if the prover can convince her of the new, simpler, reduced claim:

$$s_{i+1} = \sum_{\vec{x} \in \{0,1\}^{n-i-1}} f(\vec{r}, r_i, \vec{x}),$$

Gruen Section 3 refinement Gruen presents a refinement of the sumcheck protocol for these structured instances. Instead, in round i, the claim the prover needs to show is of the form:

$$s'_i = \sum_{\vec{x} \in \{0,1\}^{n-i}} \tilde{\mathsf{eq}}(\vec{x}, \gamma_i, \ldots, \gamma_{n-1}) \cdot g(\vec{r}, \vec{x})$$

To do so, the *refined* round i honest prover sends:

$$R'_i(X) = \sum_{\vec{x} \in \{0,1\}^{n-i-1}} \tilde{\mathsf{eq}}(\vec{x}, \gamma_{i+1}, \ldots, \gamma_{n-1}) \cdot g(\vec{r}, X, \vec{x})$$

With this modification, the verifier now checks

$$R'_{i-1}(r_{i-1}) \stackrel{?}{=} (1 - \gamma_i) R_i(0) + \gamma_i R_i(1).$$

She then samples a new challenge r_i, evaluates $s'_{i+1} := R'_i(r_i)$, and interprets the newly reduced claim as:

$$s'_{i+1} \stackrel{?}{=} \sum_{\vec{x} \in \{0,1\}^{n-i-1}} \tilde{\mathsf{eq}}(\vec{x}, \gamma_{i+1}, \ldots, \gamma_{n-1}) \cdot g(\vec{r}, r_i, \vec{x})$$

Reducing round polynomial degree. In each round i, the regular sumcheck prover sends degree d univariate polynomial $R_i(X)$, whereas the refined sumcheck prover sends degree $d-1$ univariate polynomial $R'(X)$. Combined with the simple optimization observed in Remark 2.10, this means that the refined sumcheck prover need only compute the evaluation of $R'(X)$ at $d-1$ evaluation points. This will be useful in the aggregation phase of our bespoke layer reduction subroutine.

E BhIP Operation Counts

E.1 Base BhIP Prover FHE Operations for Alternating Layered Payload Circuits

In this subsection, we derive the operation counts presented in Table 12.

E.1.1 Overview and Notation

Quick Recap. The (blind) base protocol for a d-layer alternating payload circuit (with $d = 2d_\times + 1$) consists of d layer reduction invocations. The reductions alternate between two types:

- **Affine layer reductions** (even $\to$ odd): For $i \in \{0, \ldots, d_\times\}$, the prover reduces a claim about L_{2i} to L_{2i+1}. Each such reduction is an s_{2i+1}-round sumcheck.
- **Multiplication layer reductions** (odd $\to$ even): For $i \in \{1, \ldots, d_\times\}$, the prover reduces a claim about L_{2i-1} to L_{2i}. Each such reduction is a $2s_{2i}$-round sumcheck, followed by a final round sending two claimed MLE evaluations.

Remark E.1 (Early Termination)**.** The base BhIP prover processes only the ciphertext-index variables in each layer reduction sumcheck, terminating $\log N$ rounds early and leaving the slot-index variables to the verifier. Concretely, each multiplication layer reduction runs $2s_{2i}$ rounds (not $2s_{2i} + \log N$), and each affine layer reduction runs s_{2i+1} rounds (not $t_{2i+1} = s_{2i+1} + \log N$). This is possible because the verifier can compute the $\log N$ remaining sumcheck rounds locally after decryption. As a consequence, the multiplication layer reduction costs are identical regardless of N, since the $\tilde{\mathsf{mul}}$ coefficients do not vary across SIMD slots.

Notation for homomorphic operations. We define our cost model based on BGV/BFV leveling, where Level 0 is the final level with insufficient noise budget to support another multiplication.

- **Ciphertext Types:**
  - ct: A ciphertext encrypting N base field elements $m \in \mathbb{F}_q^N$.
  - ect: A tuple of R ciphertexts, $(\mathsf{ct}_1, \ldots, \mathsf{ct}_R)$, encrypting N big field elements $m \in (\mathbb{F}_{q^R})^N$.
- **Operations and Costs:**
  - **Addition** (Cost: 0 levels):
    - $\mathsf{ct} + \mathsf{ct} \to \mathsf{ct}$. Output Lvl $= \min(\text{Lvl}_A, \text{Lvl}_B)$.
    - $\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$. Output Lvl $= \min(\text{Lvl}_A, \text{Lvl}_B)$.
  - **CT-Scalar Multiplication / Promotion** (Cost: 0.5 levels):
    - $\mathsf{ct} \cdot \mathbb{F}_q \to \mathsf{ct}$. Output Lvl $=$ InputLvl $- 0.5$.
    - $\mathsf{ect} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$. Output Lvl $=$ InputLvl $- 0.5$.
    - *(Promotion)* $\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$. Output Lvl $=$ InputLvl $- 0.5$.

    These are used when the scalar/plaintext is identical across all SIMD slots.
  - **PT-CT Multiplication** (Cost: 1.0 level):
    - $\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$. Output Lvl $=$ InputLvl $- 1.0$.

    This is used when the plaintext varies across SIMD slots (e.g., encoding slot-dependent coefficients).

– **CT-CT Multiplication** (Cost: 1.0 level):
  * $\mathsf{ct} \cdot \mathsf{ct} \to \mathsf{ct}$. Output Lvl $= \min(\mathrm{Lvl}_A, \mathrm{Lvl}_B) - 1.0$.
  * $\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$. Output Lvl $= \min(\mathrm{Lvl}_A, \mathrm{Lvl}_B) - 1.0$.
  * $\mathsf{ct} \cdot \mathsf{ect} \to \mathsf{ect}$. Output Lvl $= \min(\mathrm{Lvl}_A, \mathrm{Lvl}_B) - 1.0$.

Before we dive in, we state a lemma that will be used extensively in the multiplication layer analysis.

Lemma E.2. *Let $\tilde{L}[X_0, \ldots, X_{s-1}]$ be an s-variate MLE over $\mathbb{F}_q$ with 2^s encrypted Lagrange coefficients $\mathsf{ct}[L(\vec{b})]$ at **Level l + 0.5**. Let $j < s$ and $\vec{\gamma} = (\gamma_0, \ldots, \gamma_j) \in \mathbb{F}_{q^R}$ be some partial evaluation point. Let $\vec{x} \in \{0,1\}^{s-j-1}$ be some Boolean hypercube point. Let $c \in \mathbb{F}_{q^R}$ be some constant.*

*We can compute encrypted evaluations $\mathsf{ect}[\tilde{L}(\vec{\gamma}, X, \vec{x})]$ for all $X \in \{0, 2\}$ using $3 \cdot 2^j$ Promotions ($\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$) ops (Lvl l + 0.5 $\to$ **Lvl l**) and $3 \cdot 2^j$ CT-CT Add ($\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$) ops (at Lvl l). The resulting ciphertext ect is at **Level l**.*

Proof. For each $X \in \{0, 2\}$, the following equation holds.

$$\begin{aligned} & c \cdot \mathsf{ect}[L(\vec{\gamma}, X, \vec{x})] \\ &= \sum_{b \in \{0,1\}^j} \left((1 - X)c \cdot \tilde{\mathsf{eq}}(\vec{b}, \vec{\gamma})\right) \cdot \mathsf{ct}[L(\vec{b}, 0, \vec{x})] \\ &+ \sum_{b \in \{0,1\}^j} \left(Xc \cdot \tilde{\mathsf{eq}}(\vec{b}, \vec{\gamma})\right) \cdot \mathsf{ct}[L(\vec{b}, 1, \vec{x})] \end{aligned}$$

Each of these final sums have 2^j terms. When $X = 0$, the second sum is 0, so we can omit it. □

E.1.2 Affine Layer Reduction Costs (Even → Odd)

Fix an even-to-odd layer reduction instance, reducing a claim about $\tilde{L}_{2i}$ to $\tilde{L}_{2i+1}$. The base prover runs an s_{2i+1}-round sumcheck over the ciphertext-index variables (see Remark E.1). For $X \in \{0, 2\}$, the base verifier needs to learn:

$$R_j(X) = \sum_{\vec{x} \in \{0,1\}^{s_{2i+1}-j-1}} \sum_{\vec{l} \in \{0,1\}^{\log N}} \tilde{\mathsf{lin}}_{2i}((\vec{\gamma}_c, \vec{\gamma}_s), (\vec{r}_c, X, \vec{x}, \vec{l})) \cdot \tilde{L}_{2i+1}(\vec{r}_c, X, \vec{x}, \vec{l})$$

where $\vec{r}_c \in \mathbb{F}_{q^R}^j$ are the received sumcheck challenges for the ciphertext-index variables.

Claim E.3. *For this affine layer reduction of the base protocol, the prover uses ciphertexts that encrypt evaluations of layer L_{2i+1}. These ciphertexts need to have sufficient noise budget to support 1 multiplicative level.*

Proof. We justify Claim E.3 while analyzing the operation counts and multiplicative depth for the sumcheck rounds below. □

This is a *linear* sumcheck: there are no CT-CT multiplications. From the view of the blind prover:

$$\mathsf{emask}[R_j(X)] = \mathsf{FOLD}\left(\sum_{y \in \{0,1\}^{s_{2i+1}}} \mathsf{ept}[c_y] \cdot \mathsf{ct}[L_{2i+1}(y)]\right)$$

where, for each ciphertext index $y \in \{0,1\}^{s_{2i+1}}$, the plaintext $\mathsf{ept}[c_y]$ encodes the slot-dependent linear coefficients summed over the remaining ciphertext-index variables. Crucially, the plaintext $\mathsf{ept}[c_y]$ *varies across SIMD slots* (since the linear function lin has slot-dependent structure for general N), requiring a plaintext-ciphertext multiplication rather than a scalar promotion.

In each of the s_{2i+1} rounds, the base blind prover computes $2S_{2i+1}$ multiplications of the form $\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$ at level 1, then $2S_{2i+1}$ additions of the form $\mathsf{ect} + \mathsf{ect}$ at level 0. The resulting ciphertexts (one for $X = 0$ and one for $X = 2$) are given to the base verifier, who decrypts and performs FOLD on the plaintexts.

Total cost of affine layer sumcheck. Summing over all s_{2i+1} rounds:

- **CT-CT Multiplications: 0**.
- **PT-CT Multiplications** ($\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$) **@ Lvl 1.0:** $\sum_{j=0}^{s_{2i+1}-1} 2S_{2i+1} = 2s_{2i+1}S_{2i+1}$.
- **Additions** ($\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$) **@ Lvl 0.0:** $\sum_{j=0}^{s_{2i+1}-1} 2S_{2i+1} = 2s_{2i+1}S_{2i+1}$.

The multiplicative depth overhead for a affine layer reduction is **1.0 level** (from the plaintext-ciphertext multiplication).

Final round for affine layer. After the sumcheck, the prover sends one claimed MLE evaluation $\mathsf{ect}[\tilde{L}_{2i+1}(\vec{r}_x)]$. Since $\vec{r}_x \in \mathbb{F}_{q^R}^{s_{2i+1}}$ consists only of ciphertext-index challenges (a scalar that does not vary across SIMD slots), this costs S_{2i+1} promotions (at level 0.5), a negligible additive cost.

E.1.3 Multiplication Layer Reduction Costs (Odd → Even)

Fix an odd-to-even layer reduction instance, reducing a claim about $\tilde{L}_{2i-1}$ to $\tilde{L}_{2i}$. The base prover runs a $2s_{2i}$-round sumcheck over the ciphertext-index variables:

$$\mathsf{ct}[v_{2i-1}] \stackrel{?}{=} \mathsf{ct}[\tilde{L}_{2i-1}(\vec{\gamma})] = \sum_{\vec{x},\vec{y} \in \{0,1\}^{s_{2i}}} \tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}, \vec{x}, \vec{y}) \cdot \mathsf{ct}[L_{2i}(\vec{x})] \cdot \mathsf{ct}[L_{2i}(\vec{y})]$$

Claim E.4. *For this multiplication layer reduction of the base protocol, the prover uses ciphertexts that encrypt evaluations of layer L_{2i}. These ciphertexts need to have sufficient noise budget to support 1.5 multiplicative levels.*

Proof. We justify Claim E.4 while analyzing the operation counts and multiplicative depth for the sumcheck rounds below. □

As noted in Remark E.1, the multiplication layer reduction costs are identical for all values of N. The scalar coefficients $\tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}, \vec{x}, \vec{y})$ are uniform across SIMD slots, so all coefficient multiplications in this section are scalar promotions.

The final round requires computing $2N$ MLE evaluations: $\mathsf{ect}[\tilde{L}_{2i}(\vec{r}_x)], \mathsf{ect}[\tilde{L}_{2i}(\vec{r}_y)]$. Since $\vec{r}_x, \vec{r}_y \in \mathbb{F}_{q^R}^{s_{2i}}$, each result encapsulates N extension field elements, so their encryption can be represented with 2 ects. Concretely, $\mathsf{ect}[\tilde{L}_{2i}(\vec{r}_x)]$ is computed as the inner product $\sum_{\vec{b} \in \{0,1\}^{s_{2i}}} \mathsf{ct}[L_{2i}(\vec{b})] \cdot \tilde{\mathsf{eq}}(\vec{b}, \vec{r}_x)$. This requires S_{2i} operations of $\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$, which costs 0.5 levels. Thus, the final round is not the depth (nor operation count) bottleneck.

Indicator table sparsity.

Remark E.5. For each $X \in \{0, 2\}$ and sumcheck round j, the indicator table $\hat{M}_j[X]$ has at most $\min\{S_{2i-1}, 2^{2s_{2i}-j-1}\}$ nonzero evaluations.

Now, fix a sumcheck round $j \in \{0, \ldots, 2s_{2i} - 1\}$ with challenges $\vec{\gamma} \in \mathbb{F}_{q^R}^{j}$.

Case 1: $j < s_{2i}$ (First Half of Sumcheck)

The round polynomial simplifies (compared to the prior version with addition gates) because only multiplication gates remain:

$$R_j(X) = \sum_{\vec{x}} \sum_{\vec{y}} \tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}, \vec{r}_x, X, \vec{x}, \vec{y}) \cdot \mathsf{ect}[\tilde{L}_{2i}(\vec{r}_x, X, \vec{x})] \cdot \mathsf{ct}[L_{2i}(\vec{y})]$$

We write $R_j(X) = \sum_{\vec{x}} d_x \cdot c_x$, where:

- $d_x = \mathsf{ect}[\tilde{L}_{2i}(\vec{r}_x, X, \vec{x})]$ is a ect at **Lvl 1.0**.
- $c_x = \sum_{\vec{y}} \tilde{\mathsf{mul}}_{2i-1}(\ldots) \cdot \mathsf{ct}[L_{2i}(\vec{y})]$ is a ect at **Lvl 1.0** (a sparse inner product of $\mathsf{ct} \cdot \mathbb{F}_{q^R}$).

1. **Compute $S_{2i}/(2^{j+1})$ terms c_x:**
   - $2S_{2i-1}$ ops of $\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$ (Lvl 1.5 $\to$ Lvl 1.0) by Remark E.5.
   - $2S_{2i-1}$ ops of $\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$ (at Lvl 1.0).
2. **Compute $S_{2i}/(2^{j+1})$ terms d_x:**
   - $3 \cdot 2^j$ ops of $\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$ per term (Lvl 1.5 $\to$ Lvl 1.0) by Lemma E.2.
   - $3 \cdot 2^j$ ops of $\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$ per term (at Lvl 1.0).
3. **Compute $S_{2i}/(2^{j+1})$ products ($T_x = c_x \cdot d_x$):**
   - $2S_{2i}/(2^{j+1})$ ops of $\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$ (Lvl 1.0, Lvl 1.0 $\to$ **Lvl 0.0**).
4. **Sum $S_{2i}/(2^{j+1})$ terms (T_x):**
   - $2S_{2i}/(2^{j+1})$ ops of $\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$ (at Lvl 0.0).

Case 2: $j \geq s_{2i}$ (second half of sumcheck)

For notational convenience, let $j' = j - s_{2i}$, which means $j' \in \{0, \ldots, s_{2i} - 1\}$. The j challenges received so far are $\vec{r} = (\vec{r}_x, \vec{r}_y)$, where $\vec{r}_x \in \mathbb{F}_{q^R}^{s_{2i}}$ are the challenges from the first s_{2i} rounds (Case 1) and $\vec{r}_y \in \mathbb{F}_{q^R}^{j'}$ are the j' challenges from this second half.

We compute once and for all the value $\mathsf{ect}[a] := \mathsf{ect}[\tilde{L}_{2i}(\vec{r}_x)]$. This is a single MLE evaluation based on the challenges from Case 1. This costs S_{2i} promotions at level 1.5 $\to$ 1.0, and we note that this **checkpoint** consumes 0.5 levels from the original wire ciphertexts. We store two copies of the resulting ciphertext: one at **Level 1.0** and one at **Level 0.5** for use in different computations.

The round polynomial we need to compute is:

$$R_j(X) = \mathsf{ect}[a] \cdot \sum_{\vec{y}} \tilde{\mathsf{mul}}_{2i-1}(\vec{\gamma}, \vec{r}_x, \vec{r}_y, X, \vec{y}) \cdot \mathsf{ect}[\tilde{L}_{2i}(\vec{r}_y, X, \vec{y})]$$

Let $S'(X) = \sum_{\vec{y}} \tilde{\mathsf{mul}}_{2i-1}(\ldots) \cdot \mathsf{ect}[\tilde{L}_{2i}(\vec{r}_y, X, \vec{y})]$. This calculation proceeds in two steps:

1. **Compute $S'(X)$:** We apply Lemma E.2 (with $j = j'$) for each of the $2^{s_{2i}-j'-1}$ terms. The input cts are at Level 1.5. Cost: $3S_{2i}/2$ promotions (Lvl 1.5 $\to$ 1.0) and $3S_{2i}/2 + S_{2i}/(2^{j'+1})$ additions (at Lvl 1.0).
2. **Compute Final Product:** We multiply $S'(X)$ by $\mathsf{ect}[a]$ (using the copy at **Level 1.0**). For each $X \in \{0, 2\}$, this is one $\mathsf{ect} \cdot \mathsf{ect}$ multiplication, a negligible cost compared to step 1.

TotalCcost of All Sumcheck Rounds for a Multiplication Layer Reduction

We now sum the costs over all $2s_{2i}$ rounds. Let $S := S_{2i}$ and $s := s_{2i}$ for brevity.

- **CT-CT Multiplications ($\mathsf{ect} \cdot \mathsf{ect} \to \mathsf{ect}$) @ Lvl 1.0 $\to$ 0.0:**
  - Case 1: $\sum_{j=0}^{s-1} \frac{S}{2^j} \leq 2S$.

  Total: **2S**.
- **Promotions ($\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$) @ Lvl 1.5 $\to$ 1.0:**
  - Case 1: $\sum_{j=0}^{s-1}(2S_{2i-1} + 3S/2) = 2s \cdot S_{2i-1} + \frac{3}{2}sS$.
  - Case 2: $\sum_{j'=0}^{s-1}(3S/2) = \frac{3}{2}sS$.

  Total: $\mathbf{2s \cdot S_{2i-1} + 3sS}$.
- **Additions ($\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$) @ Lvl 1.0:**
  - Case 1: $2s \cdot S_{2i-1} + \frac{3}{2}sS$.
  - Case 2: $\frac{3}{2}sS + S$.

  Total: $\mathbf{2s \cdot S_{2i-1} + 3sS}$.
- **Additions ($\mathsf{ect} + \mathsf{ect} \to \mathsf{ect}$) @ Lvl 0.0:** Case 1 only: $\leq 2S$. Total: **2S**.

E.1.4 Total Cost of Base BhIP Prover under FHE

The full $\mathsf{BlindGKR}.P$ algorithm involves $d_\times + 1$ affine layer reductions and $d_\times$ multiplication layer reductions. We now analyze the total cost.

We recall the following parameter notation (see Section 5.1.2):

- n: The number of FHE operations to compute the payload circuit.
- $m := \sum_{i=1}^{d_\times} S_{2i-1}$: The total number of multiplication gates.
- $p := \sum_{i=0}^{d_\times} S_{2i}$: The total number of affine gates.
- $s_{\max} = \max_i\{s_i\}$: The maximum log-width of any layer.

Recall the bounds $m + p \leq n$ and $p \leq 2m$ from Section 5.2.

Remark E.6 (CT-CT vs. PT-CT Multiplication Gates)**.** Of the m total multiplication gates in the circuit, some may be PT-CT multiplications (where one operand is a known plaintext) and some may be CT-CT multiplications (where both operands are encrypted). PT-CT multiplication gates require strictly fewer FHE operations for the prover than CT-CT multiplication gates, since the PT-CT contribution to the sumcheck involves only a linear (rather than quadratic) term in the layer evaluations. For simplicity, we upper bound the FHE operation counts by making the worst-case assumption that all m multiplication gates are CT-CT. This assumption is used solely for bounding FHE operation counts of the prover, and does not limit the protocol's ability to handle PT-CT multiplications.

Total Operation Count. We sum over all layer reductions.

- **Total CT-CT Multiplications (Lvl 1.0 $\to$ 0.0):** Only multiplication layers contribute. $\sum_{i=1}^{d_\times} 2S_{2i} \le 2(p - S_0) \le 2p$.
- **Total Promotions (Lvl 1.5 $\to$ 1.0):** Only multiplication layers contribute (see Remark E.1). $\sum_{i=1}^{d_\times}(2s_{2i}S_{2i-1} + 3s_{2i}S_{2i}) \le s_{\max}(2m + 3(p - S_0)) \le (2m + 3p) \cdot s_{\max}$.
- **Total PT-CT Multiplications ($\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$) @ Lvl 1.0:** Only affine layers contribute. $\sum_{i=0}^{d_\times} 2s_{2i+1}S_{2i+1} \le 2(m + S_d) \cdot s_{\max}$. (The odd layers contribute m total multiplication gates and the input layer contributes S_d.)
- **Total Additions (Lvl 1.0):** Only multiplication layers: $\le (2m + 3p) \cdot s_{\max} \le 3n \cdot s_{\max}$.
- **Total Additions (Lvl 0.0):** Both layer types contribute.
  - Multiplication layers: $\sum_{i=1}^{d_\times} 2S_{2i} \le 2p$ (negligible, so we can omit it).
  - Affine layers: $\sum_{i=0}^{d_\times} 2s_{2i+1}S_{2i+1} \le 2(m + S_d) \cdot s_{\max}$.

  Total: $\le 2(m + S_d) \cdot s_{\max}$.

Lemma E.7. *Let $d_\times$ be the multiplicative depth of the payload circuit. The multiplicative depth required to compute the payload circuit and generate a proof (using the base protocol) is at most $d_\times + 1$, implying an additive multiplicative depth overhead of* **1** *level.*

Proof. By Claim E.3, each affine layer reduction requires 1 level, and the inputs to these reductions have consumed at most $d_\times$ levels from payload evaluation. By Claim E.4, each multiplication layer reduction requires 1.5 levels, and the inputs to these reductions have consumed at most $d_\times - 1$ levels from payload evaluation. Therefore, multiplicative depth of $d_\times + 1$ suffices to compute the payload and generate the proof. □

Total FHE operation counts (omitting negligible lower order additive terms) to generate a base protocol proof are summarized in Table 12.

E.2 Folded BhIP Prover FHE Operations for Alternating Layered Payload Circuits

In this subsection, we derive the operation counts presented in Table 13. The folded BhIP prover builds on the base BhIP prover (see Appendix E.1) by homomorphically folding the per-slot round polynomials into a single value before sending them to the verifier. This reduces communication but increases the prover's FHE work.

Concretely, whereas the base prover terminates each layer reduction sumcheck $\log N$ rounds early (see Remark E.1), the folded prover runs all $\log N$ additional rounds. In each round, the folded prover computes the base round polynomial, multiplies by a plaintext encoding the eq-slot coefficients $\mathsf{ept}[c] \equiv (\tilde{\mathsf{eq}}(\vec{\psi}, 0^{\log N}), \ldots, \tilde{\mathsf{eq}}(\vec{\psi}, 1^{\log N}))$, and then applies a FOLD operator. We analyze the two layer types separately.

E.2.1 Multiplication Layer Reduction Operation Counts

Fix a multiplication layer reduction from L_{2i-1} to L_{2i}, for $i \in \{1, \ldots, d_\times\}$. The folded layer reduction consists of $2s_{2i} + \log N$ rounds of sumcheck, where we first process the x, y (ciphertext-index) variables, then the ℓ (slot-index) variables.

Claim E.8. *For this multiplication layer reduction of the folded protocol, the prover uses ciphertexts that encrypt evaluations of layer L_{2i}. These ciphertexts need to have sufficient noise budget to support 2.5 multiplicative levels.*

Proof. We justify Claim E.8 while analyzing the operation counts and multiplicative depth for the sumcheck rounds below. □

Case 1: $0 \leq j < 2s_{2i}$. The blind prover needs to compute round polynomial $R_j(X)$ for $X \in \{0, 2\}$. The following observation is crucial.

$$\mathsf{emask}[R_j(X)] \equiv \mathsf{FOLD}\left(\mathsf{ept}[c] \cdot \mathsf{ect}[R_j(X)]\right),$$

where

- $\mathsf{ect}[R_j(X)]$ are the N round-j polynomials (one per slot) that would be computed in the *base* BhIP's corresponding layer reduction sumcheck (see Appendix E.1.3),
- and $\mathsf{ept}[c] \equiv \left(\tilde{\mathsf{eq}}(\vec{\gamma}_s, 0^{\log N}), \cdots, \tilde{\mathsf{eq}}(\vec{\gamma}_s, 1^{\log N})\right)$.

Thus, $R_j(X)$ can be computed by computing the base round polynomial $R_j(X)$ (at most 1.5 levels; see Appendix E.1.3), followed by a plaintext-ciphertext multiplication (introducing 1 additional level of multiplicative depth), followed by a FOLD operation (which introduces no additional multiplicative depth, only rotation noise).

Case 2: $2s_{2i} \leq j < 2s_{2i} + \log N$. Let $\vec{r} \in \mathbb{F}_{q^R}^{2s_{2i}}$ denote the first $2s_{2i}$ sumcheck challenges, and let $\vec{\lambda} \in \mathbb{F}_{q^R}^{j-2s_{2i}}$ denote the remaining received sumcheck challenges thus far. We can exploit an optimization due to Gruen (see Appendix D). The consequence is that we need only compute a degree 2 round polynomial at $X \in \{0, 2\}$.

Before entering this phase, the prover checkpoints the MLE evaluations $\mathsf{ect}[\tilde{L}_{2i}(\vec{r}_x)]$ and $\mathsf{ect}[\tilde{L}_{2i}(\vec{r}_y)]$. This checkpoint step costs 0.5 levels (promotions from the wire ciphertexts).

Let's use the following shorthand:

- $\mu := \tilde{\mathsf{mul}}(\vec{\gamma}_c, \vec{r})$.
- $c_l := \tilde{\mathsf{eq}}(\vec{\ell}, \gamma_s(j+1), \ldots, \gamma_s(\log N - 1))$
- $w := L_{2i}(r_0, \ldots, r_{s_{2i}-1}, \cdots)$
- $z := L_{2i}(r_{s_{2i}}, \ldots, r_{2s_{2i}-1}, \cdots)$

The blind prover needs to compute

$$\mathsf{emask}[R_j(X)] = \sum_{\vec{\ell} \in \{0,1\}^{\log N - j - 1}} \mu \cdot c_l \cdot \mathsf{emask}[\tilde{w}(\vec{\lambda}, X, \vec{\ell})] \cdot \mathsf{emask}[\tilde{z}(\vec{\lambda}, X, \vec{\ell})]$$

For all $x \in \{0,1\}^{\log N}$, let $d_x := \tilde{\mathsf{eq}}(\vec{x}, (\vec{\lambda}, X, \psi_{j+1}, \ldots, \psi_{\log N - 1}))$. Let plaintext $\mathsf{ept}[\mu]$ be equivalent to the following list of $\mathbb{F}_{q^R}$ elements: $(\mu \cdot d_{0^{\log N}}, \cdots, \mu \cdot d_{1^{\log N}})$.

Observe the following:

$$\mathsf{emask}[R_j(X)] = \mathsf{FOLD}\left(\mathsf{ept}[\mu] \cdot \mathsf{ect}[w] \cdot \mathsf{ect}[z]\right)$$

Thus computing $R_j(X)$ at $X = 0, 2$ requires:

- Two $\mathsf{ect} \cdot \mathsf{ect}$ multiplications (level $2.0 \to 1.0$)
- Followed by two $\mathsf{ept} \cdot \mathsf{ect}$ multiplications (plaintext-ciphertext, level $1.0 \to 0.0$)
- Followed by two FOLD operators (no multiplicative depth).

E.2.2 Affine Layer Reduction Operation Counts

Fix a affine layer reduction from L_{2i} to L_{2i+1}, for $i \in \{0, \ldots, d_\times\}$. The folded layer reduction consists of $s_{2i+1} + \log N$ rounds of sumcheck.

Claim E.9. *For this affine layer reduction of the folded protocol, the prover uses ciphertexts that encrypt evaluations of layer L_{2i+1}. These ciphertexts need to have sufficient noise budget to support 1.5 multiplicative levels.*

Proof. We justify Claim E.9 while analyzing the operation counts and multiplicative depth for the sumcheck rounds below. □

Case 1: $0 \le j < s_{2i+1}$ For these sumcheck rounds, the folded blind prover computes the base round polynomials (which involve PT-CT multiplications with slot-varying linear coefficients; see Appendix E.1.2). The result is then folded. Overall, this tallies to:

- $2S_{2i+1}$ $\mathsf{ept} \cdot \mathsf{ct}$ multiplications at level 1.
- $2S_{2i+1}$ $\mathsf{ect} + \mathsf{ect}$ additions at level 0.
- $2R$ FOLD operations (no multiplicative depth).

Case 2: $s_{2i+1} \le j < s_{2i+1} + \log N$ Before entering this phase, the prover checkpoints $\mathsf{ect}[\tilde{L}_{2i+1}(\vec{r}_x)]$, costing 0.5 levels from the wire ciphertexts. This contributes to the overall multiplicative depth, but the time to compute these checkpoints is negligible compared to the rest of sumcheck proving computation, so we omit it from the operation counts.

For each of these additional[44] sumcheck rounds, the folded prover computes:

$$\mathsf{emask}[R_j(X)] = \mathsf{FOLD}\left(\mathsf{ept}[c] \cdot \mathsf{ect}[\tilde{L}_{2i+1}(\vec{r}_x, \vec{\lambda}, X, \vec{\ell})]\right)$$

This requires:

- Two $\mathsf{ept} \cdot \mathsf{ect}$ multiplications (plaintext-ciphertext, costing 1 level)
- Two $\mathsf{ect} + \mathsf{ect}$ additions
- $2R$ FOLD operations (no multiplicative depth)

Totaling Operation Counts in one affine layer sumcheck. Overall, the blind prover does work bounded by:

- $2(S_{2i+1} s_{\max} + \log N)$ $\mathsf{ept} \cdot \mathsf{ct}$ multiplications at level 1.
- $2(S_{2i+1} s_{\max} + \log N)$ $\mathsf{ect} + \mathsf{ect}$ additions at level 0.
- $2R(s_{\max} + \log N)$ FOLD operations (no multiplicative depth).

[44] The base protocol skips these rounds.

E.2.3 Total Operation Counts

We now tally the operation counts across all layer reductions. Using the notation from Appendix E.1.4, the bounds $m + p \leq n$, $p \leq 2m$, and $d = 2d_\times + 1$ layers. As noted in Remark E.6, some of the m multiplication gates may be PT-CT multiplications, which require fewer FHE operations than CT-CT multiplications. We again upper bound operation counts by assuming all multiplication gates are CT-CT.

Multiplication layer totals. The folded multiplication layer reduction has $2s_{2i} + \log N$ rounds per layer, across $d_\times$ layers.

- **Case 1 costs:** The base round polynomial costs (from Appendix E.1.3) are inherited in full. Additionally, per round the folded prover performs 2 PT-CT multiplications (for the eq-slot $\mathsf{ept}[c]$) and $2R$ fold operations.
- **Case 2 costs:** Per layer, $\log N$ additional rounds, each requiring 2 CT-CT multiplications, 2 PT-CT multiplications, and $2R$ fold operations.

Across all $d_\times$ multiplication layers:

- CT-CT (2.0 $\to$ 1.0): $\leq 2p$ (from Case 1, same as base).
- CT-CT (2.0): $2d_\times \log N$ (from Case 2).
- Promotions (2.5 $\to$ 1.5): $(2m + 3p) \cdot s_{\max}$ (Case 1, base computation at raised level).
- Promotions (1.5 $\to$ 1.0): $(2m + 3p) \cdot s_{\max}$ (Case 1, from second use of d_x, c_x at lower level).
- PT-CT ($\mathsf{ect} \cdot \mathsf{ept}$, 1.0): $4d_\times(s_{\max} + \log N)$ (eq-slot multiplication in both cases).
- Additions (2.0): $(2m + 3p) \cdot s_{\max}$.
- Additions (1.0): $(2m + 3p) \cdot s_{\max} + 2d_\times \log N$.

Affine layer totals. Noting that $\sum_{i=0}^{d_\times} S_{2i+1} \leq m + S_d$, across all $d_\times + 1$ affine layers:

- PT-CT ($\mathsf{ept} \cdot \mathsf{ct}$, 1.0): $2\big((m + S_d) \cdot s_{\max} + (d_\times + 1) \log N\big)$.
- Additions (0.0): $2\big((m + S_d) \cdot s_{\max} + (d_\times + 1) \log N\big)$.
- Fold: $2R(d_\times + 1)(s_{\max} + \log N)$.

Combined totals. Combining multiplication and affine layer contributions, and noting $d = 2d_\times + 1$:

- **Fold operations:** $4dR(s_{\max} + \log N)$ total (no multiplicative depth).

Total operation counts (omitting negligible lower order additive terms) to generate a folded protocol proof are presented in Table 13.

Lemma E.10. *Let $d_\times$ be the multiplicative depth of the payload circuit. The multiplicative depth required to compute the payload circuit and generate a proof (using the folded protocol) is at most $d_\times + 1.5$, implying an additive multiplicative depth overhead of* **1.5** *levels.*

Proof. By Claim E.9, each affine layer reduction requires 1.5 levels, and the inputs to these reductions have consumed at most $d_\times$ levels from payload evaluation. By Claim E.8, each multiplication layer reduction requires 2.5 levels, and the inputs to these reductions have consumed at most $d_\times - 1$ levels from payload evaluation. Therefore, multiplicative depth of $d_\times + 1.5$ suffices to compute the payload and generate the proof. □

E.3 Laned BhIP Prover FHE Operations for Dense Alternating Layered Payload Circuits

In this subsection, we derive the operation counts presented in Table 6. The laned BhIP prover operates on packed ciphertexts produced by witness packing (Section 4.3), processing all $s'_i + \log N$ sumcheck rounds homomorphically in each layer reduction.

E.3.1 Overview and Notation

Quick Recap. The BhIP for a B-laned payload circuit with $d = 2d_\times + 1$ layers consists of d layer reduction invocations on the packed circuit. The reductions alternate between two types:

- **Affine layer reductions** (even $\to$ odd): For $i \in \{0, \ldots, d_\times\}$, the prover reduces a claim about L_{2i} to L_{2i+1}. Each such reduction is an $(s'_{2i+1} + \log N)$-round sumcheck, preceded by a precomputation step that lazily packs the odd-indexed layer (Remark 4.7).
- **Multiplication layer reductions** (odd $\to$ even): For $i \in \{1, \ldots, d_\times\}$, the prover reduces a claim about L_{2i-1} to L_{2i}. Each such reduction is an $(s'_{2i-1} + \log N)$-round sumcheck.

Packed circuit parameters. After packing, layer i has $S'_i = \lceil S_i B/N \rceil$ packed ciphertexts, each utilizing all N slots. We set $s'_i := \log_2 S'_i$ and $s'_{\max} := \max_i\{s'_i\}$. Let $m' := \sum_{i=1}^{d_\times} S'_{2i-1}$ and $p' := \sum_{i=0}^{d_\times} S'_{2i}$ denote the packed multiplication and affine gate counts, respectively. By the dense wiring constraint, $p' \le 2m' + 1$.

Notation for homomorphic operations. We use the same cost model and notation as in Appendix E.1. Additionally, we use:

- $\mathsf{ept} \cdot \mathsf{ect}$ **multiplication** (Cost: 1.0 level): A plaintext-by-ciphertext multiplication where the plaintext encodes slot-varying $\mathbb{F}_{q^R}$ elements. Concretely, this is R operations of the form $\mathsf{ept} \cdot \mathsf{ct} \to \mathsf{ect}$.
- FOLD **operation** (Cost: 0 levels): A slot-summation operator that introduces no multiplicative depth, only rotation noise.

Throughout, we use the notation $\mathsf{ect}[\cdot]$ for ciphertexts encoding $\mathbb{F}_{q^R}$ elements (tuples of R base ciphertexts) and $\mathsf{ept}[\cdot]$ for plaintexts encoding slot-varying $\mathbb{F}_{q^R}$ elements.

E.3.2 Witness Packing Costs

Packing of even-indexed layers is performed eagerly before proof generation. Each of the p even-layer ciphertexts requires one PT-CT multiplication (masking) and participates in one CT-CT addition (summing within packing groups). Packing of odd-indexed layers is performed lazily as the first step of each affine layer reduction, combined with $\tilde{\mathsf{eq}}$ weighting (Remark 4.7). Each of the m odd-layer original ciphertexts requires one PT-CT multiplication (combined masking and $\tilde{\mathsf{eq}}$ weighting) and participates in one CT-CT addition.

In total, witness packing contributes:

- $m + p \le 3m$ PT-CT multiplications ($\mathsf{ept} \cdot \mathsf{ct}$, 1 level each).
- $m + p \le 3m$ CT-CT additions.

E.3.3 Multiplication Layer Reduction Operation Counts

Fix a multiplication layer reduction from L_{2i-1} to L_{2i}, for $i \in \{1, \ldots, d_\times\}$. Let $s' := s'_{2i-1}$ for brevity. The sumcheck has $s' + \log N$ rounds.

Claim E.11. *For this multiplication layer reduction, the prover uses ciphertexts that encrypt evaluations of layer L_{2i} (after packing). These ciphertexts need to have sufficient noise budget to support 2.5 multiplicative levels.*

Proof. We justify this while analyzing the operation counts below. □

By the dense wiring structure (Remark 4.5), the assignment identity is:

$$L_{2i-1}(\vec{x}_c, \vec{x}_s) = L_{2i}(0, \vec{x}_c, \vec{x}_s) \cdot L_{2i}(1, \vec{x}_c, \vec{x}_s).$$

The sumcheck polynomial (Eq. (7)) is:

$$\tilde{L}_{2i-1}(\vec{\gamma}_c, \vec{\gamma}_s) = \sum_{\vec{x}_c \in \{0,1\}^{s'}} \sum_{\vec{x}_s \in \{0,1\}^{\log N}} \tilde{\mathsf{eq}}((\vec{x}_c, \vec{x}_s), (\vec{\gamma}_c, \vec{\gamma}_s)) \cdot \tilde{L}_{2i}(0, \vec{x}_c, \vec{x}_s) \cdot \tilde{L}_{2i}(1, \vec{x}_c, \vec{x}_s).$$

We apply the Gruen optimization (see Appendix D), so it suffices to send degree-2 round polynomials evaluated at $X \in \{0, 2\}$ (the verifier recovers $X = 1$ from $X = 0$ and the claimed sum).

Case 1: $0 \le j < s'$ (packed ciphertext index rounds). Let $\vec{r} \in \mathbb{F}_{q^R}^j$ denote the first j sumcheck challenges. The round-j polynomial is:

$$R_j(X) = \sum_{\vec{x}_s \in \{0,1\}^{\log N}} \sum_{\vec{w} \in \{0,1\}^{s'-j-1}} \tilde{\mathsf{eq}}((\vec{r}, X, \vec{w}), \vec{\gamma}_c) \cdot \tilde{\mathsf{eq}}(\vec{x}_s, \vec{\gamma}_s) \cdot \tilde{L}_{2i}(0, (\vec{r}, X, \vec{w}), \vec{x}_s) \cdot \tilde{L}_{2i}(1, (\vec{r}, X, \vec{w}), \vec{x}_s).$$

Let $c_w := \tilde{\mathsf{eq}}(\vec{w}, \vec{\gamma}_c^{>j})$ and let ept_a be the plaintext encoding $\tilde{\mathsf{eq}}(\vec{x}_s, \vec{\gamma}_s)$ for each slot $\vec{x}_s$. Then:

$$\mathsf{ect}[R_j(X)] = \mathsf{FOLD}\left(\mathsf{ept}_a \cdot \sum_{\vec{w}} \mathsf{ect}[c_w \cdot L_{2i}(0, (\vec{r}, X, \vec{w}))] \cdot \mathsf{ect}[L_{2i}(1, (\vec{r}, X, \vec{w}))]\right).$$

The computation proceeds in three steps:

Step 1 (MLE partial evaluations): For each $\vec{w} \in \{0,1\}^{s'-j-1}$, compute $\mathsf{ect}[c_w \cdot L_{2i}(0, (\vec{r}, X, \vec{w}))]$ for $X \in \{0, 2\}$. By Lemma E.2, this costs $3 \cdot 2^j$ promotions ($\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$) per $\vec{w}$. There are $2^{s'-j-1}$ values of $\vec{w}$, giving $3 \cdot 2^{s'-1}$ promotions at level $2.5 \to 2.0$.

Similarly, compute $\mathsf{ect}[L_{2i}(1, (\vec{r}, X, \vec{w}))]$ for all $\vec{w}$ and $X \in \{0, 2\}$: another $3 \cdot 2^{s'-1}$ promotions at level $2.5 \to 2.0$.

Total per round: $3 \cdot 2^{s'} = 3S'_{2i-1}$ promotions.

Step 2 (CT-CT products and sum over $\vec{w}$): For each $\vec{w}$ and each $X \in \{0, 2\}$, multiply $\mathsf{ect}[\text{left}] \cdot \mathsf{ect}[\text{right}]$ (level $2.0 \to 1.0$). There are $2^{s'-j-1}$ values of $\vec{w}$ and 2 values of X, giving $2^{s'-j}$ CT-CT multiplications. Then sum over $\vec{w}$: $2^{s'-j}$ additions at level 1.0.

Step 3 (PT-CT multiplication and fold): For each $X \in \{0, 2\}$: one $\mathsf{ept} \cdot \mathsf{ect}$ multiplication (level $1.0 \to 0.0$), then one FOLD (no depth). This gives 2 $\mathsf{ept} \cdot \mathsf{ect}$ multiplications and 2 FOLD operations per round.

Case 2: $s' \leq j < s' + \log N$ **(slot index rounds).** Before entering this phase, the prover checkpoints $\mathsf{ect}[\tilde{L}_{2i}(0, \vec{r}_c)]$ and $\mathsf{ect}[\tilde{L}_{2i}(1, \vec{r}_c)]$, where $\vec{r}_c \in \mathbb{F}_{q^R}^{s'}$ denotes the first s' sumcheck challenges. This checkpoint costs 0.5 levels from the packed wire ciphertexts (via S'_{2i} promotions), producing two ciphertexts at level 2.0.

Let $j' := j - s'$, so $j' \in \{0, \ldots, \log N - 1\}$. Let $\vec{r}_s \in \mathbb{F}_{q^R}^{j'}$ denote the slot-index challenges received so far. The round polynomial is:

$$R_j(X) = \sum_{\vec{x} \in \{0,1\}^{\log N - j' - 1}} \tilde{\mathsf{eq}}((\vec{r}_s, X, \vec{x}), \vec{\gamma}_s) \cdot \tilde{L}_{2i}(0, \vec{r}_c, (\vec{r}_s, X, \vec{x})) \cdot \tilde{L}_{2i}(1, \vec{r}_c, (\vec{r}_s, X, \vec{x})).$$

Since we have already computed $\mathsf{ect}[\tilde{L}_{2i}(0, \vec{r}_c)]$ and $\mathsf{ect}[\tilde{L}_{2i}(1, \vec{r}_c)]$ (each a single packed ciphertext), the N-slot structure handles the remaining summation. Let ept_μ encode the combined eq-coefficients $\tilde{\mathsf{eq}}((\vec{r}_s, X, \vec{x}), \vec{\gamma}_s)$ across slots. Then:

$$\mathsf{ect}[R_j(X)] = \mathsf{FOLD}\left(\mathsf{ept}_\mu \cdot \mathsf{ect}[\tilde{L}_{2i}(0, \vec{r}_c)] \cdot \mathsf{ect}[\tilde{L}_{2i}(1, \vec{r}_c)]\right).$$

For each $X \in \{0, 2\}$, this requires:

- One $\mathsf{ect} \cdot \mathsf{ect}$ multiplication (level $2.0 \to 1.0$).
- One $\mathsf{ept} \cdot \mathsf{ect}$ multiplication (level $1.0 \to 0.0$).
- One FOLD operation (no depth).

Per round: 2 CT-CT multiplications, 2 $\mathsf{ept} \cdot \mathsf{ect}$ multiplications, 2 FOLD operations.

Totaling operation counts for one multiplication layer reduction. Let $S' := S'_{2i-1}$ and $s' := s'_{2i-1}$. Summing over all $s' + \log N$ rounds:

- **Promotions** ($\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$) **@ Lvl** $2.5 \to 2.0$**:** Case 1 only. $\sum_{j=0}^{s'-1} 3S' = 3s'S'$.
- **CT-CT Multiplications** ($\mathsf{ect} \cdot \mathsf{ect}$) **@ Lvl** $2.0 \to 1.0$**:**
  - Case 1: $\sum_{j=0}^{s'-1} 2^{s'-j} \leq 2S'$.
  - Case 2: $2 \log N$.

  Total: $2S' + 2 \log N$.
- $\mathsf{ept} \cdot \mathsf{ect}$ **Multiplications @ Lvl** $1.0 \to 0.0$**:**
  - Case 1: $2s'$.
  - Case 2: $2 \log N$.

  Total: $2(s' + \log N)$.
- **Additions** ($\mathsf{ect} + \mathsf{ect}$) **@ Lvl** 1.0**:** Case 1 only: $\sum_{j=0}^{s'-1} 2^{s'-j} \leq 2S'$. Total: $2S'$.
- FOLD **Operations (no depth):** $2(s' + \log N)$.

E.3.4 Affine Layer Reduction Operation Counts

Fix an affine layer reduction from L_{2i} to L_{2i+1}, for $i \in \{0, \ldots, d_\times\}$. Let $s' := s'_{2i+1}$ for brevity. The sumcheck has $s' + \log N$ rounds, preceded by a precomputation step.

Claim E.12. *For this affine layer reduction, the prover uses ciphertexts that encrypt evaluations of layer L_{2i+1}. These ciphertexts need to have sufficient noise budget to support 1.5 multiplicative levels.*

Proof. We justify this while analyzing the operation counts below. □

Precomputation (lazy packing and $\tilde{\mathsf{eq}}$ weighting). By Remark 4.7, the prover combines masking and $\tilde{\mathsf{eq}}$ weighting into a single PT-CT multiplication per original ciphertext in layer $2i+1$. This produces S'_{2i+1} packed ciphertexts encoding $\tilde{\mathsf{eq}}(\vec{\gamma}_\ell, \vec{\gamma}_\ell) \cdot L_{2i+1}(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$. Cost: S_{2i+1} PT-CT multiplications (consuming 1 level) and $S_{2i+1} - S'_{2i+1}$ additions. This cost is accounted for in the witness packing totals (Appendix E.3.2).

Sumcheck. After precomputation, the full layer evaluation (Eq. (12)) is:

$$\tilde{L}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q, \vec{\gamma}_\ell) = \tilde{\mathsf{add}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q) + \sum_{\vec{x}_c \in \{0,1\}^{s'}} \sum_{\vec{x}_q \in \{0,1\}^{\log(N/B)}} \sum_{\vec{x}_\ell \in \{0,1\}^{\log B}} g(x_c, x_q, x_\ell)$$

$$g(x_c, x_q, x_\ell) := \tilde{\mathsf{lin}}_{2i}\big((\vec{\gamma}_c, \vec{\gamma}_q), (\vec{x}_c, \vec{x}_q)\big) \cdot \tilde{\mathsf{eq}}(\vec{\gamma}_\ell, \vec{x}_\ell) \cdot \tilde{L}_{2i+1}(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$$

The verifier subtracts $\tilde{\mathsf{add}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q)$ from the initial claim, and the sumcheck is then run on the adjusted target:

$$\tilde{L}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q, \vec{\gamma}_\ell) - \tilde{\mathsf{add}}_{2i}(\vec{\gamma}_c, \vec{\gamma}_q) = \sum_{\vec{x}_c} \sum_{\vec{x}_q} \sum_{\vec{x}_\ell} \tilde{\mathsf{lin}}_{2i}\big((\vec{\gamma}_c, \vec{\gamma}_q), (\vec{x}_c, \vec{x}_q)\big) \cdot \hat{L}_{2i+1}(\vec{x}_c, \vec{x}_q, \vec{x}_\ell),$$

where $\hat{L}_{2i+1}(\vec{x}_c, \vec{x}_s) := \tilde{\mathsf{eq}}(\vec{x}_\ell, \vec{\gamma}_\ell) \cdot L_{2i+1}(\vec{x}_c, \vec{x}_q, \vec{x}_\ell)$ encodes the precomputed, $\tilde{\mathsf{eq}}$-weighted packed values (with $\vec{x}_s = (\vec{x}_q, \vec{x}_\ell)$).

Note that $\tilde{\mathsf{lin}}_{2i}((\vec{\gamma}_c, \vec{\gamma}_q), (\vec{x}_c, \vec{x}_q))$ does *not* depend on $\vec{x}_\ell$: it is constant across the B intra-chunk slots. Therefore, the scalar coefficient acts as a slot-varying plaintext (varying across the N/B chunk positions, but constant within each chunk of B slots).

For each round, the prover computes:

$$\mathsf{ect}[R_j(X)] = \mathsf{FOLD}\left(\sum_{\vec{w} \in \{0,1\}^{s'-j-1}} \mathsf{ept}_w \cdot \mathsf{ect}[\hat{L}_{2i+1}((\vec{r}, X, \vec{w}))]\right)$$

where ept_w encodes the summed linear coefficients $\tilde{\mathsf{lin}}_{2i}((\vec{\gamma}_c, \vec{\gamma}_q), (\vec{r}, X, \vec{w}, \vec{x}_q))$ across slots.

Case 1: $0 \le j < s'$ (packed ciphertext index rounds).

Step 1 (MLE partial evaluations): For each $\vec{w} \in \{0,1\}^{s'-j-1}$, compute $\mathsf{ect}[\hat{L}_{2i+1}((\vec{r}, X, \vec{w}))]$ for $X \in \{0, 2\}$. By Lemma E.2, this costs $3 \cdot 2^j$ promotions per $\vec{w}$. There are $2^{s'-j-1}$ values of $\vec{w}$, giving $3 \cdot 2^{s'-1} = 3S'/2$ promotions at level $1.5 \to 1.0$.

Step 2 (PT-CT multiplication and sum): For each $\vec{w}$ and each $X \in \{0, 2\}$, multiply $\mathsf{ept}_w \cdot \mathsf{ect}[\hat{L}_{2i+1}(\cdots)]$ (level $1.0 \to 0.0$), then sum over $\vec{w}$. Total per round: $2^{s'-j}$ $\mathsf{ept} \cdot \mathsf{ect}$ multiplications and $2^{s'-j}$ additions at level 0.0.

Step 3 (Fold): For each $X \in \{0, 2\}$: one FOLD operation. Total: 2 FOLD operations per round.

Case 2: $s' \le j < s' + \log N$ **(slot index rounds).** Before entering this phase, the prover checkpoints $\mathsf{ect}[\hat{L}_{2i+1}(\vec{r}_c)]$, where $\vec{r}_c \in \mathbb{F}_{q^R}^{s'}$ are the first s' sumcheck challenges. This costs 0.5 levels from the packed wire ciphertexts (via S' promotions), producing one ciphertext at level 1.0.

For each of the remaining $\log N$ rounds, the prover computes:

$$\mathsf{ect}[R_j(X)] = \mathsf{FOLD}\left(\mathsf{ept} \cdot \mathsf{ect}[\hat{L}_{2i+1}(\vec{r}_c)]\right)$$

where ept encodes the combined linear and slot-indexing coefficients.

Per round: 2 $\mathsf{ept} \cdot \mathsf{ect}$ multiplications (for $X \in \{0, 2\}$, level $1.0 \to 0.0$) and 2 FOLD operations.

Totaling operation counts for one affine layer reduction. Let $S' := S'_{2i+1}$ and $s' := s'_{2i+1}$. Summing over all $s' + \log N$ rounds:

- **Promotions ($\mathsf{ct} \cdot \mathbb{F}_{q^R} \to \mathsf{ect}$) @ Lvl** $1.5 \to 1.0$**:** Case 1 only. $\sum_{j=0}^{s'-1} 3S'/2 = 3s'S'/2$.
- $\mathsf{ept} \cdot \mathsf{ect}$ **Multiplications @ Lvl** $1.0 \to 0.0$**:**
  - Case 1: $\sum_{j=0}^{s'-1} 2^{s'-j} \le 2S'$.
  - Case 2: $2 \log N$.

  Total: $2S' + 2 \log N$.
- **Additions ($\mathsf{ect} + \mathsf{ect}$) @ Lvl** 0.0**:** Case 1 only: $\sum_{j=0}^{s'-1} 2^{s'-j} \le 2S'$. Total: $2S'$.
- FOLD **Operations (no depth):** $2(s' + \log N)$.

E.3.5 Total Operation Counts

We now tally the operation counts across all layer reductions. We use the packed circuit parameters from Section 4.4.3: $m' = \sum_{i=1}^{d_\times} S'_{2i-1}$ packed multiplication gates, $p' = \sum_{i=0}^{d_\times} S'_{2i}$ packed affine gates, $p' \le 2m' + 1$, and $s'_{\max} = \max_i\{s'_i\}$. For cleanliness, we bound the packed odd-layer count $\sum_{i=0}^{d_\times} S'_{2i+1} \le m' + S'_d \le m' + O(1)$ by m' (ignoring the packed input layer size S'_d, which is negligible relative to m').

Multiplication layer totals. Across all $d_\times$ multiplication layers ($s' = s'_{2i-1}$, $S' = S'_{2i-1}$):

- Promotions $(2.5 \to 2.0)$: $\sum_{i=1}^{d_\times} 3s'_{2i-1}S'_{2i-1} \le 3m' \cdot s'_{\max}$.
- CT-CT $(2.0 \to 1.0)$: $\sum_{i=1}^{d_\times}(2S'_{2i-1} + 2\log N) = 2m' + 2d_\times \log N$.
- $\mathsf{ept} \cdot \mathsf{ect}$ $(1.0 \to 0.0)$: $\sum_{i=1}^{d_\times} 2(s'_{2i-1} + \log N) = 2\sum s'_{2i-1} + 2d_\times \log N$.
- Additions (1.0): $\sum_{i=1}^{d_\times} 2S'_{2i-1} = 2m'$.
- FOLD: $\sum_{i=1}^{d_\times} 2(s'_{2i-1} + \log N) = 2\sum s'_{2i-1} + 2d_\times \log N$.

Affine layer totals. Across all $d_\times + 1$ affine layers ($s' = s'_{2i+1}$, $S' = S'_{2i+1}$):

- Promotions $(1.5 \to 1.0)$: $\sum_{i=0}^{d_\times} 3s'_{2i+1}S'_{2i+1}/2 \le 3m' \cdot s'_{\max}/2$.
- $\mathsf{ept} \cdot \mathsf{ect}$ $(1.0 \to 0.0)$: $\sum_{i=0}^{d_\times}(2S'_{2i+1} + 2\log N) = 2m' + 2(d_\times + 1)\log N$.
- Additions (0.0): $\sum_{i=0}^{d_\times} 2S'_{2i+1} = 2m'$.

- FOLD: $\sum_{i=0}^{d_\times} 2(s'_{2i+1} + \log N) = 2\sum s'_{2i+1} + 2(d_\times + 1)\log N$.

Combined totals. Writing $d = 2d_\times + 1$ for the total number of layers, and noting that the ept·ect and FOLD counts from the $\sum s'_i$ terms are bounded by $d \cdot s'_{\max}$:

Total operation counts (omitting negligible lower-order additive terms in d and $\log N$) are presented in Table 6.

Multiplicative depth.

Lemma E.13. *Let $d_\times$ be the multiplicative depth of the payload circuit. The multiplicative depth required to compute the payload circuit and generate a proof (using the laned BhIP) is at most $d_\times + 2.5$, implying an additive multiplicative depth overhead of* **2.5** *levels.*

Proof. By Claim E.12, each affine layer reduction requires 1.5 levels. The inputs to these reductions have consumed at most $d_\times + 1$ levels: $d_\times$ from the payload computation and 1 from witness packing. Therefore, affine layer reductions require at most $d_\times + 1 + 1.5 = d_\times + 2.5$ levels.

By Claim E.11, each multiplication layer reduction requires 2.5 levels. The inputs to these reductions are the packed even-layer ciphertexts that have consumed at most $d_\times - 1 + 1 = d_\times$ levels: $d_\times - 1$ from payload computation (since the deepest even layer feeding a multiplication reduction is at depth $d_\times - 1$) and 1 from packing. Therefore, multiplication layer reductions require at most $d_\times + 2.5$ levels.

The overall multiplicative depth is $\max(d_\times + 2.5, d_\times + 2.5) = d_\times + 2.5$. □